Joint Density Construction for Multi-Currency FX Derivatives Compared to a Multi-Heston FX Stochastic Model

by

Lewis Wilcher

Department of Mathematics
King's College London
The Strand, London WC2R 2LS
United Kingdom

Email: k24097977@kcl.ac.uk

29 August 2025

Report submitted in partial fulfilment of the requirements for the degree of MSc in Financial Mathematics in the University of London

Abstract

We construct a joint risk-neutral density for the two underlying rates of a currency triangle that is consistent, by construction, with the observed European option smiles on all three pairs. Following Guyon's [18] dispersion-constrained approach, we recover each marginal from its smile through Gatheral's [11] SVI fit and the Breeden-Litzenberger [10] formula, then couple them with an entropic optimal transport scheme. This scheme is solved by a Sinkhorn iteration, and the resulting minimum-relative-entropy coupling is the joint law closest to independence that still matches every smile. We use it to price basket, quanto, best-of and quadratic options across two triangles - EUR/USD/GBP and AUD/USD/JPY - and five market regimes spanning the 2008 crisis, the 2022 mini-budget, and calmer periods.

We benchmark this entropic method against a multi-factor Heston model of the same triangles, calibrated both by a full joint characteristic-function fit and by a short-maturity shortcut. Two findings stand out. First, the entropic scheme is markedly the more market-consistent of the two, recovering the smiles to a few basis points where the Heston fit is an order of magnitude worse, and its weakest leg still matches the parametric model's typical one. Second, and despite this, the two methods disagree on the exotic prices, and we trace that disagreement to its true source. They agree on the linear dependence they encode - the implied correlations coincide to within Monte-Carlo noise - but differ systematically in the tails. Comparing the recovered cross-rate densities directly, the Heston coupling carries a higher excess kurtosis in every regime and tenor tested. The entropic coupling, being the joint closest to independence, produces the thinnest-tailed cross the smiles allow, and therefore under-states cross-rate tail risk compared to the Heston model which could be seen to overstate tail risk. The price gaps are thus a higher-moment effect, not a correlation one, and they are largest for the payoffs most sensitive to that tail shape.

We conclude that fit quality, price agreement and dynamic usability are three distinct axes on which no single method dominates. The entropic scheme delivers the best static, market-consistent fit but yields only a terminal coupling, with no dynamics to simulate or hedge. The Heston model is worse-fitting but genuinely simulatable. Neither is truly assumption-light. Each is simply light on different assumptions.

Contents

1 Introduction and Literature Review **5**

1.1 FX options pre-smile: Garman-Kohlhagen 5

1.2 Modelling a volatility smile 6

1.3 Model-free density recovery 8

1.4 The multi-asset problem 10

1.5 Multi-Currency Stochastic Volatility Model 11

1.6 Entropic optimal transport 12

2 Methodology: Joint Density Construction and Pricing **14**

2.1 Task 1: SVI calibration and risk-neutral density recovery . . . 15

2.1.1 SVI parametrisation and objective function 15

2.1.2 Calibration results 16

2.1.3 Breeden-Litzenberger density recovery 17

2.2 Task 2: Sinkhorn joint density construction 20

2.2.1 Constraints and the exponential tilt 20

2.2.2 Deriving the Sinkhorn fixed-point scheme 21

2.2.3 Numerical implementation 22

2.3 Task 3: Potential Functions and Pricing exotic FX derivatives 24

3 Methodology: Multi-Currency Stochastic Volatility **27**

3.1 The Multi-Heston Model 27

3.2 Single-maturity calibration via short-maturity expansion (MH Option 1) 29
3.3 Multi-tenor calibration via the characteristic function (MH Option 2) 32

4 Results and Discussion **37**
4.1 Discussion 37
4.1.1 Two opposite treatments of dependence 38
4.1.2 Direct evidence: the cross-density tails 39
4.1.3 Corroboration: correlation and the quanto gap 40
4.1.4 The cross-rate: each method fails, but differently . . . 42
4.1.5 Fit, agreement and usability do not coincide 42
4.1.6 Neither method is assumption-light 43
4.2 Conclusion 44

AI Usage Declaration **48**

5 Appendix **50**
.1 EUR/USD/GBP Data - 16th March 2024 50
.2 Joint Density Pricing - Model vs Market Implied Volatility Tables 51
.3 EUR/USD/GBP Data - 22nd July 2026 52
.4 Short Maturity Expansion Sign Error Comparison 54
.5 Multi Heston Option 1 - Calibrated Parameter Values 55
.6 Multi Heston Option 1 - Market vs Model Implied Volatility Curves 56
.7 Multi Heston Option 2 - Market vs Model Implied Volatility Curves 57
.8 Historical Datasets 58
.9 Exotic Option Prices Across Historical Scenarios 63

A Code: Task 1-3 Sinkhorn Method Implementation **68**

A.1 SVI Calibration 68

A.2 SVI Fit Plots 72

A.3 Breeden-Litzenberger Density Recovery 72

A.4 Normalised Density Plots 74

A.5 Sinkhorn Joint Density Construction 75

A.6 Consistency Check Against Market Smiles 78

A.7 Potential Functions and Exotic Option Pricing 80

B Sinkhorn Method Applied to Multi-Tenors **82**

B.1 Data Setup 82

B.2 Calibration 85

B.3 Density Plots 88

B.4 Normalised Density Plots 89

B.5 Sinkhorn 91

B.6 Market Consistency Check 94

B.7 Sinkhorn Potential Functions, Pricing and Covariance Table . 96

C Code: Multi-Heston Option 1: Short-Maturity Expansion **99**

C.1 Data 99

C.2 Parameterisation via Least-Squares and Pricing using Monte-Carlo Simulation 101

C.3 Calibration Fit Plots : Prop2 vs Market Data 106

C.4 Market Vs Model (Using Characteristic Function) 107

C.5 RMS Error Market vs Model 109

D Code: Multi-Heston Option 2: Multi-Tenor Calibration via characteristic function **112**

D.1 Data 112

D.2 Calibration via Characteristic Function 114
D.3 Market vs Model Implied Volatility 120
D.4 Parameter Table and RMS by Tenor 121
D.5 RMS Breakdown 122

E Code Add-Ons **124**

E.1 Orientation Residual from inverted correlation - Option 1 . . . 124
E.2 Orientation Residual from inverted correlation - Option 2 . . . 125
E.3 Cross-Rate Normalised Density Comparison 126
E.4 Residual Code for Sinkhorn (AUD/USD/JPY Triangle) 129

Chapter 1

Introduction and Literature Review

1.1 FX options pre-smile: Garman-Kohlhagen

In December 1982 the Philadelphia Stock Exchange (PHLX) introduced the first exchange-traded FX (Foreign Exchange) option, following the collapse of fixed FX rates in the early 1970s and the shift to floating FX rates that unraveled over the subsequent years. This created the first genuine need for FX derivatives pricing, specifically for European call and put options. The Garman-Kohlhagen [1] paper was the first analytical solution proposed for pricing an FX European option. The model follows the same geometric Brownian motion assumption for the FX spot as the original Black-Scholes [2] framework - $\frac{dS_t}{S_t} = rdt + \sigma dW_t$, with the key development being the presence of two risk-free rates - domestic and foreign - in the drift term rather than one. The drift coefficient can be justified directly from a no-arbitrage argument. To rule out any risk-free profit, converting domestic currency into foreign currency, letting it grow at the foreign risk-free rate r_f, and converting back to domestic currency at maturity must yield the same expected return as simply letting the domestic currency grow at the domestic risk-free rate r_d. Enforcing this equality under the risk-neutral measure Q gives

$$\mathbb{E}^Q[S_T] = S_0\, e^{(r_d - r_f)T}, \tag{1.1}$$

so the exchange rate must have risk-neutral drift $r_d - r_f$ rather than r_d. Garman & Kohlhagen's own derivation formalises this same intuition using a continuous-time no-arbitrage argument that the risk-adjusted excess return

- the Sharpe ratio $(\mu_i - r)/\sigma_i$ – must be identical across all traded securities in an arbitrage-free scenario, where this common ratio is the market price of risk, λ. As with the original Black-Scholes model, the Garman-Kohlhagen framework assumes constant volatility - everything in the model is deterministic other than the brownian factor. Consequently, the model cannot reproduce the volatility smile observed empirically in FX markets, nor price options whose payoff depends jointly on more than one exchange rate - limitations that motivate the developments discussed in the remainder of this section.

1.2 Modelling a volatility smile

The assumption of constant volatility underlying both Black-Scholes [2] and Garman-Kohlhagen [1] implies that implied volatility, when inverted from market option prices, should be flat across all strikes and maturities. Empirically this is not observed, implied volatility instead varies systematically with strike, producing the volatility smile. This departure from constant volatility was first documented in equity index markets, becoming particularly pronounced following the 1987 crash, well before FX options had matured as a liquid, exchange-traded market. Therefore, by the time FX volatility surfaces were a problem to be solved, a substantial body of literature modelling the smile already existed. The FX options literature was largely able to adapt models originally developed for equities, allowing FX-specific extensions to follow comparatively quickly.

Within the setting of solving the implied volatility smile in equity derivatives, Merton [3] proposed combining geometric Brownian motion with a compound Poisson jump process, allowing the underlying to experience sudden discontinuous moves in addition to continuous diffusion. The resulting terminal distribution exhibits heavier tails and skew due to the size and direction of the Poisson jumps, generating a smile without requiring volatility itself to be random. Merton's poisson jump addition was used by Bates [4] in an FX setting (based on PHLX traded products), combined with Heston [5] type volatility modelling - which will be introduced next. However Bates found no empirical reason for poisson jumps within stochastic modelling over his 1984-1991 DM/USD options sample.

Heston's approach (without poisson jumps) involves a model in which the volatility governing the spot dynamics is not a fixed constant, but is itself a separate stochastic process with its own dynamics, mean-reverting

and correlated with the spot process. Heston [5] models the spot price S_t and its instantaneous variance v_t as the coupled system of stochastic differential equations

$$dS_t = \mu S_t\,dt + \sqrt{v_t}\,S_t\,dW_t^S, \tag{1.2}$$
$$dv_t = \kappa(\theta - v_t)\,dt + \xi\sqrt{v_t}\,dW_t^v, \tag{1.3}$$

where W_t^S and W_t^v are standard Brownian motions with correlation $dW_t^S\,dW_t^v = \rho\,dt$. Here κ is the rate of mean reversion of the variance process, θ is the long-run variance level, and ξ is the volatility of variance (often referred to as the "vol-of-vol"). The variance process v_t is a Cox-Ingersoll-Ross (CIR)-type square-root diffusion, chosen so that v_t remains non-negative almost surely, and strictly positive provided the Feller condition $2\kappa\theta \geq \xi^2$ is satisfied. If this condition fails, the process can reach zero variance, a boundary behaviour that is both financially questionable and a recurring source of numerical difficulty in CIR-based models, including the multi-currency extension used later in this dissertation.

Crucially, Heston's original paper derives this closed-form solution not only for equity-style options, but explicitly extends the framework to bond options and foreign currency options, incorporating stochastic interest rates directly into the same characteristic-function machinery. This makes the model directly applicable to the FX setting without modification to the underlying solution technique. However, Heston's model remains a single-asset model, it describes the dynamics of one exchange rate in isolation, and says nothing about how a second, correlated exchange rate should be modelled jointly, still less about the triangulation constraint linking two dollar-denominated rates to their own cross-rate. It nonetheless provides the natural parametric foundation for the multi-currency stochastic volatility model of De Col, Gnoatto & Grasselli [6] used later in this dissertation, which extends exactly this single-currency structure to a genuinely joint, multi-currency setting.

A third approach, local volatility, was introduced by Dupire [7], who showed that volatility could instead be modelled as a deterministic function of the underlying spot level and time, $\sigma_{loc}(S_t, t)$, chosen so that the model reproduces the entire observed implied volatility surface exactly at a single point in time. Dupire's original derivation was motivated by equity index smiles, but is still directly applicable to the FX setting once the Garman-Kohlhagen drift $r_d - r_f$ is substituted for the single risk-free rate. Wystup [8] and Clark [9] separately address the practical work this requires, converting the delta-quoted smiles standard in FX markets into a continuous,

arbitrage-free strike-based surface Dupire's formula can be applied to. While local volatility models are therefore exactly calibrated to market prices by construction, the resulting dynamics are widely considered less realistic than stochastic volatility, since local volatility models tend to flatten the forward smile too quickly relative to what is observed in practice - a key reason stochastic volatility models such as Heston's remain the preferred parametric benchmark for exotic option pricing.

A common thread runs through all three approaches described above is that each commits to a specific parametric structure for the underlying process. From the models mentioned prior a jump distribution, a mean-reverting CIR-type variance process, or a deterministic local volatility surface, this all means that the resulting fit is only ever as good as that structural assumption. In particular, the CIR-type variance dynamics underlying Heston's model impose mean reversion in volatility as a modelling choice, rather than a property inferred directly from the market. More recent literature has increasingly turned to a model-free perspective, attempting to recover a distribution from purely market data, this is examined next.

1.3 Model-free density recovery

An alternative response to the failure of constant-volatility pricing is to abandon the search for a single underlying process altogether, and instead ask whether the risk-neutral distribution of an asset can be recovered directly from observed option prices, without imposing any dynamics assumption at all.

The risk-neutral pricing framework demonstrates that the value of a European option is determined by the risk-neutral distribution of the underlying asset at maturity, this naturally raises the question of whether the relationship can be inverted. This problem was addressed by Breeden & Litzenberger [10], who showed that under suitable regularity conditions, the risk-neutral density can be extracted from the second derivative of the European call price with respect to strike. For a European call option, the payoff depends on the terminal value of the underlying asset, and therefore the option price contains information about the risk-neutral distribution of S_T. Specifically, the call price can be written as

$$C(K) = e^{-rT} \int_K^\infty (s - K)\mu(s)\, ds, \tag{1.4}$$

where s is a dummy variable representing possible values of the terminal

asset price, and $\mu(s)$ is the associated risk-neutral density. Since the payoff is zero whenever $s < K$, the integral is taken only over values exceeding the strike price.

The relationship formalised by Breeden & Litzenberger [10] is

$$\mu(K) = \frac{\partial^2 C(K)}{\partial K^2} \tag{1.5}$$

assuming zero interest rates, consistent within this dissertation.

The Breeden-Litzenberger result establishes a one-to-one relationship between the European call price surface and the corresponding risk-neutral density. Therefore, if call prices are known for a continuum of strikes $K \in (0, \infty)$ at a fixed maturity, and the call price function is twice differentiable with respect to strike, $C \in C^{1,2}$, the entire risk-neutral distribution of the underlying asset can be recovered without assuming a particular stochastic process for the asset price. This represents one of the fundamental links between observed market prices and implied probability distributions in quantitative finance. Despite the theoretical elegance of the B-L formula, it is not simple to implement due to noisy, discrete market data which will only become amplified by taking a second derivative. There is a clear need for a smooth, arbitrage-free call option surface.

This motivated literature within SVI parameterisation by Gatheral in 2004 [11], which models the total implied variance $w(k) = \sigma_{imp}^2 T$ directly as a function of log-moneyness $k = \log(K/F_0)$, where F_0 is the forward price and $w(k)$ is:

$$w(k) = a + b\left(\rho(k-m) + \sqrt{(k-m)^2 + \sigma^2}\right). \tag{1.6}$$

The parameters a, b, ρ, m and σ control the overall level, slope, skew, horizontal displacement and curvature of the implied volatility smile respectively. SVI is attractive as, once again, it follows this assumption free modeling technique, and instead focuses directly on producing a smooth, arbitrage-free description of the smile actually observed in the market - one that can subsequently be differentiated safely to recover a risk-neutral density.

An arbitrarily-fitted SVI curve can itself admit butterfly arbitrage, in which case the recovered density becomes negative over some range of strikes - an impossibility for a genuine probability measure. Equivalently, the call price must remain convex with respect to strike, $\partial^2 C(K)/\partial K^2 \geq 0$. Financially, a violation of this condition corresponds to the existence of butterfly arbitrage, whereby a portfolio of long positions in two call options and a

short position in two appropriately weighted call options at intermediate strikes could be constructed to produce a non-negative payoff at maturity while having a negative initial cost - an opportunity inconsistent with an arbitrage-free market, and one that must therefore be ruled out when calibrating the implied volatility smile.

Gatheral & Jacquier [12] derive explicit necessary and sufficient conditions under which the raw SVI parametrisation is free of both butterfly and calendar-spread arbitrage (total variance decreasing with maturity). The restriction most directly relevant to the present work is on the asymptotic wing slope of the total implied variance,

$$b(1 + |\rho|) \leq \frac{4}{T}, \tag{1.7}$$

which follows from Lee's moment formula [13], relating the steepness of the implied volatility smile at extreme strikes to the existence of moments of the underlying risk-neutral distribution. For raw SVI, the left wing has slope $b(1 - \rho)$ and the right wing has slope $b(1 + \rho)$. Since ρ lies between -1 and 1, whichever wing is steeper has slope $b(1 + |\rho|)$. Keeping this below the bound above stops the wings getting too steep, which is what prevents butterfly arbitrage. Enforcing this bound during calibration is therefore what guarantees the recovered marginal densities are valid.

1.4 The multi-asset problem

The methods discussed so far - whether parametric or model free - are concerned specifically with recovering the marginal distribution or dynamics of a single underlying asset alone. Many FX derivatives, however, have payoffs that depend jointly on two or more exchange rates simultaneously. Basket options, whose payoff depends on a weighted combination of several rates, and best-of (rainbow) options, whose payoff depends on the maximum or minimum of several rates, are standard examples requiring knowledge of the joint distribution or dynamics - both are examples of payoffs that cannot be priced from the marginals alone, since the dependence between the two assets matters too.

Multi-asset exotic payoffs of exactly this kind have a long history predating joint-density literature, however they were still priced. Margrabe [14] priced an option to exchange one correlated lognormal asset for another as early as 1978, Stulz [15] extended this to options on the maximum or minimum of two assets in 1982, and Johnson [16] generalised to n assets in 1987,

all using a single, exogenously assumed correlation parameter between the underlying lognormal processes. What is new in the literature surveyed in the following sections is not the ability to price such payoffs at all, but the requirement that the resulting joint distribution be simultaneously consistent with every observed smile across the whole currency triangle, rather than an assumed or estimated correlation alone - a considerably more demanding standard that only becomes tractable with the entropic and shared-factor constructions discussed in the following sections.

Constructing a joint distribution or dynamics consistent with all three smiles poses a technical challenge for each of the two philosophies compared in this dissertation. For a model-free approach, the difficulty is that a large number of possible joint densities $\mu(x, y)$ can satisfy the X, Y and Z marginal constraints simultaneously, marginal constraints alone do not pin down a single joint distribution. For a parametric approach, the difficulty is instead in constructing dynamics that preserve triangulation in the first place, even a seemingly reasonable choice of correlation between X and Y does not, in general, produce a cross-rate $Z = X/Y$ that follows the same type of process, so the model must be constructed carefully so that triangulation is preserved by design, rather than imposed after the fact.

1.5 Multi-Currency Stochastic Volatility Model

We will firstly look at the parametric approach looking at a model that has the property of the FX triangulation and therefore can be used to price these multi-asset FX derivatives. De Col, Gnoatto & Grasselli [6] propose exactly such a model, extending the single-currency Heston framework [5] to a multi-currency setting in which the stochastic variance of each exchange rate and the correlation structure between them, is driven by a shared set of underlying CIR variance factors, rather than by a separate volatility process for each currency pair.

The principal advantage of this approach is that, once calibrated, the model produces a fully specified joint law for all exchange rates in the triangle at any $t > 0$, not just the maturities used in calibration. On top of this, it also offers pricing of vanilla European claims on a single currency pair via the joint characteristic function and Fourier inversion, up to a single numerical quadrature. This does not extend as directly to genuinely multi-asset exotic payoffs such as those priced in this dissertation, which in general require a two-dimensional Fourier inversion or are not tractable in closed form at all.

For this reason, the exotic option prices reported later in this dissertation are instead obtained via Monte Carlo simulation of the calibrated dynamics. This is a materially different guarantee to the entropic optimal transport approach, which reproduces the calibration inputs essentially exactly but is, by construction, a single-maturity, static construction.

Unlike naive multi-asset extensions of models such as SABR, this model achieves triangulation consistency by construction, rather than by calibration. Each exchange rate is built from the same shared set of CIR variance factors, so that any cross-rate formed by dividing two exchange rates in the triangle is automatically driven by the same factors, and is therefore of the same functional form as the pairs it was constructed from.

The practical cost of the parametric approach is correspondingly higher, identifying the full set of model parameters, including the mean-reversion speed of the variance process, generally requires option data across multiple maturities rather than a single smile snapshot, a point returned to directly in the methodology and results of this dissertation. De Col, Gnoatto & Grasselli also report violations of the Feller condition and finite-time explosion of higher moments in their own calibrated parameters, both well-known features of Heston-type models that can lead to instability in numerical pricing routines, particularly at longer maturities.

1.6 Entropic optimal transport

The idea of calibrating a model to reproduce market option prices exactly, was pursued early on by Guyon & Henry-Labordère [17], who developed a particle-method technique for calibrating multi-factor local-stochastic volatility models so that they reprice the observed smile at every strike and maturity simultaneously. Their approach, based on McKean's particle method, showed that exact calibration was achievable numerically, and marked a shift in the literature away from best-fit, approximate calibration towards models that match market prices precisely by construction.

Exact calibration as a constraint however, does not choose a singular model and in most cases requires more. An additional criterion is therefore needed to select a single model from among all those that fit. Guyon [18] addressed this directly by proposing entropy minimisation as the selection principle, among all models jointly consistent with two related sets of observed prices, select the one closest, in relative entropy, to a natural reference

model. Guyon applied this idea to solve a long-standing calibration problem in equity derivatives, jointly and exactly calibrating a stochastic volatility model to both S&P 500 (SPX) smiles and VIX smiles simultaneously - two markets linked by a known relationship, since VIX is itself defined as a function of the SPX's own future implied volatility. Formally, this amounts to choosing, from the set Π of joint measures satisfying the required marginal constraints, the measure μ^* minimising the Kullback-Leibler divergence to a reference measure $\bar{\mu}$,

$$\mu^* = \arg\min_{\mu \in \Pi} D_{KL}(\mu \,\|\, \bar{\mu}), \qquad D_{KL}(\mu \| \bar{\mu}) = \int\int \mu(x,y) \log\left(\frac{\mu(x,y)}{\bar{\mu}(x,y)}\right) dx\, dy. \tag{1.8}$$

The mathematical foundations underlying this minimisation - including the existence of a solution and its characterisation as an exponential tilt of the reference measure via a pair of dual potential functions - are developed in full generality in Nutz's lecture notes [19], which also establish the convergence of the Sinkhorn algorithm used to compute the optimal potentials numerically.

Forde [20] extends this entropic framework specifically to the FX cross-smile problem considered in this dissertation. Using the independent coupling $\bar{\mu}(x,y) = \mu_X(x)\mu_Y(y)$ as the reference measure, the entropy-minimising joint density consistent with all three FX smiles takes the exponential form

$$\mu^*(x,y) = e^{u(x)+v(y)+yw(x/y)}\, \bar{\mu}(x,y), \tag{1.9}$$

where u, v and w are our potential functions enforcing the X,Y and Z marginal constraints respectively, and are solved for numerically via a Sinkhorn-type fixed-point iteration. Unlike the parametric approach of the previous section, no dynamics are assumed anywhere in this construction - μ^* says nothing about how X and Y evolve before or after the calibration maturity, only that their terminal joint law is consistent with all three observed smiles at that one maturity. This method is used as our model free pricing method in this dissertation.

The trade-off between the two approaches compared in this dissertation is therefore a direct one. The parametric model commits to a specific dynamics, at the cost of requiring richer, multi-maturity data to identify its parameters reliably. The entropic approach commits to no dynamics at all, and in exchange is limited to the single maturity it is calibrated to. This trade-off is exactly what is tested empirically in the results that follow.

Chapter 2

Methodology: Joint Density Construction and Pricing

Before describing the calibration process it is important to establish the meaning of each of the marginal densities represent in regards to probability measure. FX pairs X and Y will be under the same domestic risk-neutral measure Q and the deterministic cross-rate $Z = X/Y$ will be under different measure we will call $\tilde{Q}$.

Through equating the GBP payoffs of the following two options:

$$\frac{1}{Y_0}E^Q\big(Y(Z-K)^+\big) = E^{\tilde{Q}}\big((Z-K)^+\big) \tag{2.1}$$

we see the extra factor which acts as the Radon-Nikodym derivative between the two measures meaning we can define the new measure:

$$\tilde{Q}(A) := E^Q\left(\frac{Y}{Y_0}\mathbf{1}_A\right), \tag{2.2}$$

For $\tilde{Q}$ to be a genuine probability measure it must also satisfy $\tilde{Q}(\Omega) = E^Q(Y/Y_0) = 1$, assigning total mass 1 to the whole sample space. This holds due to Y being a Q-martingale - so $E^Q(Y) = Y_0$. This also motivates the later normalisation of our forward rates to simplify the measure change, making $Y_0 = 1$.

2.1 Task 1: SVI calibration and risk-neutral density recovery

The data used for this methodology is shown in Appendix .1, gathered from Bloomberg on the 16th March 2024 for the EUR/USD/GBP FX triangle (X/Y/Z).

As detailed previously, the first step of our model-free process is to use Gatheral's [11] SVI parameterisation to create a smooth differentiable implied volatility curves that we can use the Breeden-Litzenberger formula for. The following sections we will detail the stages of the calibration and present the resulting fits of the SVI and the resulting marginal densities for the Table .1 data.

2.1.1 SVI parametrisation and objective function

Following Gatheral [11], the total implied variance - $w(k) = \hat{\sigma}(K)^2 T$ - is modelled as a function of log-moneyness $k = \log(K/F_0)$,

$$w(k) = a + b\left(\rho(k-m) + \sqrt{(k-m)^2 + \sigma^2}\right), \tag{2.3}$$

with implied volatility recovered as $\hat{\sigma}(K) = \sqrt{w(k)/T}$. For each currency pair, the five parameters (a, b, ρ, m, σ) are calibrated by minimising the sum of squared errors between the model-implied volatility and the market quotes,

$$\text{Error}(a, b, \rho, m, \sigma) = \sum_{i=1}^{5} \left(\hat{\sigma}(K_i) - \sigma_i^{\text{mkt}}\right)^2. \tag{2.4}$$

Implied volatility was chosen over option price as the basis for the least-squares error, due to the way vega varies with strike. Since vega is largest near the at-the-money strike, a price-space objective would implicitly prioritise fitting volatility accurately there, and pay comparatively little attention to the wings of the smile.

To ensure the fitted smile is free of butterfly arbitrage, the Gatheral-Jacquier [12] wing-slope condition, following from Lee's moment formula [13],

$$b(1 + |\rho|) \leq \frac{4}{T}, \tag{2.5}$$

is enforced via a soft penalty added to the objective whenever it is violated,

$$\text{Penalty} = \max\left(0,\ b(1 + |\rho|) - \frac{4}{T}\right), \tag{2.6}$$

with the total objective given by Error + λ Penalty, where $\lambda = 10^3$. A `differential_evolution` method was used for the least-squares calibration, which is the reason a soft penalty was needed rather than a hard constraint. This method is well suited to the non-convex SVI objective surface, unlike a method such as SLSQP (Sequential Least Squares Quadratic Programming), which would allow the wing-slope condition to be enforced as a hard constraint, but being gradient-based, risks becoming caught in a local minimum rather than finding the best-fitting parameters. `differential_evolution`'s use of a population of candidate parameter sets avoids this problem, but at the cost of only ever seeing a single score for each candidate, with no separate mechanism for ruling out arbitrage-violating regions outright. This is why the choice of $\lambda = 10^3$ is key, it must dominate the least-squares error in the objective function, so that any fit which introduces arbitrage is never worth the resulting penalty.

With the calibration being performed via `differential_evolution`, it was followed by a local polish step to refine the final result. To further guard against the optimiser settling in a suboptimal basin, calibration is repeated across five different random seeds for each currency pair, with the lowest-error result retained.

The parameter bounds for m and σ are scaled to the observed log-moneyness range of each pair, $x_{\text{range}} = \max(k) - \min(k)$, rather than being fixed across all three pairs, since the three smiles span quite different relative strike ranges.

2.1.2 Calibration results

The calibrated SVI parameters for each pair, to four significant figures, are given in Table 2.1.

Table 2.1: Calibrated SVI parameters, $T = 1/12$.

	a	b	σ	ρ	m
EUR/USD	0.0001740	0.002357	0.01975	−0.1920	0.002396
GBP/USD	0.0002580	0.001314	0.01182	−0.4738	0.000132
EUR/GBP	0.0000890	0.001832	0.01414	0.09302	−0.002134

The resulting root-mean-square calibration errors are 5.37×10^{-18} (EUR/USD), 2.98×10^{-17} (GBP/USD) and 3.12×10^{-17} (EUR/GBP) – all at the level of floating-point precision, indicating an essentially exact fit to the

five market quotes on each pair. This is expected given that SVI has five free parameters and each smile consists of exactly five observed points, so the calibration is not meaningfully over-determined. Figure 2.1 shows the fitted SVI curves against the original market implied volatilities for all three pairs.

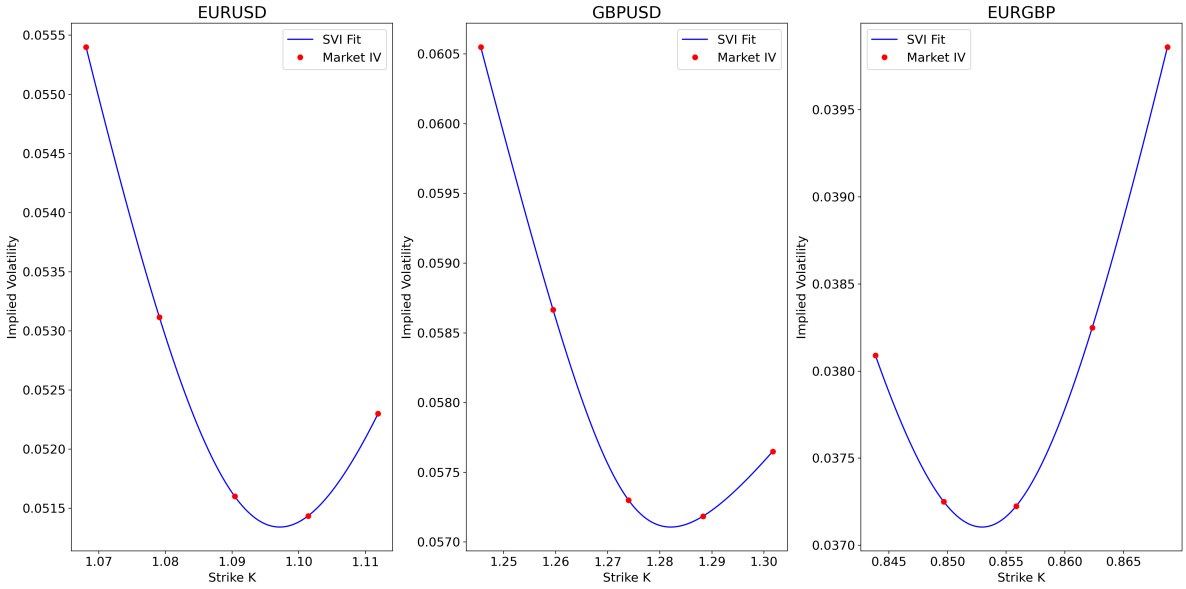


Figure 2.1: Fitted SVI implied volatility smiles (blue) against market implied volatilities (red) for EUR/USD, GBP/USD and EUR/GBP, $T = 1/12$.

2.1.3 Breeden-Litzenberger density recovery

Given the calibrated SVI curve, the corresponding Black-Scholes call price is defined at any strike K, not only the five market strikes, by substituting the SVI-implied volatility $\hat{\sigma}(K)$ into the Black-76 [21] formula,

$$C(K) = F_0 N(d_1) - K N(d_2), \tag{2.7a}$$

$$d_1 = \frac{\log(F_0/K) + \frac{1}{2}\hat{\sigma}(K)^2 T}{\hat{\sigma}(K)\sqrt{T}}, \tag{2.7b}$$

$$d_2 = d_1 - \hat{\sigma}(K)\sqrt{T}. \tag{2.7c}$$

The risk-neutral density is then recovered via the Breeden-Litzenberger formula [10], $\mu(K) = \partial^2 C(K)/\partial K^2$, computed numerically as a central second difference,

$$\mu(K) \approx \frac{C(K+h) - 2C(K) + C(K-h)}{h^2}, \qquad h = 10^{-3}. \tag{2.8}$$

This numerical approach was used in preference to differentiating the SVI-composed Black-Scholes formula analytically. As a sanity check, the central finite-difference density was compared against the full analytic second derivative. The two agreed to within a negligible relative error across the strike range considered, consistent with the expected truncation error of the finite-difference scheme, confirming that central differencing introduces no meaningful loss of accuracy relative to the exact analytic result.

As a further validation, the finite-difference density was checked directly against three required properties of a risk-neutral density - that it integrates to 1, that its mean recovers the forward price F_0, and that it is everywhere non-negative, the numerical manifestation of the absence of butterfly arbitrage. These checks are summarised in Table 2.2, the corresponding densities are plotted in Figure 2.2, and finally the normalised densities overlaid in Figure 2.3.

All three densities integrate to 1, recover their respective forwards as the mean, and are non-negative across the strike range considered, confirming that the calibrated marginals μ_X, μ_Y and μ_Z are valid probability densities and are suitable as inputs to the Sinkhorn scheme described in the following section.

Table 2.2: Density sanity checks, four significant figures.

Pair	Total Mass	Mean	Density Minimum
EUR/USD	1.0000	1.0903 ($F_0 = 1.0903$)	0.0000
GBP/USD	1.0000	1.2738 ($F_0 = 1.2738$)	0.0000
EUR/GBP	1.0000	0.8559 ($F_0 = 0.8559$)	0.0000

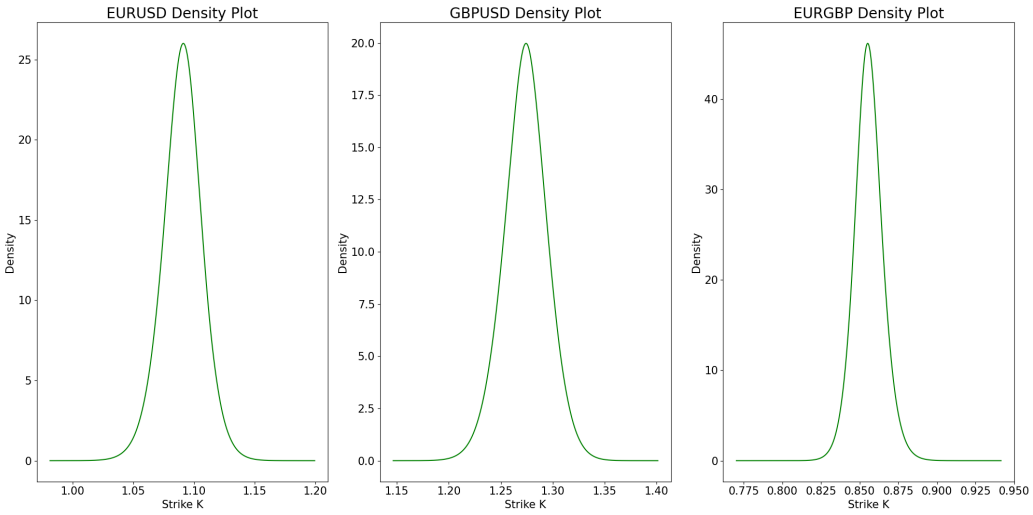


Figure 2.2: Recovered risk-neutral densities for EUR/USD, GBP/USD and EUR/GBP, via central finite-difference Breeden–Litzenberger applied to the fitted SVI call-price curve.

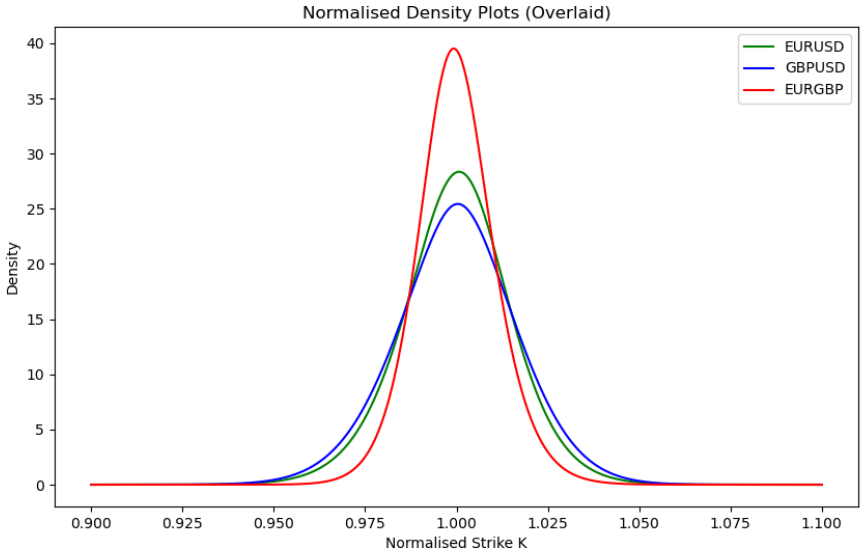


Figure 2.3: Recovered risk-neutral densities normalised to mean 1 for EUR/USD, GBP/USD and EUR/GBP.

2.2 Task 2: Sinkhorn joint density construction

Given the three normalised marginal densities μ_X, μ_Y and μ_Z recovered in Task 1, the objective is to construct a joint density $\mu^*(x, y)$ for (X, Y) under Q such that X has marginal μ_X, Y has marginal μ_Y, and the cross-rate $Z = X/Y$ has marginal μ_Z under the measure $\tilde{Q}$ derived in the previous section. Matching all three marginals simultaneously guarantees, via Breeden-Litzenberger, that μ^* is consistent with the observed option prices at all strikes on all three pairs. Since infinitely many joint distributions share the same pair of marginals μ_X, μ_Y, an additional selection principle is required to pick out a single, well-justified candidate. This is the role played by the entropy-minimisation framework introduced in Section 1.6.

2.2.1 Constraints and the exponential tilt

Section 1.6 established that minimising the Kullback-Leibler divergence to a reference density $\bar{\mu}$, subject to a set of marginal constraints, yields an optimal density that is an exponential tilt of $\bar{\mu}$, with one dual potential function per constraint. Taking $\bar{\mu}(x, y) = \mu_X(x)\mu_Y(y)$ as the reference density - the independent coupling, which already satisfies two of the three constraints automatically - and noting that three constraints must be enforced here (the X marginal, the Y marginal, and the $Z = X/Y$ marginal under $\tilde{Q}$), the optimal joint density takes the form

$$\mu^*(x, y) = e^{u(x)+v(y)+yw(x/y)}\, \mu_X(x)\mu_Y(y), \tag{2.9}$$

where $u(x)$ and $v(y)$ are the potentials associated with the two marginal constraints in the usual way, and $w(x/y)$ is the potential associated with the cross-rate constraint. The factor of y multiplying w is not an additional free choice, but a direct consequence of the change of measure derived earlier. Since $\tilde{Q}(A) = E^Q(Y\mathbf{1}_A)$ in normalised units, any constraint imposed under $\tilde{Q}$ enters the exponent of a Q-density carrying an explicit factor of Y, exactly analogous to how a Radon-Nikodym derivative reweights an expectation under a change of measure.

2.2.2 Deriving the Sinkhorn fixed-point scheme

Our fixed-point scheme equations must enforce our constraints and therefore be derived from them. We start with our constraints on X and Y where integrating $\mu^*(x, y)$ over y must recover $\mu_X(x)$, and integrating over x must recover $\mu_Y(y)$:

$$\mu_X(x) = e^{u(x)} \int_0^\infty e^{v(y)+yw(x/y)}\, \mu_Y(y)\, dy, \tag{2.10}$$

$$\mu_Y(y) = e^{v(y)} \int_0^\infty e^{u(x)+yw(x/y)}\, \mu_X(x)\, dx. \tag{2.11}$$

For our third constraint we use the price of a call on Z written as an integral over the region $X/Y > K$ weighted by Y:

$$\int_0^\infty (z-K)^+ \mu_Z(z)\, dz = \int_0^\infty \int_0^\infty y \left(\frac{x}{y} - K\right)^+ e^{u(x)+v(y)+yw(x/y)}\, \mu_X(x)\mu_Y(y)\, dy\, dx. \tag{2.12}$$

Differentiating twice with respect to K - once to remove the $(z-K)^+$ payoff structure via the Leibniz rule, and once more to isolate the density itself, exactly as in the Breeden-Litzenberger derivation of Task 1 - and relabelling K as z, gives us:

$$\int_0^\infty e^{u(x)+v(x/z)+\frac{x}{z}w(z)}\, \frac{x^2}{z^3}\, \mu_X(x)\mu_Y(x/z)\, dx = \mu_Z(z). \tag{2.13}$$

Note that this cannot be rearranged explicitly for $w(z)$.

From equations 2.10, 2.11 and 2.13 we find our iterative equations:

$$u^{n+1}(x) = -\log \int_0^\infty e^{v^n(y)+yw^n(x/y)}\, \mu_Y(y)\, dy, \tag{2.14}$$

$$v^{n+1}(y) = -\log \int_0^\infty e^{u^{n+1}(x)+yw^n(x/y)}\, \mu_X(x)\, dx, \tag{2.15}$$

$$\int_0^\infty e^{u^{n+1}(x)+v^{n+1}(x/z)+\frac{x}{z}w^{n+1}(z)}\, \frac{x^2}{z^3}\, \mu_X(x)\mu_Y(x/z)\, dx \quad = \quad \mu_Z(z), \tag{2.16}$$

initialised at $u^0 \equiv v^0 \equiv w^0 \equiv 0$, with each new potential used immediately in the update that follows it within the same iteration. Our third equation presents a root-finding problem to determine the unknown $w^{n+1}(z)$. This scheme is implemented numerically as described below.

2.2.3 Numerical implementation

Quadrature domain and truncation

We evaluate the X and Y marginals on $[0.9, 1.1]$, discretised using $N = 800$ Gauss-Legendre nodes and weights. This domain balances two competing needs. It must be wide enough to capture the full probability mass of each marginal - for the 1-month maturity considered here, $\pm 10\%$ around the forward is sufficient, though a longer maturity would need a wider domain, since the density develops heavier tails as maturity grows. But the domain also cannot be too wide, because w is evaluated over the ratio x/y and this ratio's own range grows much faster than x and y's. Widening the bounds from $[0.9, 1.1]$ to $[0.8, 1.2]$ pushes the ratio's range from roughly [0.9/1.1, 1.1/0.9] out to [0.8/1.2, 1.2/0.8], a substantially wider spread. Since w is usually solved for on the same domain, $[0.9, 1.1]$ in this case, a wider X, Y domain pushes more of Z's mass outside the region where w can actually be determined.

As stated previously the Z marginal is evaluated on the same domain, $[0.9, 1.1]$, but discretised using $N_z = 100$ uniformly spaced points rather than Gauss-Legendre nodes. This is because z is never itself an integration variable, each z-node is instead the location of an independent root-find for w.

Iteration and spline interpolation

The fixed-point scheme above is iterated 40 times. Since w is only known at the $N_z = 100$ grid points, but must be evaluated at arbitrary ratios x/y inside the double sums for u and v, a cubic spline is fitted through the current w values at each iteration, with query points clipped to the spline's domain to prevent unreliable extrapolation.

Root-finding for w

For each fixed z, writing $\text{target}(w) = \int_0^\infty e^{u(x)+v(x/z)+\frac{x}{z}w} \frac{x^2}{z^3} \mu_X(x)\mu_Y(x/z)\, dx - \mu_Z(z)$, equation (2.16) is solved using bisection. Since the integrand is increasing in w, target is monotonic and admits at most one root, making bisection an appropriate and robust choice given no closed-form derivative is available. The bracket $[-60, 60]$ balances two numerical risks: too narrow a bracket risks missing a genuine root, while too wide a bracket risks numerical overflow in $\exp(\cdot)$. Since w enters directly in the exponent, $\exp(60) \approx 10^{26}$ remains comfortably within double-precision floating-point range. If the target function does not change sign across the bracket, w is instead pinned

at whichever endpoint is closer to the true root, rather than allowing the bisection routine to fail. This fallback was not found to be triggered systematically once the domain choices above were adopted, but is retained as a safeguard.

Joint density recovery

Once the potentials have converged, the joint density is evaluated directly on the $N \times N$ grid of (x, y) nodes,

$$\mu^*(x_i, y_j) = e^{u(x_i)+v(y_j)+y_j w(x_i/y_j)}\, \mu_X(x_i)\mu_Y(y_j), \tag{2.17}$$

and its total mass is checked via quadrature, $\sum_{i,j} \mu^*(x_i, y_j)\, \omega_i^x\, \omega_j^y$, confirming the density integrates to 1 as required.

Consistency check against market smiles

As a final validation of the whole construction, the joint density is used to price a European call at an array of strikes on each pair, and the resulting price is inverted back to an implied volatility for direct comparison with the market quotes as well as the overall fit. Calls on X and Y are priced directly, $E[(X-K)^+]$ and $E[(Y-K)^+]$, by summing the payoff against μ^* under the quadrature weights, the cross-rate call is priced under $\tilde{Q}$, $E[Y(X/Y-K)^+]$, consistent with the derivation in the previous section. In each case, the resulting model price is inverted to an implied volatility using `scipy.optimize.brentq` applied to the Black-Scholes formula, providing a like-for-like comparison with the market-quoted volatilities rather than comparing prices directly.

The model-implied volatilities agree with the market quotes to within approximately 0.02-0.04 vol points across all fifteen strikes considered - as shown in Appendix .2, and the fitted curves in Figure 2.4 closely track the market points across the full range plotted. This confirms that the joint density μ^* constructed via the Sinkhorn scheme is consistent not only with the two marginal constraints on X and Y by which it was directly parametrised, but also with the cross-rate constraint on $Z = X/Y$ enforced only implicitly through the potential w, and validates the joint density as a suitable basis for the exotic option pricing carried out in Task 3.

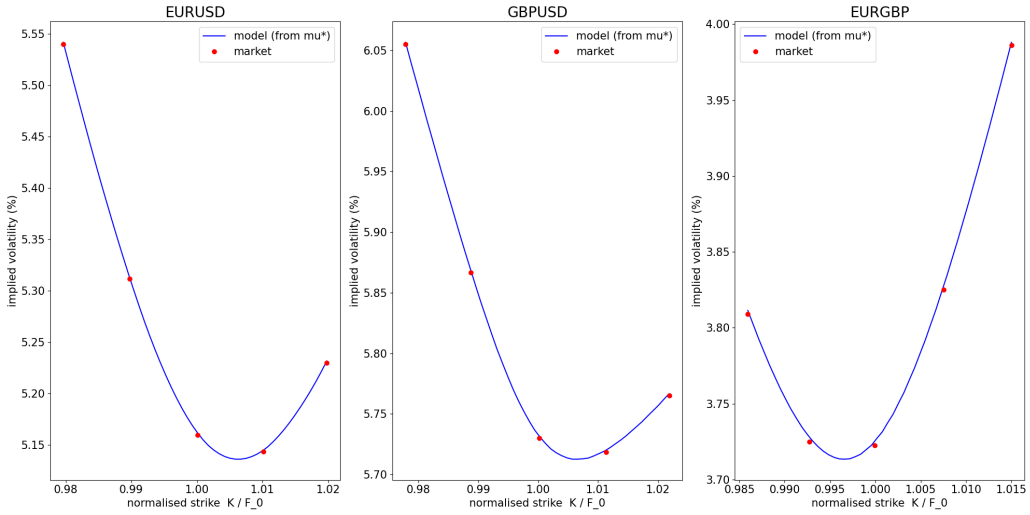


Figure 2.4: Model-implied volatility, recovered from μ^*, against market-quoted implied volatility, for EUR/USD, GBP/USD and EUR/GBP.

2.3 Task 3: Potential Functions and Pricing exotic FX derivatives

The potential functions

Figure 2.5 plots the three converged potential functions $u(x)$, $v(y)$ and $w(z)$ over their respective quadrature domains. These are the dual potentials associated with the marginal constraints on X and Y, and the cross-rate constraint on $Z = X/Y$ respectively, and together they fully determine the joint density μ^* via the exponential ansatz.

Pricing formula

With the joint density μ^* constructed and validated in Task 2, the price of any payoff $c(X, Y)$ is obtained directly by numerical integration against μ^* over the same quadrature grid used throughout the Sinkhorn scheme,

$$E\big[c(X,Y)\big] = \sum_{i=1}^{N}\sum_{j=1}^{N} c(x_i, y_j)\,\mu^*(x_i, y_j)\,\omega_i^x\,\omega_j^y, \tag{2.18}$$

with interest rates assumed zero throughout, as specified. Unlike the parametric approach of the following chapter, no additional simulation step is required. This is because μ^* is already a fully specified joint density on a

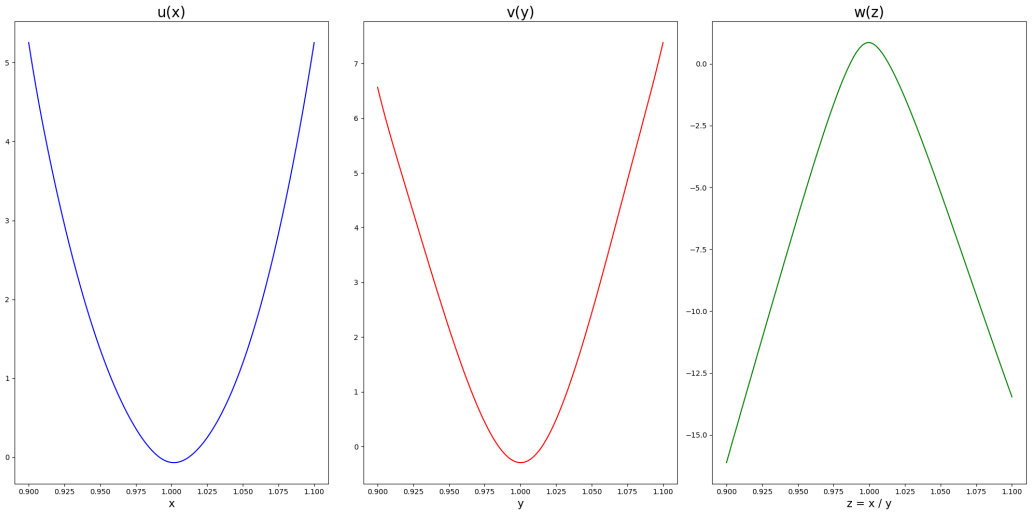


Figure 2.5: Converged potential functions $u(x)$, $v(y)$ and $w(z)$ from the Sinkhorn iteration.

fixed quadrature grid, any payoff can be priced by a single weighted sum over the same $N \times N$ grid used to construct μ^* itself.

Four payoffs are considered: a basket option, a quanto call, a best-of (rainbow) option, and a quadratic option,

$$\text{basket} : \left(\tfrac{1}{2}(X+Y) - 1\right)^+, \tag{2.19}$$

$$\text{quanto} : \left(\tfrac{X}{Y} - 1\right)^+, \tag{2.20}$$

$$\text{best-of} : \left(\max(X,Y) - 1\right)^+, \tag{2.21}$$

$$\text{quadratic} : (X-Y)^2. \tag{2.22}$$

Each payoff is evaluated on the full (x_i, y_j) grid and weighted by the corresponding joint density and quadrature weights, exactly as in the mass and consistency checks of Task 2.

Results

The resulting prices are given in Table 2.3.

As a sanity check on these results, note that $\max(x,y) \geq \frac{1}{2}(x+y)$ for all x, y - the larger of two numbers is always at least their average. Since $t \mapsto t^+$ only ever increases as t increases, this inequality still holds once we take the positive part of each side: $(\max(X,Y)-1)^+ \geq (\frac{1}{2}(X+Y)-1)^+$ everywhere. Taking expectations, the best-of price must therefore be at least as large as the basket price. This is confirmed by the results in Table 2.3, with the

Table 2.3: Exotic option prices computed from μ^*, zero interest rates.

Payoff	Price
Basket, $E[(\frac{1}{2}(X+Y)-1)^+]$	0.005888
Quanto, $E[(X/Y-1)^+]$	0.004336
Best-of, $E[(\max(X,Y)-1)^+]$	0.008435
Quadratic, $E[(X-Y)^2]$	1.220486×10^{-4}

best-of price of 0.008435 exceeding the basket price of 0.005888, providing a model-independent check that the priced payoffs and the underlying joint density are consistent with one another.

The quadratic price, $E[(X-Y)^2]$, is substantially smaller in magnitude than the other three payoffs, reflecting the fact that X and Y are highly correlated normalised FX rates both centred near 1, so their difference is typically small. This payoff is provides a further test of the joint density's dependence structure, since it is the only one of the four payoffs considered that is non-monotonic in $X-Y$ and therefore sensitive to the full shape of μ^* rather than only its tail behaviour.

Chapter 3

Methodology: Multi-Currency Stochastic Volatility

Section 1.5 established triangulation symmetry as one of the two structural properties a consistent multi-currency FX model must satisfy, and noted that the De Col, Gnoatto & Grasselli [6] model is constructed specifically to satisfy it by design. A second, related property is also required: inversion symmetry. The reciprocal of an exchange rate, $1/S^{i,l}$, expressed under the foreign risk-neutral measure, must follow the same type of process as $S^{i,l}$ itself - after all, $1/S^{i,l}$ is just the same exchange rate viewed from the other currency's perspective, not a different kind of object.

All the data used within this methodology was obtained from Bloomberg on the 22nd July 2026 for the EUR/USD/GBP FX triangle and is shown in Appendix .3.

3.1 The Multi-Heston Model

Dynamics

The model achieves both structural properties using a universal numéraire. Rather than modelling each exchange rate directly, we express the value of each currency $i = 1, \ldots, N$ relative to an artificial reference currency, $S^{0,i}(t)$. Any real exchange rate is then just a ratio of two of these, $S^{i,j} = S^{0,j}/S^{0,i}$. Each $S^{0,i}$ follows

$$\frac{dS^{0,i}(t)}{S^{0,i}(t)} = (r^0 - r^i)\,dt - (\mathbf{a}^i)^\top \sqrt{\mathrm{Diag}(\mathbf{V}(t))}\,d\mathbf{Z}(t), \qquad i = 1, \ldots, N, \quad (3.1)$$

where $\mathbf{V}(t) = (V_1(t), \ldots, V_d(t))$ is a vector of d independent Cox-Ingersoll-Ross variance factors, shared across all N currencies,

$$dV_k(t) = \kappa_k(\theta_k - V_k(t))\,dt + \xi_k\sqrt{V_k(t)}\,dW_k(t), \qquad k = 1, \ldots, d, \tag{3.2}$$

correlated with the currency-driving Brownian motions only through $d\langle Z_k, W_h\rangle_t = \rho_k\delta_{kh}\,dt$. Every currency in the system is driven by the same shared factors $\mathbf{V}(t)$, just weighted differently. This means taking the ratio $S^{i,j} = S^{0,j}/S^{0,i}$ preserves this same structure automatically.

Parameter meanings

Each parameter in the model has a direct financial interpretation, and we need to understand these to follow the calibration approaches described in the rest of this chapter.

The vector $\mathbf{a}^i \in \mathbb{R}^d$ is currency-specific. It determines how strongly currency i is exposed to each of the d shared variance factors, with the total instantaneous variance of $S^{0,i}$ given by $(\mathbf{a}^i)^\top \mathrm{Diag}(\mathbf{V}(t))\mathbf{a}^i$. For any two currencies i, j, the loading difference $\Delta\mathbf{a} = \mathbf{a}^i - \mathbf{a}^j$ plays the same role for the cross-rate $S^{i,j}$, again as a direct consequence of the ratio construction above. A naive choice, such as setting each $\mathbf{a}^i$ to a canonical basis vector, would force the correlation between any two currency pairs to be positive, which does not match observed FX behaviour. Allowing $\mathbf{a}^i$ to be a general vector, with $d \geq 2$, is what lets the model reproduce correlations of either sign. A single factor ($d = 1$) is not enough on its own. If we had only one shared source of randomness, the correlation between any two pairs is fixed by their loadings alone and cannot evolve - there is no mechanism for the model to produce the kind of time-varying, stochastic skewness observed empirically in FX markets. Two factors are the minimum needed to let this happen, with ρ_1 and ρ_2 free to take opposite signs, the model's net skew becomes a weighted blend of the two, reweighted continuously by each factor's own current variance level $V_k(t)$. We therefore use $d = 2$ throughout the calibrations that follow, balancing this minimum requirement for stochastic skewness against the practical cost of calibrating additional factors from a limited number of market-quoted tenors and strikes.

This also explains a related feature of the model, skew is generated specifically by the correlation ρ_k between each factor and the spot, together with that factor's vol-of-vol ξ_k - a factor with $\rho_k = 0$ contributes no skew at all, however large its variance or vol-of-vol. The same parameter combination reappears directly in the skew term of the short-maturity implied volatility expansion used in the single-maturity calibration method.

3.2 Single-maturity calibration via short-maturity expansion (MH Option 1)

For short maturities, the total implied variance of a smile generated by this class of stochastic volatility model admits an asymptotic expansion in log-moneyness. From De Col's paper, proposition 2 is derived in two stages, following the method of Da Fonseca & Grasselli [22]. First, the option price is expanded as a small-vol-of-vol series (Proposition 1), reading as the Black-Scholes price plus a series of stochastic-volatility terms. Implied variance is then posited to admit its own expansion in the same small parameter, $v = \zeta_0 + \alpha\zeta_1 + \alpha^2\zeta_2$, and substituted into an ordinary Black-Scholes price in place of the unknown variance v. Then matching the resulting expansion, order by order, against Proposition 1's price expansion yields exact - but still maturity-dependent - expressions for ζ_1 and ζ_2 as ratios of Black-Scholes derivatives. Second, these coefficients are themselves expanded for short maturity and truncated to leading order - the step that eliminates κ_k and θ_k entirely - leaving the closed-form implied volatility expansion used throughout this dissertation.

The resulting expansion decomposes the smile into a base level, anchored at the at-the-money implied variance, together with skew and curvature adjustments expressed relative to that level - mirroring closely the structure of the SVI parametrisation used in Task 1, though here the shape is derived from the model's own short-maturity dynamics rather than fitted as a free-form curve. To leading order, the at-the-money variance depends only on the initial variance factors and loadings,

$$\sigma^2_{\mathrm{ATM}} = \sum_{k=1}^{d} (\Delta a_k)^2 V_{0,k}, \tag{3.3}$$

where, again, Δa_k denotes the relevant loading difference for the pair being priced. This same quantity also sets the scale against which the skew and curvature adjustments below are normalised. These adjustments enter as functions of log-moneyness $x = \log(F_0/K)$,

$$\begin{aligned}\hat{\sigma}(K)^2 \approx \sigma^2_{\mathrm{ATM}} &- \frac{x}{\sigma^2_{\mathrm{ATM}}} \sum_k \tfrac{1}{2}\rho_k \xi_k (\Delta a_k)^3 V_{0,k} \\ &+ \frac{x^2}{12\sigma^4_{\mathrm{ATM}}} \left(\sum_k (1 + 2\rho_k^2)\xi_k^2 (\Delta a_k)^4 V_{0,k} - \frac{15}{4\sigma^2_{\mathrm{ATM}}} \left(\sum_k \rho_k \xi_k (\Delta a_k)^3 V_{0,k} \right)^2 \right)\end{aligned} \tag{3.4}$$

During implementation, we found a sign issue in the equation given in the paper's own Proposition 2, this is the linear term in the expansion. The corrected sign, used throughout this dissertation, is given above. For a standard negative-correlation case ($\rho < 0$), the paper's printed Proposition 2 predicts an implied volatility that is skewed toward high strikes - rising, near the money, as strike increases. Whereas the model's own verified characteristic function, and standard textbook Heston behaviour, predict the opposite: a smile skewed toward low strikes, with volatility falling as strike increases near the money (the usual downside-skew tilt, before the curvature term eventually pulls both wings back up at sufficiently extreme strikes). The corrected sign restores this expected tilt direction. We attempt to also give some visual evidence for the error by showing the corrected version more closely mimics market implied volatility curves, shown in Appendix .4.

Calibration

For each tenor independently, the ten free parameters (two loading vectors, V_0, ρ and ξ, each of dimension $d = 2$) are calibrated by least squares against that tenor's three smiles (EUR/USD, GBP/USD, EUR/GBP) simultaneously, using `scipy.optimize.least_squares` with parameter bounds and 15 random restarts, retaining the lowest-cost result. The calibrated parameters are shown in Appendix .5.

The per-tenor root-mean-square implied volatility errors are reported in Table 3.1. All six tenors are fit closely, with errors ranging from 1.70 to 4.55 basis points of volatility. It is clear that the shorter maturities fit closer which is expected given the nature of our 'short-maturity' expansion giving the lack of mean-reversion and a long-run level.

Table 3.1: Option 1 Calibration: per-tenor RMS implied volatility error.

Tenor	RMS Vol Error
1M	0.000170
2M	0.000218
3M	0.000209
6M	0.000305
9M	0.000413
1Y	0.000455

The closeness of these fits confirms the short-maturity expansion is flexible enough to match each smile well. However this is not proof of model correctness, we instead compare our market data to the implied volatility

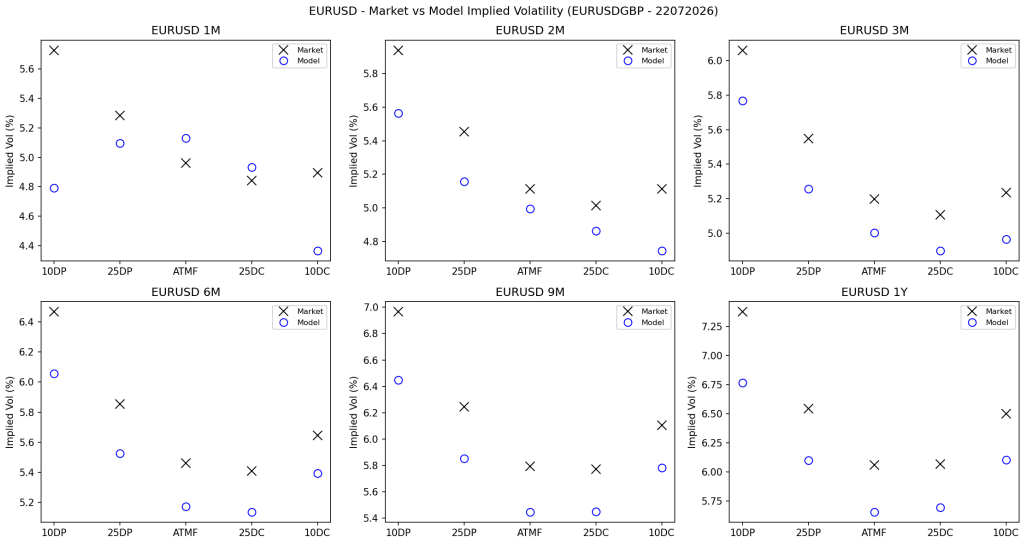


Figure 3.1: Model Implied volatility compared to Market Implied volatility on the EURUSD pair

curve that our parameterised model produces, this shows clearly that the expansion struggles to exactly replicate the market - we show the comparison of the EURUSD pair and have the other pairs comparisons in Appendix .6.

Monte Carlo pricing

Under the calibrated parameters, the two main exchange rates $X = S^{\mathrm{EUR/USD}}$ and $Y = S^{\mathrm{GBP/USD}}$ evolve according to log-price dynamics, specialised to the loadings $\Delta a_k^X, \Delta a_k^Y$ calibrated for each pair and to zero interest rates,

$$d\ln X(t) = -\tfrac{1}{2}\sum_{k=1}^{d}(\Delta a_k^X)^2 V_k(t)\,dt + \sum_{k=1}^{d}\Delta a_k^X\sqrt{V_k(t)}\,dZ_k(t), \tag{3.5}$$

$$d\ln Y(t) = -\tfrac{1}{2}\sum_{k=1}^{d}(\Delta a_k^Y)^2 V_k(t)\,dt + \sum_{k=1}^{d}\Delta a_k^Y\sqrt{V_k(t)}\,dZ_k(t). \tag{3.6}$$

Both rates are driven by the same d shared Brownian motions Z_k, weighted by each pair's own loadings. This is the sole source of correlation between X and Y in the model - there is no separate cross-pair correlation parameter. Since κ and θ are not calibrated under this approach, we include no mean-reversion term when simulating the variance factors themselves. Each factor is instead simulated as a driftless CIR process, consistent with the short-

maturity regime the calibration is actually valid in. We simulate both the variance factors (3.5)-(3.6) and the log-price processes using a simple Euler discretisation [23], with full truncation (negative variance values floored to zero) applied at each step. Antithetic variates reduce Monte Carlo variance (200,000 base paths, doubled to 400,000 via antithetic pairing, with 100 time steps). Prices for the basket, quanto, best-of and quadratic payoffs are simple averages over the simulated terminal (X, Y) pairs, computed separately for each tenor using that tenor's own independently calibrated parameters, shown below.

Table 3.2: Option 1: exotic option prices by tenor, EUR/USD/GBP.

Tenor	Basket	Quanto	Best-of	Quadratic
1M	0.005635	0.004050	0.008163	0.000112
2M	0.008313	0.005922	0.011397	0.000233
3M	0.010387	0.007420	0.014201	0.000364
6M	0.015398	0.011422	0.021106	0.000842
9M	0.019695	0.015403	0.027301	0.001489
1Y	0.023492	0.018741	0.032690	0.002177

As in Task 3 of the entropic optimal transport method, the best-of price exceeds the basket price at every tenor, this holds regardless of the pricing method or the assumed dynamics, and serves as a useful cross-check here as well.

3.3 Multi-tenor calibration via the characteristic function (MH Option 2)

Within the multi-tenor calibration, there are no parameters taken out. Within Appendix A of De Col's [6] paper, they derive the characteristic function of their model's dynamics through the use of the backward Kolmogorov equation and using an affine ansatz equation for the characteristic function to guess the solution and solving the PDEs that arise from this to give the finalised closed form characteristic function, given below:

$$G^{i,j}(\omega, t, \tau, x, \mathbf{V}) = \exp\left[\omega x + (r^i - r^j)\,\omega\tau + \sum_{k=1}^{d}\left(A_k^{i,j}(\tau) + B_k^{i,j}(\tau)V_k\right)\right], \tag{3.7}$$

where, for $k = 1, \ldots, d$:

$$A_k^{i,j}(\tau) = \frac{2\kappa_k\theta_k}{\xi_k^2}\log\left(\frac{\lambda_k^+ - \lambda_k^-}{\lambda_k^+ e^{\lambda_k^-(\tau)} - \lambda_k^- e^{\lambda_k^+(\tau)}}\right); \tag{3.8}$$

$$B_k^{i,j}(\tau) = \frac{\omega^2 - \omega}{2}\left(a_k^i - a_k^j\right)^2 \frac{1 - e^{-\sqrt{\Delta_k}\tau}}{\lambda_k^+ e^{-\sqrt{\Delta_k}\tau} - \lambda_k^-}; \tag{3.9}$$

$$\Delta_k = \left(-\kappa_k + \omega\left(a_k^i - a_k^j\right)\rho_k\xi_k\right)^2 - \xi_k^2(\omega^2 - \omega)\left(a_k^i - a_k^j\right)^2; \tag{3.10}$$

$$\lambda_k^{\pm} = \frac{\left(-\kappa_k + \omega\left(a_k^i - a_k^j\right)\rho_k\xi_k\right) \pm \sqrt{\Delta_k}}{2}. \tag{3.11}$$

Evaluated exactly as printed, however $A_k^{i,j}(\tau)$ is numerically unstable. The formula contains the logarithm of a complex number, which has no single correct value. Infinitely many answers are equally valid, each differing by a fixed rotation of phase θ, and standard numerical libraries resolve this by always returning the value lying within one fixed range. As maturity τ varies continuously, the true value can wind past the edge of this range, and it silently returns a value that is off by a fixed rotation, with no error or warning raised. This is a recognised numerical pitfall in Heston-type models, known as the "Little Heston trap" [24]. We therefore use the standard, algebraically-equivalent reformulation, substituting $g_k = \lambda_k^+/\lambda_k^-$:

$$A_k^{i,j}(\tau) = \frac{\kappa_k\theta_k}{\xi_k^2}\left(-2\lambda_k^+\tau - 2\log\frac{1 - g_k e^{-\sqrt{\Delta_k}\tau}}{1 - g_k}\right), \tag{3.12}$$

De Col et al's paper describes that by applying arguments given by Lewis [25], Lipton [26], Sepp [27], the value of a vanilla option can be determined from the integral of the product of the fourier transform of the payoff and the characteristic function of the log-price. The general Lewis contour-integral approach involves integrating along a horizontal line $\{a + bi : a \in \mathbb{R}\}$ for some fixed b, chosen to lie within the model's strip of regularity – the band $b_{\min} < b < b_{\max}$ over which the characteristic function remains finite and analytic, this band depends on both the model and the specific parameters chosen. This works well for pricing at a fixed, known set of parameters, but is a poor fit for our setting as calibration calls this pricing step repeatedly, at

every optimiser iteration, for parameter sets chosen automatically and not known in advance, so a contour valid for one candidate parameter vector isn't guaranteed to stay valid for the next.

To avoid this, we recover call prices using the Attari [28] representation instead - a Fourier-inversion formula that integrates over the fixed real line. It does this by evaluating ϕ at the fixed complex shift $u - \frac{i}{2}$, rather than searching for a suitable contour for each candidate parameter set, which makes it robust to repeated, automated use inside the calibration loop. With $x = \log(K/F_0)$,

$$C(F_0, K, T) = F_0 - \frac{\sqrt{F_0 K}}{\pi} \int_0^\infty \frac{\mathrm{Re}[\phi(u - \frac{i}{2})] \cos(u\,x) + \mathrm{Im}[\phi(u - \frac{i}{2})] \sin(u\,x)}{u^2 + \frac{1}{4}}\, du. \tag{3.13}$$

We evaluate this integral via 64-node Gauss-Legendre quadrature, truncated at an upper limit of $u = 100$ – comfortably enough to capture the decay of the integrand, while keeping the per-strike pricing cost low, since equation (3.13) gets evaluated repeatedly inside the calibration objective described next.

Since calibration is done in implied-volatility space rather than price space, for the same reasons as Task 1, we convert each model price from equation (3.13) back to an implied volatility by numerically inverting the Black-76 [21] formula via `scipy.optimize.brentq`, exactly as in the consistency check of Task 2. This gives, for any candidate parameter set, a model-implied volatility directly comparable to the market-quoted volatility at the same strike, tenor and pair.

Joint calibration across tenors and pairs

All fourteen parameters (two loading vectors, κ, θ, ξ, ρ and V_0, each of dimension $d = 2$) are calibrated jointly against all three pairs and all six tenors simultaneously - ninety market quotes in total - using `scipy.optimize.least_squares` with bounds and three random restarts. Fewer restarts were used than in the short maturity expansion calibration, owing to the substantially higher per-evaluation cost of this objective - computing the characteristic function and inverting it to an implied volatility at every strike, tenor and pair.

Calibration results

The joint calibration achieves an overall RMS implied volatility error of 0.001861 (18.6 basis points). The calibrated parameters are given in Table

3.3.

The RMS error per tenor is given by the table below, it is clear to see that the closeness of fit is worse than that of our earlier 'shortcut' calibration method, but that is expected due to the lack of flexibility when calibrating to all ninety market quotes simultaneously. However we can see in Figure 3.2, showing the EURUSD pair's comparison, that this model's calibrated parameters match the market far closer, other than on the 1M maturity fit which is a problem that De Col's paper also mentions.

Table 3.3: Option 2 Calibration: per-tenor RMS implied volatility error

Tenor	RMS Vol Error
1M	0.003980
2M	0.001452
3M	0.000988
6M	0.001053
9M	0.000541
1Y	0.000678

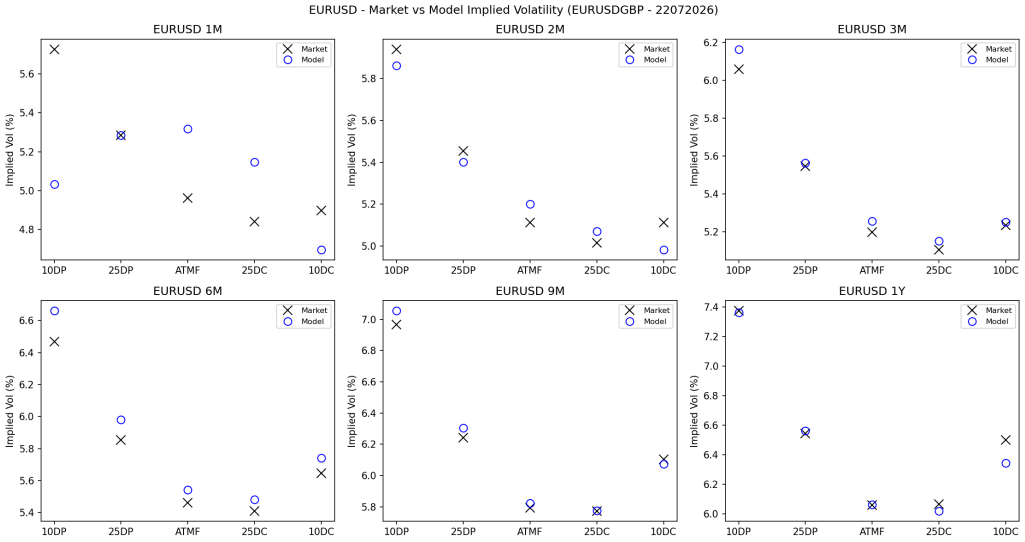


Figure 3.2: Model Implied volatility compared to Market Implied volatility on the EURUSD pair - Multi Tenor Calibration

The other FX pairs model implied volatility comparisons to the market implied volatility is shown in Appendix .7.

Monte Carlo pricing

With κ and θ now identified, the variance factors are simulated with their full mean-reverting CIR dynamics, $dV_k = \kappa_k(\theta_k - V_k)\,dt + \xi_k\sqrt{V_k}\,dW_k$, using the same Euler discretisation, full truncation, correlation structure and antithetic variance reduction as before. The same calibrated parameters are used across every tenor - only the simulation horizon T changes - reflecting the fact that, unlike Option 1's six independent single-tenor calibrations, Option 2 produces one consistent set of dynamics valid at any maturity.

Table 3.4 reports the resulting prices across all six tenors.

Table 3.4: Full-Maturity Calibration Method : Exotic option prices by tenor, EUR/USD/GBP.

Tenor	Basket	Quanto	Best-of	Quadratic
1M	0.005982	0.004365	0.008729	0.000129
2M	0.008535	0.006377	0.012606	0.000284
3M	0.010633	0.008097	0.015838	0.000463
6M	0.015898	0.012737	0.024059	0.001139
9M	0.020554	0.017049	0.031415	0.001991
1Y	0.024830	0.021060	0.038140	0.002970

The best-of price again exceeds the basket price at every tenor, consistent with the model-independent check applied throughout this dissertation. Prices under the full method are close to, but consistently slightly larger than, those under our 'shortcut' approach at every tenor and for every payoff - reflecting the presence of mean reversion under Option 2, which alters the variance paths' distribution over longer horizons relative to the driftless factors used in Option 1.

Chapter 4

Results and Discussion

Within the following section we will evaluate and compare our explained methods on various datasets which will prove to stress test in differing market environments. The datasets can be found in Appendix .8 - they include data from the financial crisis of 2008, Liz Truss budget cuts of 2022, calmer periods in 2005 and a different currency triangle: AUS/USD/JPY. Our results of RMS error with market consistency are given in Table 4.1 and exotic prices given in Tables 11, 12, 13, 14, and 15 - within Appendix .9.

4.1 Discussion

The three methods are best understood not as competitors on a single scale but as occupying three distinct measures: fit quality - how well each reproduces the observed smiles, price agreement - how closely their exotic prices coincide, and dynamic usability - whether the object produced can be simulated, evolved or hedged. These do not coincide, and the central message of this chapter is that the best-fitting method is not the most usable one. The joint density (sinkhorn) method produces a static terminal coupling selected by minimum relative entropy. The multi-Heston models produce a genuinely simulatable process at the cost of fit. We develop the comparison along these axes rather than declaring a single winner.

Table 4.1: By pair RMS error, model vs market (bps), across the entropic optimal transport (Sinkhorn), Option 1, and Option 2 methods, both triangles. Pair 1 / Pair 2 refer to EUR/USD, GBP/USD (or AUD/USD, USD/JPY for the AUD/USD/JPY triangle); Cross Pair refers to EUR/GBP (or AUD/JPY).

Method	Dataset	Pair 1 (bps)	Pair 2 (bps)	Cross Pair (bps)
Sinkhorn	2005 (Calm)	0.8	1.1	2.6
	2008 (Financial Crash)	13.9	4.6	33.9
	2026 (Baseline)	6.4	0.5	11.6
	2022 (Budget Cut)	17.2	0.6	43.2
	2026 (AUD/USD/JPY)	2.4	12.3	2.3
Full Multi-Heston	2005 (Calm)	12.71	11.54	15.47
	2008 (Financial Crash)	37.99	53.69	33.93
	2026 (Baseline)	17.42	11.52	24.56
	2022 (Budget Cut)	16.86	33.37	39.89
	2026 (AUD/USD/JPY)	10.30	31.64	18.14
Shortcut M-H	2005 (Calm)	32.22	30.69	29.28
	2008 (Financial Crash)	148.48	151.52	73.61
	2026 (Baseline)	37.52	41.97	44.70
	2022 (Budget Cut)	72.33	136.51	86.40
	2026 (AUD/USD/JPY)	49.06	63.27	79.26

4.1.1 Two opposite treatments of dependence

The two families sit at opposite ends of the dependence spectrum by construction. The entropic coupling is the joint law closest to independence ($\bar{\mu} = \mu_X \otimes \mu_Y$) consistent with the three marginal smiles. It injects the least dependence the data will allow. The multi-Heston models instead impose dependence parametrically, through shared and loaded variance factors. One would therefore expect the methods to disagree most on the payoffs most sensitive to dependence, and to agree on those that are not. The rest of this section tests that prediction, and traces the disagreement to its true source.

Table 4.2: Higher moments of the recovered cross-rate density, Sinkhorn versus Multi-Heston (Option 2), obtained directly from each method's own terminal law (Heston by Fourier inversion of its calibrated characteristic function, so the tails are the model's own and not SVI-smoothed). All densities are forward-normalised. The standard deviations agree with each other and with the market-implied value $\sigma_{\mathrm{ATM}}\sqrt{T}$ to within a few percent, so the comparison is on a common scale.

		Excess kurtosis		Std. dev.	
Dataset	Tenor	Sinkhorn	Heston	Sinkhorn	Heston
2005 (Calm), EUR/GBP	1M	0.94	1.19	0.0167	0.0171
	1Y	0.96	1.89	0.0623	0.0643
2008 (Crisis), EUR/GBP	1M	1.06	1.61	0.0515	0.0512
	1Y	2.25	3.62	0.1254	0.1325
2026, AUD/JPY	1M	2.80	3.20	0.0225	0.0230
	1Y	2.13	3.07	0.1051	0.1086

4.1.2 Direct evidence: the cross-density tails

The clearest way to see where the two methods differ is to compare the recovered cross-rate densities directly. For each method we take the forward-normalised density of the cross rate. For Sinkhorn this is the target marginal μ_Z. For Multi-Heston it is the density obtained by Fourier inversion of its own calibrated characteristic function, so its tails are genuinely the model's and not re-smoothed through SVI. Their moments are given in Table 4.2.

The result is unambiguous and holds across differing regimes, tenors and triangles tested. The two methods agree on the second moment but differ systematically in the fourth. Every standard deviation pair agrees to within a few percent, and each brackets the market-implied $\sigma_{\mathrm{ATM}}\sqrt{T}$, so the densities are genuinely on the same scale. Yet the Multi-Heston cross carries a higher excess kurtosis in all six comparisons, without exception. This is the density-level statement of the chapter's thesis. The Multi-Heston model over states tail risk in the cross-rate compared to the sinkhorn - which is almost exaclty the market, therefore it is fair to say that the Multi-Heston overstates tail risk compared to where the market sees it.

The gap is present everywhere but is not uniform. On the EUR/GBP triangle it widens with maturity (1M to 1Y), from 0.25 to 0.93 in the calm

2005 data and from 0.55 to 1.37 in the 2008 crisis. On the higher-volatility AUD/JPY triangle both densities are already strongly leptokurtic at one month, with excess kurtosis of roughly 2.8 and 3.2, so there is little room for a maturity trend. The Heston tail stays the fatter of the two at both tenors. Both methods also agree on the sign of the skew throughout - positive for EUR/GBP, strongly negative for AUD/JPY, as the risk-on/risk-off asymmetry of that triangle demands - with the entropic coupling again the more moderate, consistent with its smoothing toward the independent reference.

4.1.3 Corroboration: correlation and the quanto gap

Table 4.3: Implied correlation of the two base rates, recovered from each method's own joint terminal law. For EUR/USD/GBP the rates are EUR/USD and GBP/USD. For AUD/USD/JPY both methods are placed on the common Corr(AUD/USD, USD/JPY) convention - the Sinkhorn value, computed on the inverted JPY/USD leg, is sign-flipped to match. Monte-Carlo standard error ≈ 0.002, so differences above ~ 0.005 are genuine.

Dataset	Tenor	Sinkhorn ρ	Multi-Heston ρ	Difference
2005 (Calm), EUR/GBP	1M	0.806	0.800	+0.006
	1Y	0.783	0.782	+0.001
2008 (Crisis), EUR/GBP	1M	0.785	0.793	−0.008
	1Y	0.739	0.787	−0.048
2022 (Budget), EUR/GBP	1M	0.617	0.678	−0.061
	1Y	0.617	0.671	−0.054
2026 (Baseline), EUR/GBP	1M	0.765	0.773	−0.008
	1Y	0.724	0.726	+0.002
2026, AUD/JPY	1M	−0.377	−0.321	−0.056
	1Y	−0.300	−0.247	−0.053

Two further diagnostics support the density result and rule out the obvious alternative - that the two methods simply disagree on how much the rates co-move. The first is the implied correlation of the base rates, recovered from each joint terminal law and given in Table 4.3. Across every dataset and both triangles the two methods agree. The absolute level is regime-dependent - near 0.80 in the calm 2005 data, falling to ≈ 0.62 in the sterling-specific 2022

stress, and negative on AUD/USD/JPY, where AUD/USD rises in risk-on markets and USD/JPY in risk-off markets - but at each point the Sinkhorn and Multi-Heston values track each other to within about 0.05 to 0.06 at worst. The largest gaps sit at the long end of the 2008 crisis and in the 2022 stress, and even these are an order of magnitude smaller than the price disagreements they are meant to explain.

This agreement is the key control. The two methods encode essentially the same linear dependence, on both triangles, yet still price the exotics differently in areas. The disagreement therefore cannot be a first-moment effect. It has to reside in the higher moments - exactly what the kurtosis of Table 4.2 shows directly. Correlation and kurtosis together isolate the difference to the fourth moment.

The fact that the correlations agree does not mean the dependence is fully linear. The orientation residual $r = \mathrm{Corr}(X, \mathrm{USD/JPY}) + \mathrm{Corr}(X, \mathrm{JPY/USD})$ on AUD/USD/JPY measures the non-linear part directly, being zero only when the dependence is purely linear (Table 4.4). The Sinkhorn coupling and the per-tenor Option 1 calibration both hold residuals within noise of zero, so their dependence is near-linear. Only the full joint Option 2 calibration develops a residual that grows with maturity, reaching -0.059 at one year. The small excess kurtosis carried by Option 2, then, is not a correlation effect but a genuinely non-linear feature of its joint stochastic-volatility dynamics, and one that accumulates over longer horizons - which shows similarities to the widening of the EUR/GBP kurtosis gap in Table 4.2.

Table 4.4: AUD/USD/JPY orientation residual, $r = \mathrm{Corr}(X, \mathrm{USD/JPY}) + \mathrm{Corr}(X, \mathrm{JPY/USD})$, which measures non-linear dependence beyond linear correlation (exactly zero under the first-order approximation). Sinkhorn and the per-tenor Option 1 calibration stay near zero, only the joint Option 2 calibration develops non-linear dependence with maturity.

Tenor	1M	2M	3M	6M	9M	1Y
Sinkhorn	-0.003	-0.009	-0.011	-0.013	-0.015	-0.015
Option 1	-0.009	-0.014	$+0.002$	$+0.001$	$+0.000$	-0.000
Option 2	-0.011	-0.020	-0.026	-0.042	-0.053	-0.059

The second diagnostic is the exotic prices themselves, since the four payoffs probe different regions of the joint. The quadratic $\mathbb{E}[(X-Y)^2]$ reads pure covariance, the quanto $\mathbb{E}[(X/Y-1)^+]$ is maximally cross-sensitive, the best-of $\mathbb{E}[(\max(X,Y)-1)^+]$ is sensitive to tail dependence, and the basket $\mathbb{E}[(\frac{1}{2}(X+Y)-1)^+]$ is comparatively insensitive to dependence. This or-

dering explains the price spreads. On the EUR triangle the three-method spread on the basket stays narrow even into the 2008 crisis (Tables 11-14). On AUD/USD/JPY the quanto disagreement is large and persistent, with Sinkhorn's quanto sitting 28-33% below both Multi-Heston variants at every tenor (Table 15). It is the payoff's dependence-sensitivity that predicts how far the methods diverge, not the calibration method or the regime alone. The quanto is the most cross-sensitive payoff, so it is where the fatter Option 2 cross tail is priced most visibly.

4.1.4 The cross-rate: each method fails, but differently

The cross-rate is where each method is weakest, but for structurally distinct reasons (Table 4.1). For Sinkhorn the cross is worst by construction. It is the only marginal not enforced by a closed-form update, being imposed instead through the one-dimensional root-find for $w(z)$ and its cubic-spline interpolation. Its cross RMS is consistently the largest of the three legs, at 2.6 bp in the calm 2005 conditions, rising to 33.9 bp in 2008 and 43.2 bp in 2022. This is the key comparison. Even Sinkhorn's worst leg, the cross at 33.9 bp, is comparable to or better than the multi-Heston models' typical legs, such as the 37.99-53.69 bp on the base pairs in 2008.

The multi-Heston failure is different in both cause and location. On the EUR triangle the cross also fits worst, reflecting its derived loading and lack of dedicated degrees of freedom. But this pattern breaks on AUD/USD/JPY. Under the joint Option 2 calibration it is USD/JPY that fits worst at 31.6 bp, not the cross, because a single parameter set $(\kappa, \theta, \xi, \rho)$ cannot track USD/JPY's term-structure skew reversal across six maturities at once. This being a joint-calibration artefact rather than a property of the triangle is confirmed by Option 1, which calibrates each tenor independently and reverts to the ordinary pattern with the cross worst, at 8.3 bp against USD/JPY's 6.7 bp. The controlled comparison isolates the failure precisely to term-structure sharing.

4.1.5 Fit, agreement and usability do not coincide

These strands resolve into the chapter's thesis. Sinkhorn attains the best fit essentially everywhere, as shown in Table 4.1, yet is the clear outlier on three of the four AUD/USD/JPY payoffs (Table 15). Best-fitting is not closest-agreeing, and neither is most usable. The deepest asymmetry is structural

rather than numerical. The entropic scheme produces a single terminal coupling, so it cannot price path-dependent payoffs, cannot be evolved forward in time, and offers no hedging story. The multi-Heston models deliver all three, at the documented market consistency cost. The comparison is thus not a ranking but a fit-for-purpose decision. Sinkhorn is suited to market-consistent terminal-payoff pricing where fit is paramount and no dynamics are required. Multi-Heston is suited to cases where a simulatable, hedgeable process is needed and the fit gap is acceptable.

4.1.6 Neither method is assumption-light

Finally, neither family is really as assumption-light as previously thought. The entropic method's "assumption-free" character means only free of a parametric dynamics assumption. It still assumes SVI marginals are unimodal, parabolic and arbitrage-free. A W-shaped, bimodal, pre-event smile violates the first, and a poorly-fit SVI slice can produce a negative density, which breaks the $-\log$ step of the scheme outright rather than merely fitting poorly. The multi-Heston side makes equally load-bearing choices. The factor count $d = 2$ is chosen, not derived, and Option 1's $\kappa = 0$ is not a neutral simplification. Holding $\mathbb{E}[V_t] = V_0$ fixed via $\theta = V_0$, removing mean reversion shifts the ATM implied volatility dwon from 18.33% to 16.97%, which is clearly the reason for the M-H option 1 understating the market implied volatility across the board. The two families are not assumption-light. They are light on different assumptions.

A parallel holds for uniqueness. The entropic coupling is structurally non-unique. Infinitely many joints share the given marginals, and minimum relative entropy selects one. Multi-Heston is a single point once calibrated, but not unconditionally. Multi-start calibration reveals optimiser non-uniqueness on at least the 2022 dataset, where one restart settled on an objective of 1.12×10^{-3} against 4.48×10^{-4} for the others. The non-uniqueness is of different kinds, structural versus numerical, but neither method delivers a single unconditional answer.

4.2 Conclusion

The central finding is a clean separation between where the methods agree and where they do not. They agree on the amount of dependence they encode - the implied correlations coincide to within a few hundredths on every dataset and both triangles - yet they still price dependence-sensitive exotics differently. We traced this to the higher moments. Comparing the recovered cross-rate densities directly, the Heston coupling carries a systematically fatter tail than the entropic one in every case, at matching variance. The entropic scheme, being the joint closest to independence, produces the thinnest-tailed cross-rate the market smiles would allow, and so under-states tail risk. The price disagreements are a fourth-moment effect, not a first-moment one, and they surface most visibly in the quanto, the payoff most exposed to the tail of the cross.

This resolves into a fit-for-purpose verdict rather than a ranking. The entropic scheme is markedly the more market-consistent, recovering the smiles to a few basis points where the Heston fit is an order of magnitude worse, and its weakest leg still matches the parametric model's typical one. But it yields only a static terminal coupling - it cannot price path-dependent payoffs, cannot be evolved forward, and offers no hedge. The Heston model is worse-fitting but genuinely simulatable. Fit quality, price agreement and dynamic usability are three distinct axes, and no method dominates all three. Nor is either as assumption-light as its framing suggests - the entropic scheme still rests on SVI marginals and a minimum-relative-entropy dependence selection, and the Heston model on its factor count and mean-reversion structure.

Several directions would extend this work. Replacing SVI with a more flexible arbitrage-free parametrisation, one admitting bimodal smiles, would widen the entropic scheme's reach without touching the coupling machinery. Imposing a dependence constraint beyond the marginals would move it toward a martingale optimal transport formulation that returns price bounds rather than a single value, closing the higher-moment gap from the entropic side. Extending the comparison from a triangle to a full currency basket would test the entropic scheme against the curse of dimensionality far harder than the parametric one. In each case the central distinction of this chapter would only sharpen - that market consistency and dynamic usability are separate virtues, bought at each other's expense.

Bibliography

[1] M. B. Garman and S. W. Kohlhagen, *Foreign Currency Option Values*, Journal of International Money and Finance, Vol. 2, No. 3, pp. 231–237, 1983.

[2] F. Black and M. Scholes, *The Pricing of Options and Corporate Liabilities*, Journal of Political Economy, Vol. 81, No. 3, pp. 637–654, 1973.

[3] R. C. Merton, *Option Pricing when Underlying Stock Returns are Discontinuous*, Journal of Financial Economics, Vol. 3, No. 1–2, pp. 125–144, 1976.

[4] D. S. Bates, *Jumps and Stochastic Volatility: Exchange Rate Processes Implicit in Deutsche Mark Options*, The Review of Financial Studies, Vol. 9, No. 1, pp. 69–107, 1996.

[5] S. L. Heston, *A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options*, The Review of Financial Studies, Vol. 6, No. 2, pp. 327–343, 1993.

[6] A. De Col, A. Gnoatto, and M. Grasselli, *Smiles all around: FX Joint Calibration in a Multi-Heston Model*, arXiv:1201.1782, 2013.

[7] B. Dupire, *Pricing and Hedging with Smiles*, Paribas Capital Markets, Swaps and Options Research Team, Working Paper, April 1993.

[8] U. Wystup, *FX Options and Structured Products*, Wiley, 2006.

[9] I. Clark, *Foreign Exchange Option Pricing: A Practitioner's Guide*, Wiley, 2011.

[10] D. T. Breeden and R. H. Litzenberger, *Prices of State-Contingent Claims Implicit in Option Prices*, Journal of Business, Vol. 51, No. 4, pp. 621–654, 1978.

[11] J. Gatheral, *A Parsimonious Arbitrage-Free Implied Volatility Parameterization with Application to the Valuation of Volatility Derivatives*, Presentation at the Global Derivatives & Risk Management Conference, Madrid, 2004.

[12] J. Gatheral and A. Jacquier, *Arbitrage-Free SVI Volatility Surfaces*, Quantitative Finance, Vol. 14, No. 1, pp. 59–71, 2014.

[13] R. W. Lee, *The Moment Formula for Implied Volatility at Extreme Strikes*, Mathematical Finance, Vol. 14, No. 3, pp. 469–480, 2004.

[14] W. Margrabe, *The Value of an Option to Exchange One Asset for Another*, Journal of Finance, Vol. 33, No. 1, pp. 177–186, 1978.

[15] R. M. Stulz, *Options on the Minimum or the Maximum of Two Risky Assets: Analysis and Applications*, Journal of Financial Economics, Vol. 10, No. 2, pp. 161–185, 1982.

[16] H. Johnson, *Options on the Maximum or the Minimum of Several Assets*, Journal of Financial and Quantitative Analysis, Vol. 22, No. 3, pp. 227–283, 1987.

[17] J. Guyon and P. Henry-Labordère, *The Smile Calibration Problem Solved*, preprint, 2011.

[18] J. Guyon, *Dispersion-Constrained Martingale Schrödinger Bridges: Joint Entropic Calibration of Stochastic Volatility Models to S&P 500 and VIX Smiles*, SSRN Working Paper 4165057, 2020.

[19] M. Nutz, *Introduction to Entropic Optimal Transport*, Lecture Notes, 2022.

[20] M. Forde, *Solving the FX Cross-Smiles Problem: Rate of Convergence for Sinkhorn Marginals, and the Finite-Option Case*, Statistics & Probability Letters, to appear.

[21] Black, F. (1976). The pricing of commodity contracts. *Journal of Financial Economics*, 3(1–2), 167-179.

[22] J. Da Fonseca and M. Grasselli, *Riding on the Smiles*, Quantitative Finance, Vol. 11, No. 11, pp. 1609-1632, 2011.

[23] P. E. Kloeden and E. Platen, *Numerical Solution of Stochastic Differential Equations*, Springer-Verlag, 1992.

[24] H. Albrecher, P. Mayer, W. Schoutens, and J. Tistaert, *The Little Heston Trap*, Wilmott Magazine, pp. 83-92, 2007.
`https://api.unil.ch/iris/server/api/core/bitstreams/86f00be0-bbb6-4cd2-a536-6f1fbb1766bc/content`

[25] A. L. Lewis, *Option Valuation under Stochastic Volatility: With Mathematica Code*, Finance Press, Newport Beach, 2000.

[26] A. Lipton, *Mathematical Methods for Foreign Exchange: A Financial Engineer's Approach*, World Scientific Publishing Co Pte Ltd, Singapore, 2001.

[27] A. Sepp, *Pricing European-Style Options under Jump Diffusion Processes with Stochastic Volatility: Applications of Fourier Transform*, University of Tartu, Preprint, 2003.

[28] M. Attari, *Option Pricing Using Fourier Transforms: A Numerically Efficient Simplification*, SSRN Working Paper, 2004.

AI Usage Declaration

I declare that I have used the AI tool(s) listed below, that I have carefully reviewed and verified all AI-generated content, and that I fully understand every part of this submission.

Tool used: Claude (Anthropic), various versions across the course of this project.

I used the AI tool for the following purposes:

- **Brainstorming and ideation:** developing the overall dissertation structure, chronological framing and gaining ideas of relevant topics.
- **Code assistance:** Explaining the methodology and giving some specific code for sections (e.g multi-start calibration loop, central finite-difference method, covariance and density comparison etc), all of which I have rewritten and fully checked and understood. Also explained some particular choices in the code (e.g. calibration bounds, quadrature domain selection, root-finding safeguards) and debugging suggestions. For the multi-heston model code, use of Claude to create similar calibration codes to my joint density method, while being thoroughly checked. As well as help for the monte carlo pricing.
- **Literature summarisation and verification:** summarising and cross-checking papers cited in this dissertation, and locating correct bibliographic details for citations.
- **Language polish:** grammar correction, clarity improvements, and stylistic edits.

In all cases I carefully verified the result, corrected and removed any irrelevant information, hallucinations, or inaccuracies, and rewrote material to the extent that I can certify the result is a true reflection of my own

understanding, thoughts and knowledge. I discussed the use of AI tools with my supervisor regularly during supervisory sessions and followed their guidance. I understand that I may not receive credit, and can be penalised, for improper use of AI-generated content.

Signed: *Lewis Wilcher*

Chapter 5

Appendix

.1 EUR/USD/GBP Data - 16th March 2024

Table 1: (Mid) implied volatilities with 1-month maturity on 16th March 2024, with strikes listed horizontally. Forward prices: EUR/USD = 1.0903, GBP/USD = 1.2738, EUR/GBP = 0.85590. Data obtained from Bloomberg.

EUR/USD					
Strike	1.0681	1.0791	1.0904	1.1014	1.1119
Implied Vol	0.0554	0.053115	0.0516	0.051435	0.0523
GBP/USD					
Strike	1.2456	1.2595	1.274	1.2883	1.3017
Implied Vol	0.06055	0.058665	0.0573	0.057185	0.05765
EUR/GBP					
Strike	0.84386	0.84969	0.85585	0.86234	0.86875
Implied Vol	0.03809	0.03725	0.037225	0.03825	0.03986

.2 Joint Density Pricing - Model vs Market Implied Volatility Tables

Table 2: Model-implied volatility (from μ^*) versus market-quoted implied volatility, at the five original market strikes.

Pair	Normalised Strike K/F_0	Model IV	Market IV
EUR/USD	0.9796	0.05542	0.05540
	0.9897	0.05313	0.05312
	1.0001	0.05162	0.05160
	1.0102	0.05146	0.05144
	1.0198	0.05232	0.05230
GBP/USD	0.9779	0.06057	0.06055
	0.9888	0.05868	0.05867
	1.0002	0.05732	0.05730
	1.0114	0.05720	0.05718
	1.0219	0.05767	0.05765
EUR/GBP	0.9859	0.03811	0.03809
	0.9927	0.03728	0.03725
	0.9999	0.03725	0.03723
	1.0075	0.03827	0.03825
	1.0150	0.03988	0.03986

.3 EUR/USD/GBP Data - 22nd July 2026

Table 3: Bloomberg OVML implied volatility quotes, as of 22 July 2026, for the EUR/USD/GBP triangle, across six tenors and five delta conventions (ATMF, 25-delta call/put, 10-delta call/put). Vol figures in percent (mid). Spot rates (constant across tenors): EUR/USD = 1.1410, GBP/USD = 1.3366, EUR/GBP = 0.85344.

		EUR/USD		GBP/USD		EUR/GBP	
Tenor	Quote	Vol	Strike	Vol	Strike	Vol	Strike
	ATMF	4.962	1.1425	5.864	1.3366	3.656	0.85455
	25DF Call	4.8415	1.1531	5.6605	1.3511	3.9485	0.86099
1 Month	10DF Call	4.897	1.163	5.6285	1.3642	4.3755	0.86819
	25DF Put	5.2845	1.1311	6.323	1.3207	3.5825	0.84875
	10DF Put	5.727	1.1192	6.8935	1.3039	3.685	0.84327
	ATMF	5.113	1.1447	6.0345	1.3367	3.733	0.85564
	25DF Call	5.015	1.1601	5.792	1.3584	4.095	0.86544
2 Month	10DF Call	5.113	1.1754	5.771	1.3782	4.607	0.87679
	25DF Put	5.4545	1.1268	6.5565	1.3125	3.6175	0.84708
	10DF Put	5.9395	1.1088	7.21	1.2868	3.7105	0.83908
	ATMF	5.199	1.1455	6.179	1.3367	3.8315	0.85673
	25DF Call	5.106	1.1655	5.9115	1.3637	4.2475	0.86915
3 Month	10DF Call	5.235	1.1849	5.896	1.3886	4.8235	0.88385
	25DF Put	5.547	1.1242	6.762	1.3065	3.6865	0.84611
	10DF Put	6.06	1.1019	7.525	1.2739	3.7605	0.83632
	ATMF	5.461	1.1498	6.591	1.3366	4.163	0.86005
	25DF Call	5.4085	1.1799	6.2945	1.3774	4.6915	0.87957
6 Month	10DF Call	5.6445	1.2107	6.3385	1.4164	5.422	0.90373
	25DF Put	5.854	1.1182	7.312	1.2908	3.938	0.84405
	10DF Put	6.4665	1.0848	8.274	1.2409	3.972	0.82975
	ATMF	5.793	1.1539	6.889	1.3362	4.5295	0.86335
	25DF Call	5.7725	1.1936	6.5615	1.3886	5.156	0.88981
9 Month	10DF Call	6.105	1.2356	6.642	1.4396	6.008	0.92349
	25DF Put	6.244	1.1127	7.7375	1.2773	4.2715	0.84211
	10DF Put	6.966	1.0689	8.863	1.2126	4.314	0.82324
	ATMF	6.059	1.1579	7.104	1.3356	4.766	0.86677
	25DF Call	6.0655	1.2064	6.7585	1.3981	5.4635	0.89939
1 Year	10DF Call	6.501	1.2598	6.866	1.46	6.434	0.94217
	25DF Put	6.542	1.1081	8.013	1.2656	4.4765	0.84105
	10DF Put	7.376	1.0548	9.287	1.1881	4.521	0.81836

.4 Short Maturity Expansion Sign Error Comparison

Table 4: Calibration fit quality versus true-model error, for both the printed (believed incorrect) and corrected Proposition 2 skew sign. Fit RMS is identical between the two signs at every tenor, confirming that calibration alone cannot detect the error. True RMS - computed by checking the calibrated parameters against the independently-verified characteristic function - reveals the proposition 2 sign to be consistently worse.

	Fit RMS (bp)		True RMS (bp)		
Tenor	Prop 2	Corrected	Prop 2	Corrected	Prop 2 / Corrected
1M	1.70	1.70	82.01	55.74	1.47
2M	2.18	2.18	97.61	34.92	2.79
3M	2.09	2.09	81.16	29.65	2.74
6M	3.05	3.05	90.21	34.25	2.63
9M	4.13	4.13	102.57	41.48	2.47
1Y	4.55	4.55	111.44	47.29	2.36
Average	2.95	2.95	94.17	40.56	2.41

.5 Multi Heston Option 1 - Calibrated Parameter Values

Table 5: Calibrated loading parameters, EUR/USD/GBP triangle, 22 July 2026 (EURUSD = Loading1, GBPUSD = Loading2). Each tenor is calibrated independently.

	Loading1 (EURUSD)		Loading2 (GBPUSD)	
Tenor	Factor 1	Factor 2	Factor 1	Factor 2
1M	0.1825	-0.0987	0.2261	0.0764
2M	0.2379	0.0933	0.1558	0.2712
3M	0.2261	0.0751	0.1486	0.2184
6M	0.1512	0.0868	0.1016	0.2650
9M	0.0631	0.1986	0.2026	0.1293
1Y	0.0513	0.1877	0.1719	0.1212

Table 6: Calibrated variance-factor parameters, EUR/USD/GBP triangle, 22 July 2026, continued from Table 5.

	V_0		ρ		ξ	
Tenor	Factor 1	Factor 2	Factor 1	Factor 2	Factor 1	Factor 2
1M	0.0629	0.0400	-0.3096	-0.2232	0.9544	0.8925
2M	0.0408	0.0360	-0.2075	-0.3458	0.5659	0.6357
3M	0.0466	0.0587	-0.1996	-0.3654	0.5179	0.6965
6M	0.1160	0.0455	-0.1627	-0.3860	0.6354	0.4659
9M	0.0860	0.0769	-0.3973	-0.1473	0.5513	0.4460
1Y	0.1263	0.0954	-0.4049	-0.1338	0.6023	0.4482

.6 Multi Heston Option 1 - Market vs Model Implied Volatility Curves

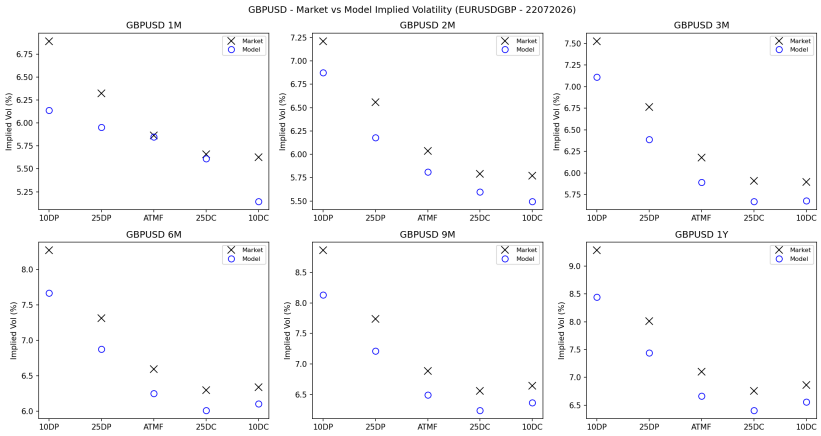


Figure 1: Model Implied volatility compared to Market Implied volatility on the GBPUSD pair

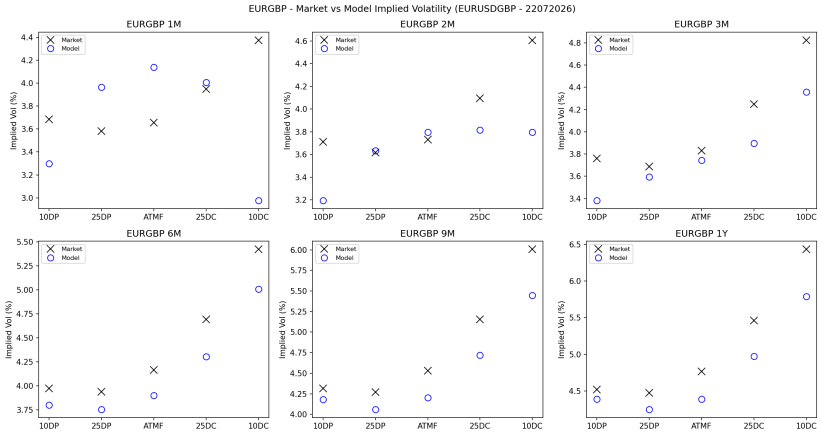


Figure 2: Model Implied volatility compared to Market Implied volatility on the EURGBP pair

.7 Multi Heston Option 2 - Market vs Model Implied Volatility Curves

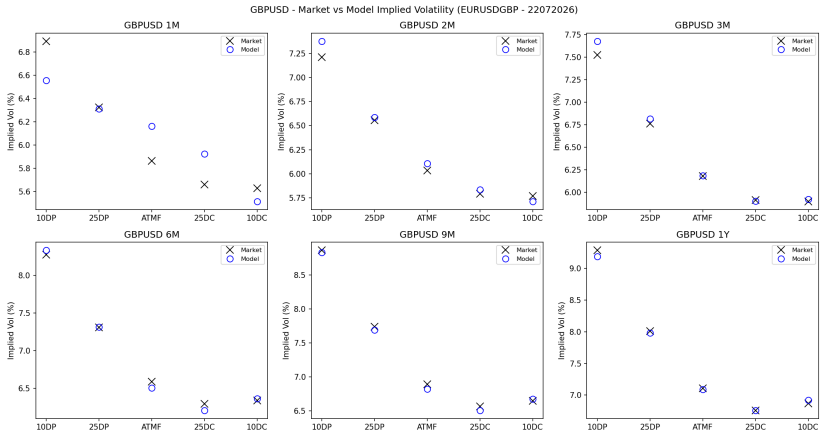


Figure 3: Model Implied volatility compared to Market Implied volatility on the GBPUSD pair - Multi Tenor Calibration

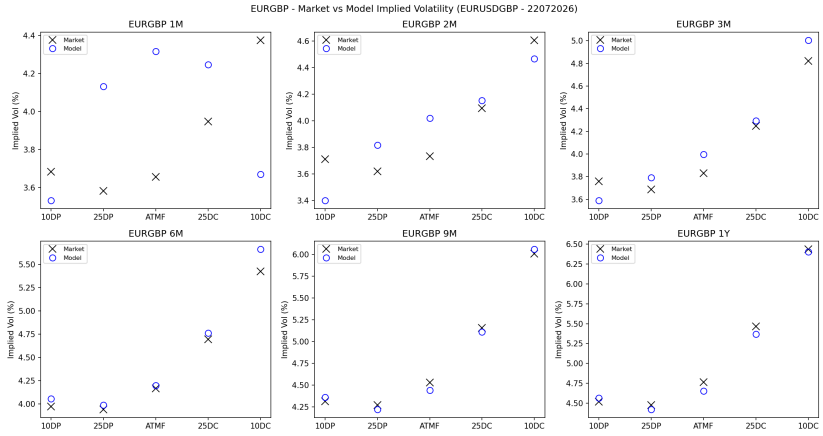


Figure 4: Model Implied volatility compared to Market Implied volatility on the EURGBP pair - Multi Tenor Calibration

.8 Historical Datasets

Table 7: Implied volatility quotes, as of 26 September 2022 (the Truss mini-budget crisis), for the EUR/USD/GBP triangle, across six tenors and five delta conventions (ATMF, 25-delta call/put, 10-delta call/put). Vol figures in percent (mid). Forward rates vary by tenor, see strike column for the ATMF level at each tenor.

		EUR/USD		GBP/USD		EUR/GBP	
Tenor	Quote	Vol	Strike	Vol	Strike	Vol	Strike
1 Month	10DF Put	15.475	0.91034	26.900	0.97113	16.3313	0.84851
	25DF Put	14.200	0.93754	24.104	1.02296	16.1262	0.87347
	ATMF	13.025	0.96301	21.525	1.06952	16.6800	0.90036
	25DF Call	12.500	0.98738	20.241	1.11441	18.2637	0.93425
	10DF Call	12.455	1.00908	20.350	1.15514	20.2338	0.97200
2 Month	10DF Put	15.585	0.89138	24.775	0.94448	14.4525	0.83795
	25DF Put	14.126	0.92987	21.971	1.01097	14.2625	0.86892
	ATMF	12.900	0.96515	19.395	1.06970	14.7800	0.90220
	25DF Call	12.319	0.99971	18.044	1.12725	16.3975	0.94598
	10DF Call	12.270	1.03043	18.105	1.17920	18.3625	0.99596
3 Month	10DF Put	15.415	0.87893	23.823	0.92475	13.5962	0.83076
	25DF Put	13.845	0.92539	20.834	1.00250	13.2725	0.86661
	ATMF	12.540	0.96730	18.295	1.06965	13.7650	0.90429
	25DF Call	11.935	1.00882	16.911	1.13648	15.3575	0.95517
	10DF Call	11.855	1.04548	17.008	1.19713	17.4887	1.01540
6 Month	10DF Put	14.833	0.85635	21.603	0.88935	12.1050	0.81947
	25DF Put	13.130	0.91900	18.633	0.98674	11.7175	0.86457
	ATMF	11.685	0.97418	16.145	1.06912	12.1425	0.91113
	25DF Call	11.050	1.03003	14.798	1.15359	13.7375	0.97743
	10DF Call	10.938	1.07890	14.788	1.22913	15.8850	1.05886
9 Month	10DF Put	14.450	0.84083	20.401	0.86373	11.6750	0.81086
	25DF Put	12.704	0.91487	17.601	0.97348	11.1813	0.86432
	ATMF	11.203	0.97939	15.158	1.06643	11.5300	0.91834
	25DF Call	10.551	1.04601	13.834	1.16451	13.0988	0.99776
	10DF Call	10.425	1.10402	13.834	1.25236	15.1150	1.09541
1 Year	10DF Put	14.398	0.82672	19.978	0.83967	11.2450	0.80632
	25DF Put	12.524	0.91140	17.123	0.96126	10.8462	0.86522
	ATMF	10.943	0.98399	14.640	1.06324	11.1950	0.92544
	25DF Call	10.246	1.05995	13.328	1.17362	12.7887	1.01709
	10DF Call	10.148	1.12643	13.233	1.27082	14.8950	1.13257

Table 8: Implied volatility quotes, as of 24 October 2008 (peak of the global financial crisis), for the EUR/USD/GBP triangle, across six tenors and five delta conventions (ATMF, 25-delta call/put, 10-delta call/put). Vol figures in percent (mid). Forward rates vary by tenor, see strike column for the ATMF level at each tenor.

		EUR/USD		GBP/USD		EUR/GBP	
Tenor	Quote	Vol	Strike	Vol	Strike	Vol	Strike
1 Month	10DF Put	29.855	1.13397	32.3625	1.41439	17.5087	0.74499
	25DF Put	27.125	1.20046	29.4875	1.50421	17.0712	0.76881
	ATMF	24.9925	1.26170	27.0600	1.58734	17.2000	0.79383
	25DF Call	24.265	1.32599	25.8725	1.67402	17.8788	0.82304
	10DF Call	24.705	1.38596	25.9775	1.75238	18.8112	0.85229
2 Month	10DF Put	27.5088	1.09935	29.3412	1.36985	15.9000	0.73260
	25DF Put	24.7313	1.18452	26.3738	1.48322	15.4075	0.76299
	ATMF	22.6125	1.26154	23.9725	1.58572	15.5375	0.79448
	25DF Call	21.9688	1.34562	22.8863	1.69625	16.2525	0.83267
	10DF Call	22.6313	1.42619	23.1187	1.79759	17.2600	0.87172
3 Month	10DF Put	25.7163	1.07857	27.0325	1.34459	14.6275	0.72590
	25DF Put	22.8763	1.17533	23.9925	1.47176	14.0950	0.76007
	ATMF	20.8525	1.26131	21.6200	1.58435	14.2100	0.79510
	25DF Call	20.3737	1.35805	20.6475	1.70768	14.9550	0.83857
	10DF Call	21.3138	1.45413	21.0475	1.82317	16.0325	0.88396
6 Month	10DF Put	22.7375	1.03932	24.0375	1.28823	13.2312	0.71047
	25DF Put	19.7288	1.15872	20.8975	1.44470	12.6900	0.75367
	ATMF	17.8075	1.26072	18.6050	1.57878	12.7700	0.79748
	25DF Call	17.5013	1.38100	17.7425	1.73176	13.5900	0.85482
	10DF Call	18.7625	1.50758	18.3525	1.88020	14.7387	0.91640
9 Month	10DF Put	21.6925	1.00852	22.9575	1.24568	12.6300	0.69865
	25DF Put	18.6687	1.14524	19.7513	1.42472	12.0737	0.74865
	ATMF	16.8275	1.26060	17.4500	1.57572	12.1450	0.79898
	25DF Call	16.6013	1.40340	16.6387	1.75468	12.9962	0.86747
	10DF Call	17.9675	1.55754	17.3125	1.93112	14.2300	0.94281
1 Year	10DF Put	20.8863	0.98616	22.0288	1.21575	12.2938	0.68879
	25DF Put	17.8412	1.13598	18.8813	1.41041	11.7138	0.74455
	ATMF	16.0225	1.26102	16.5650	1.57367	11.7725	0.80026
	25DF Call	15.8887	1.42150	15.8088	1.77275	12.6563	0.87858
	10DF Call	17.3538	1.59900	16.3913	1.96778	13.9662	0.96649

Table 9: Implied volatility quotes, as of 22 July 2026, for the AUD/USD/JPY triangle, across six tenors and five delta conventions (ATMF, 25-delta call/put, 10-delta call/put). Vol figures in percent (mid). Forward rates vary by tenor, see strike column for the ATMF level at each tenor.

		AUD/USD		USD/JPY		AUD/JPY	
Tenor	Quote	Vol	Strike	Vol	Strike	Vol	Strike
1 Month	10DF Put	8.0870	0.6789	8.7585	157.60	9.9900	109.693
	25DF Put	7.3955	0.6892	7.3025	160.41	8.3730	111.910
	ATMF	6.8750	0.6990	6.2080	162.65	7.1630	113.704
	25DF Call	6.6680	0.7079	6.0180	164.52	6.6880	115.160
	10DF Call	6.7020	0.7162	6.4030	166.47	6.6910	116.495
2 Month	10DF Put	8.4335	0.6683	8.7625	155.24	10.3835	107.590
	25DF Put	7.6345	0.6839	7.5360	159.02	8.8075	110.730
	ATMF	7.0530	0.6986	6.6210	162.24	7.5710	113.355
	25DF Call	6.8405	0.7120	6.4790	165.07	7.0650	115.513
	10DF Call	6.8840	0.7246	6.8845	168.02	7.0745	117.511
3 Month	10DF Put	8.7995	0.6601	8.8290	152.94	10.8780	105.429
	25DF Put	7.9155	0.6798	7.7090	157.64	9.2560	109.516
	ATMF	7.2760	0.6982	6.9140	161.81	8.0045	112.995
	25DF Call	7.0605	0.7152	6.8355	165.60	7.5030	115.907
	10DF Call	7.1070	0.7311	7.2730	169.61	7.5375	118.649
6 Month	10DF Put	9.6205	0.6397	9.0685	148.02	11.7720	100.799
	25DF Put	8.5520	0.6695	8.0960	154.50	10.0455	106.757
	ATMF	7.7990	0.6973	7.5260	160.57	8.7440	111.984
	25DF Call	7.5640	0.7230	7.6385	166.55	8.2675	116.505
	10DF Call	7.6970	0.7483	8.2315	173.16	8.4315	120.987
9 Month	10DF Put	10.3330	0.6221	9.2920	143.97	12.5495	96.833
	25DF Put	9.1340	0.6604	8.4090	151.76	10.7045	104.310
	ATMF	8.2770	0.6965	7.9700	159.38	9.3065	111.010
	25DF Call	8.0315	0.7300	8.2175	167.24	8.8400	116.913
	10DF Call	8.2005	0.7638	8.9425	176.26	9.0480	122.916
1 Year	10DF Put	10.8220	0.6073	9.4260	140.50	13.0685	93.483
	25DF Put	9.5360	0.6526	8.6060	149.34	11.1150	102.189
	ATMF	8.6240	0.6957	8.2575	158.22	9.6825	110.094
	25DF Call	8.3690	0.7363	8.5735	167.67	9.2400	117.199
	10DF Call	8.5725	0.7778	9.3985	178.84	9.5115	124.626

Table 10: Implied volatility quotes, as of 13 June 2005 (a calm, pre-crisis baseline), for the EUR/USD/GBP triangle, across five tenors and five delta conventions (ATMF, 25-delta call/put, 10-delta call/put). Vol figures in percent (mid). Forward rates vary by tenor; see strike column for the ATMF level at each tenor. The 9-month tenor was unavailable for this date.

		EUR/USD		GBP/USD		EUR/GBP	
Tenor	Quote	Vol	Strike	Vol	Strike	Vol	Strike
1 Month	10DF Put	10.525	1.16625	9.0375	1.74577	6.2250	0.65653
	25DF Put	9.975	1.18918	8.5500	1.77526	5.8375	0.66422
	ATMF	9.525	1.21200	8.1500	1.80451	5.5750	0.67171
	25DF Call	9.375	1.23477	8.0500	1.83352	5.5375	0.67908
	10DF Call	9.425	1.25546	8.1625	1.86035	5.6750	0.68606
2 Month	10DF Put	10.525	1.14935	9.1625	1.71925	6.2625	0.65172
	25DF Put	9.950	1.18148	8.6250	1.76120	5.8625	0.66262
	ATMF	9.525	1.21330	8.2500	1.80241	5.6000	0.67321
	25DF Call	9.400	1.24603	8.1750	1.84447	5.5875	0.68382
	10DF Call	9.525	1.27626	8.3375	1.88386	5.7875	0.69410
3 Month	10DF Put	10.500	1.13731	9.1875	1.69947	6.3375	0.64818
	25DF Put	9.925	1.17624	8.6250	1.75063	5.9625	0.66157
	ATMF	9.525	1.21479	8.3000	1.80063	5.7500	0.67471
	25DF Call	9.475	1.25564	8.2750	1.85317	5.7875	0.68829
	10DF Call	9.650	1.29378	8.5625	1.90393	6.0125	0.70152
6 Month	10DF Put	10.4125	1.11311	9.1750	1.65676	6.3625	0.64168
	25DF Put	9.8375	1.16685	8.6625	1.72713	6.0250	0.66044
	ATMF	9.5500	1.21994	8.4000	1.79661	5.8750	0.67908
	25DF Call	9.6125	1.28013	8.4875	1.87420	5.9750	0.69933
	10DF Call	9.9875	1.33884	8.8250	1.94998	6.2375	0.71927
1 Year	10DF Put	10.3625	1.08439	9.2750	1.59825	6.3125	0.63519
	25DF Put	9.8375	1.15828	8.7250	1.69627	6.0750	0.66096
	ATMF	9.5500	1.23177	8.5000	1.79225	5.9750	0.68734
	25DF Call	9.6625	1.32087	8.6250	1.90668	6.1750	0.71794
	10DF Call	10.0375	1.40794	9.0250	2.02022	6.4875	0.74850

.9 Exotic Option Prices Across Historical Scenarios

Tables 11-15 extend the comparison of Chapter 4 to four additional historical dates - a calm pre-crisis baseline (13 June 2005), the 2008 global financial crisis (24 October 2008), and the 2022 UK mini-budget crisis (26 September 2022) - alongside the original 22 July 2026 baseline, for all three methods (Sinkhorn, Option 1, and Option 2).

Table 11: EUR/USD/GBP, 13 June 2005 (calm baseline): exotic option prices, all three methods. No 9-month tenor was available for this date.

Tenor	Method	Basket	Quanto	Best-of	Quadratic
1M	Sinkhorn	0.009658	0.006448	0.013429	2.820×10^{-4}
	Option 1	0.009411	0.006203	0.013188	2.590×10^{-4}
	Option 2	0.009676	0.006525	0.013623	2.890×10^{-4}
2M	Sinkhorn	0.013720	0.009191	0.019091	5.760×10^{-4}
	Option 1	0.013262	0.008824	0.018631	5.260×10^{-4}
	Option 2	0.013564	0.009269	0.019214	5.960×10^{-4}
3M	Sinkhorn	0.016790	0.011594	0.023564	9.060×10^{-4}
	Option 1	0.016355	0.011280	0.022592	8.350×10^{-4}
	Option 2	0.016627	0.011477	0.023661	9.200×10^{-4}
6M	Sinkhorn	0.023856	0.016883	0.033666	1.911×10^{-3}
	Option 1	0.022822	0.016126	0.032627	1.738×10^{-3}
	Option 2	0.023842	0.016806	0.034145	1.941×10^{-3}
1Y	Sinkhorn	0.033835	0.024639	0.048005	3.937×10^{-3}
	Option 1	0.033082	0.023606	0.046368	3.597×10^{-3}
	Option 2	0.034775	0.024998	0.050034	4.133×10^{-3}

Table 12: EUR/USD/GBP, 22 July 2026 (baseline): exotic option prices, all three methods.

Tenor	Method	Basket	Quanto	Best-of	Quadratic
1M	Sinkhorn	0.005868	0.004316	0.008376	1.310×10^{-4}
	Option 1	0.005635	0.004050	0.008163	1.120×10^{-4}
	Option 2	0.005982	0.004365	0.008729	1.290×10^{-4}
2M	Sinkhorn	0.008553	0.006299	0.012155	2.780×10^{-4}
	Option 1	0.008313	0.005922	0.011397	2.330×10^{-4}
	Option 2	0.008535	0.006377	0.012606	2.840×10^{-4}
3M	Sinkhorn	0.010692	0.008012	0.015213	4.470×10^{-4}
	Option 1	0.010387	0.007420	0.014201	3.640×10^{-4}
	Option 2	0.010633	0.008097	0.015838	4.630×10^{-4}
6M	Sinkhorn	0.015986	0.012644	0.022910	1.082×10^{-3}
	Option 1	0.015398	0.011422	0.021106	8.420×10^{-4}
	Option 2	0.015898	0.012737	0.024059	1.139×10^{-3}
9M	Sinkhorn	0.020482	0.017150	0.029795	1.933×10^{-3}
	Option 1	0.019695	0.015403	0.027301	1.489×10^{-3}
	Option 2	0.020554	0.017049	0.031415	1.991×10^{-3}
1Y	Sinkhorn	0.024443	0.021158	0.035824	2.880×10^{-3}
	Option 1	0.023492	0.018741	0.032690	2.177×10^{-3}
	Option 2	0.024830	0.021060	0.038140	2.970×10^{-3}

Table 13: EUR/USD/GBP, 24 October 2008 (financial crisis): exotic option prices, all three methods.

Tenor	Method	Basket	Quanto	Best-of	Quadratic
1M	Sinkhorn	0.028217	0.021086	0.040325	2.681×10^{-3}
	Option 1	0.026761	0.020035	0.039200	2.468×10^{-3}
	Option 2	0.027750	0.020179	0.040235	2.494×10^{-3}
2M	Sinkhorn	0.035644	0.027573	0.051217	4.516×10^{-3}
	Option 1	0.033378	0.025733	0.049438	4.051×10^{-3}
	Option 2	0.035768	0.026871	0.052634	4.462×10^{-3}
3M	Sinkhorn	0.039741	0.031302	0.057257	5.811×10^{-3}
	Option 1	0.038368	0.028451	0.052462	4.699×10^{-3}
	Option 2	0.041073	0.031332	0.060739	6.102×10^{-3}
6M	Sinkhorn	0.047740	0.041204	0.070191	9.463×10^{-3}
	Option 1	0.045662	0.036755	0.063408	7.526×10^{-3}
	Option 2	0.052677	0.040772	0.077897	1.0124×10^{-2}
9M	Sinkhorn	0.054838	0.049497	0.081040	1.2977×10^{-2}
	Option 1	0.052362	0.043370	0.072983	1.0153×10^{-2}
	Option 2	0.062584	0.048259	0.091860	1.3691×10^{-2}
1Y	Sinkhorn	0.060004	0.056497	0.089383	1.6306×10^{-2}
	Option 1	0.057455	0.048961	0.080560	1.2706×10^{-2}
	Option 2	0.071367	0.055197	0.104442	1.7327×10^{-2}

Table 14: EUR/USD/GBP, 26 September 2022 (UK mini-budget crisis): exotic option prices, all three methods.

Tenor	Method	Basket	Quanto	Best-of	Quadratic
1M	Sinkhorn	0.018132	0.021221	0.029593	2.653×10^{-3}
	Option 1	0.017460	0.019856	0.027564	2.279×10^{-3}
	Option 2	0.018873	0.020198	0.028992	2.288×10^{-3}
2M	Sinkhorn	0.023968	0.027404	0.038495	4.253×10^{-3}
	Option 1	0.023012	0.025116	0.035462	3.517×10^{-3}
	Option 2	0.025001	0.027243	0.037931	4.013×10^{-3}
3M	Sinkhorn	0.028112	0.032137	0.044763	5.720×10^{-3}
	Option 1	0.026892	0.028930	0.040840	4.544×10^{-3}
	Option 2	0.029310	0.032370	0.044215	5.457×10^{-3}
6M	Sinkhorn	0.035853	0.041987	0.056836	9.090×10^{-3}
	Option 1	0.034020	0.036820	0.051015	6.952×10^{-3}
	Option 2	0.038473	0.044215	0.057987	9.188×10^{-3}
9M	Sinkhorn	0.041418	0.050285	0.066045	1.2290×10^{-2}
	Option 1	0.039323	0.043819	0.059110	9.357×10^{-3}
	Option 2	0.045893	0.053976	0.069445	1.2658×10^{-2}
1Y	Sinkhorn	0.046428	0.058364	0.074120	1.5399×10^{-2}
	Option 1	0.044091	0.049612	0.065948	1.1527×10^{-2}
	Option 2	0.052472	0.063102	0.079749	1.6245×10^{-2}

Table 15: AUD/USD/JPY, 22 July 2026 (baseline): exotic option prices, all three methods.

Tenor	Method	Basket	Quanto	Best-of	Quadratic
1M	Sinkhorn	0.006331	0.008354	0.011624	5.250×10^{-4}
	Option 1	0.003794	0.012541	0.013271	1.027×10^{-3}
	Option 2	0.004289	0.013136	0.014328	1.101×10^{-3}
2M	Sinkhorn	0.009323	0.012557	0.017296	1.168×10^{-3}
	Option 1	0.005658	0.018569	0.019619	2.209×10^{-3}
	Option 2	0.006616	0.019315	0.021006	2.398×10^{-3}
3M	Sinkhorn	0.011786	0.016348	0.022138	1.934×10^{-3}
	Option 1	0.007481	0.022662	0.024131	3.372×10^{-3}
	Option 2	0.008715	0.024470	0.026600	3.864×10^{-3}
6M	Sinkhorn	0.017899	0.025674	0.033905	4.644×10^{-3}
	Option 1	0.011449	0.035632	0.037197	7.848×10^{-3}
	Option 2	0.014047	0.038345	0.041039	9.284×10^{-3}
9M	Sinkhorn	0.023174	0.033945	0.044052	7.929×10^{-3}
	Option 1	0.014871	0.047215	0.048665	1.3304×10^{-2}
	Option 2	0.018482	0.051188	0.053701	1.5888×10^{-2}
1Y	Sinkhorn	0.027721	0.041354	0.052836	1.1531×10^{-2}
	Option 1	0.017858	0.057562	0.058588	1.9188×10^{-2}
	Option 2	0.022430	0.063512	0.065350	2.3634×10^{-2}

Appendix A

Code: Task 1-3 Sinkhorn Method Implementation

This appendix contains the full Python code underlying the entropic optimal transport methodology described in Chapter 2, covering SVI calibration, Breeden-Litzenberger density recovery, the Sinkhorn joint density construction, the consistency check against market smiles, and the exotic option pricing of Task 3.

A.1 SVI Calibration

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import differential_evolution

# SVI Calibration

# Data - Provided within the brief.

T = 1/12

EURUSD_K = np.array([1.0681, 1.0791, 1.0904, 1.1014, 1.1119])
EURUSD_IV = np.array([0.0554, 0.053115, 0.0516, 0.051435,
    0.0523])

```

```
GBPUSD_K = np.array([1.2456, 1.2595, 1.2740, 1.2883, 1.3017])
GBPUSD_IV = np.array([0.06055, 0.058665, 0.0573, 0.057185,
    0.05765])

EURGBP_K = np.array([0.84386, 0.84969, 0.85585, 0.86234,
    0.86875])
EURGBP_IV = np.array([0.03809, 0.03725, 0.037225, 0.03825,
    0.03986])

# Creating our dictionary

Market_Data = {
    "EURUSD" : {
        "F_0" : 1.0903,
        "K" : EURUSD_K,
        "x" : np.log(EURUSD_K/1.0903),
        "IV" : EURUSD_IV
    },
    "GBPUSD" : {
        "F_0" : 1.2738,
        "K" : GBPUSD_K,
        "x" : np.log(GBPUSD_K/1.2738),
        "IV" : GBPUSD_IV
    },
    "EURGBP" : {
        "F_0" : 0.8559,
        "K" : EURGBP_K,
        "x" : np.log(EURGBP_K/0.8559),
        "IV" : EURGBP_IV
    }
}

# Black-76 Forward Formula . (Given the data we have)

def BlackScholesEuropeanCallOption(sigma, T, F_0, K):
    d1 = (np.log(F_0/K) + T*((sigma**2)/2))/(sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    return F_0*norm.cdf(d1) - K*norm.cdf(d2)

# Gatheral's equation for w, Total Variance.

def w(Params,x):
    a,b,rho,m,sigma = Params
```

```
56      return a + b*(rho*(x-m) + np.sqrt((x-m)**2 + sigma**2))
57
58  # Inverting another equation for Total Variance to find what
        sigma is equal to in Gatheral's SVI
59
60  def Volfrom_w(Params,x,T):
61      return np.sqrt(w(Params,x)/T)
62
63  # Our objective function that we least to minimize via a Least-
        Squares method.
64
65  def LeastSquaresError(Params,x,T,Market_IV,
        RegularisationForPenalty):
66
67      a,b,rho,m,sigma = Params
68
69      Model_IV = Volfrom_w(Params,x,T)
70
71      Error = np.sum((Model_IV - Market_IV)**2)
72
73      Penalty = 0.0
74      if b*(1+np.abs(rho)) > 4/T:
75          Penalty += b*(1+np.abs(rho)) - 4/T
76
77      return Error + (RegularisationForPenalty*Penalty)
78
79  # The function which calibrates the parameter values (with bounds
        )
80
81  def CalibrationTime(x,T,Market_IV,reruns,RegularisationForPenalty
        ):
82      x_range = x.max() - x.min()
83      bounds = [
84          (0.0,0.5),            #a - controls vertical shift
85          (1e-6,2.0),           #b - controls the slope of the wings
                (must be positive)
86          (-0.999,0.999),       #rho - skew parameter
87          (x.min() - x_range,x.max() + x_range), #m - horizontal
               shift of the curve's min
88          (1e-4,x_range)        #sigma - curvature of the smile
89      ]
90
```

```
    seed = range(reruns) # set the seeds so its repeatable, but
        changes initial population.

    BestParameters = None
    BestError = np.inf

    for s in seed:
        results = differential_evolution(
            LeastSquaresError,
            args=(x,T,Market_IV,RegularisationForPenalty),
            bounds=bounds,
            seed=s,
            tol=1e-10,
            polish=True # Final polish to find perfect minimum
        )
        if results.fun < BestError:
            BestParameters = results.x
            BestError = results.fun

    return BestParameters

RegularisationForPenalty = 1e3

CalibratedPairs = {}

# Calibrating each pair using the market data, plus printing
    results.

for Pair,Value in Market_Data.items():
    parameters = CalibrationTime(Value["x"],T,Value["IV"],reruns
        =5,RegularisationForPenalty=RegularisationForPenalty)
    CalibratedPairs[Pair] = parameters
ParameterTable = pd.DataFrame(CalibratedPairs,index=["a","b","rho
    ","m","sigma"]).T
print(ParameterTable)

# RMS Error for each pair

print()
for Pair2, Value2 in Market_Data.items():
    params = CalibratedPairs[Pair2]
    Calibrated_IV = Volfrom_w(params,Value2["x"],T)
```

```
    print(f"{Pair2} RMS Error : ", np.sqrt(np.mean((Calibrated_IV
        - Value2["IV"])**2)))
```

A.2 SVI Fit Plots

```
fig, axes = plt.subplots(1,3,figsize=(20,10))

# Plotting our SVI curves vs the Market Discrete data.

names = ["EURUSD", "GBPUSD", "EURGBP"]

for i,name in enumerate(names):
    F_0 = Market_Data[name]["F_0"]
    K_Market = Market_Data[name]["K"]
    IV_Market = Market_Data[name]["IV"]
    K_grid = np.linspace(K_Market.min(),K_Market.max(),400)
    x_grid = np.log(K_grid/F_0)
    Parameters = CalibratedPairs[name]
    SVI_Fit = Volfrom_w(Parameters,x_grid,T)

    axes[i].plot(K_grid,SVI_Fit, "b-",label="SVI Fit")
    axes[i].plot(K_Market,IV_Market,"ro",label="Market IV")
    axes[i].set_xlabel("Strike K",fontsize=15)
    axes[i].set_ylabel("Implied Volatility",fontsize=15)
    axes[i].set_title(f"{name}",fontsize=20)
    axes[i].legend(fontsize=15)
    axes[i].tick_params(axis='both', labelsize=15)
plt.tight_layout()
plt.savefig("SVI_Fit.png",dpi=200)
plt.show()
```

A.3 Breeden-Litzenberger Density Recovery

```
# Now a function to create a Euro Call Price from our smooth SVI
    curves

def CallPrice2(params,T,F_0,K):
    x = np.log(K/F_0)
```

```
    sigma = Volfrom_w(params,x,T)
    return BlackScholesEuropeanCallOption(sigma, T, F_0, K)

# Second Central Difference equation

def CentralSecondDifference(params,T,F_0,K,h=1e-3):
    CKplush = CallPrice2(params,T,F_0,K+h)
    CKminush = CallPrice2(params,T,F_0,K-h)
    TwoCK = 2*CallPrice2(params,T,F_0,K)
    return (CKplush + CKminush - TwoCK)/h**2

fig, axes = plt.subplots(1,3,figsize=(20,10))

# Sanity Checks and Plots for each

names = ["EURUSD", "GBPUSD", "EURGBP"]
print("Density Sanity Checks (4 s.f.):")
for i, Pair in enumerate(names):
    F_0 = Market_Data[Pair]["F_0"]
    x = Market_Data[Pair]["x"]
    K_grid = np.linspace(0.9*F_0, 1.1*F_0, 4000)
    Parameters = CalibratedPairs[Pair]

    Density = CentralSecondDifference(Parameters,T,F_0,K_grid)

    print(f"{Pair} : ")
    print(f"Total Mass = {np.trapz(Density,K_grid):.4f}\t\t Mean
        = {np.trapz(Density*K_grid,K_grid):.4f} Should be F_0 = {
        F_0:.4f}\t\t Density Minimum : {Density.min():.4f}")

    axes[i].plot(K_grid,Density,"g-")
    axes[i].set_title(f"{Pair} Density Plot",fontsize=20)
    axes[i].set_xlabel("Strike K",fontsize=15)
    axes[i].set_ylabel("Density",fontsize=15)
    axes[i].tick_params(axis='both', labelsize=15)

plt.tight_layout()
plt.savefig("Density_Plots.png")
plt.show()
```

A.4 Normalised Density Plots

```
# Plotting Densities Normalised

Lower = 0.9
Upper = 1.1

def BL_Density(name,k):
    Density = CentralSecondDifference(CalibratedPairs[name],T
        ,1.0,k)
    Density[(k<Lower) | (k>Upper)] = 0.0
    return Density

fig, axes = plt.subplots(1,3,figsize=(20,10))

names = ["EURUSD", "GBPUSD", "EURGBP"]

for i, Pair in enumerate(names):
    F_0 = Market_Data[Pair]["F_0"]
    x = Market_Data[Pair]["x"]
    K_grid2 = np.linspace(0.9, 1.1, 4000)

    Density2 = BL_Density(Pair,K_grid2)

    print(f"{Pair} : ")
    print(f"Total Mass = {np.trapz(Density,K_grid):.4f}\t\t Mean
        = {np.trapz(Density*K_grid,K_grid):.4f} Should be F_0 = {
        F_0:.4f}\t\t Density Minimum : {Density.min():.4f}")

    axes[i].plot(K_grid2,Density2,"g-")
    axes[i].set_title(f"{Pair} Density Plot",fontsize=20)
    axes[i].set_xlabel("Normalised Strike K",fontsize=15)
    axes[i].set_ylabel("Density",fontsize=15)
    axes[i].tick_params(axis='both',labelsize=15)
plt.tight_layout()
plt.savefig("Normalised_Density_Plots.png")
plt.show()

# Plotting Densities Normalised (Overlaid)

fig, ax = plt.subplots(1,1,figsize=(10,6))

colours = ["g", "b", "r"]
```

```

for i, Pair in enumerate(names):
    F_0 = Market_Data[Pair]["F_0"]
    x = Market_Data[Pair]["x"]
    K_grid2 = np.linspace(0.9, 1.1, 4000)

    Density2 = BL_Density(Pair,K_grid2)

    ax.plot(K_grid2,Density2,color=colours[i],label=Pair)

ax.set_title("Normalised Density Plots (Overlaid)")
ax.set_xlabel("Normalised Strike K")
ax.set_ylabel("Density")
ax.legend()
plt.savefig("Overlaid_Normalised_Density_Plots.png")
plt.show()
```

A.5 Sinkhorn Joint Density Construction

```
from numpy.polynomial.legendre import leggauss
from scipy.optimize import bisect, brentq
from scipy.interpolate import CubicSpline

# Sinkhorn Setup

Lower = 0.9
Upper = 1.1

# Note we already have the normalised density function

# Creating our G-L nodes

def Gauss_Legendre_Setup(b,a,n):
    nodes, weights = leggauss(n)
    nodes = 0.5*(a-b)*nodes + 0.5*(a+b)
    weights = 0.5*(a-b)*weights
    return nodes,weights

N = 800 # Smoother Output
x_nodes, x_weights = Gauss_Legendre_Setup(Lower,Upper,N)
y_nodes, y_weights = x_nodes.copy(), x_weights.copy()
```

```
23
24 ratioXY = x_nodes[:,None] / y_nodes[None,:]        # [:,None]
       turns the xnodes into a column vector of shape (N,1)
25                                                    # [None,:]
                                                          turns the
                                                          ynodes into
                                                           a row
                                                          vector of
                                                          shape (1,N)
26                                                    # This overall
                                                           creates an
                                                           x,y matrix
27
28
29 # EURUSD and GBPUSD marginal densities on the x and y nodes
30
31 MuX = BL_Density("EURUSD",x_nodes)
32 MuY = BL_Density("GBPUSD",y_nodes)
33
34 N_z = 100
35 z_nodes = np.linspace(0.9,1.1, N_z)
36 MuZ = BL_Density("EURGBP", z_nodes)
37
38
39 u = np.zeros(N)
40 v = np.zeros(N)
41 w_values = np.zeros(N_z)
42
43 for iter in range(40):
44
45     w_spline = CubicSpline(z_nodes,w_values) # Smooth function
46     W = w_spline(np.clip(ratioXY,z_nodes[0],z_nodes[-1])) # So no
           extropolation
47
48     # From the Sinkhorn Equations
49
50     u_exponent = v[None,:] + y_nodes[None,:] * W
51     u = -np.log(np.sum(y_weights[None,:] * np.exp(u_exponent) *
           MuY[None,:],axis=1))
52
53     v_exponent = u[:,None] + y_nodes[None,:] * W
54     v = -np.log(np.sum(x_weights[:,None] * np.exp(v_exponent) *
           MuX[:,None], axis=0))
```

```
55
56      v_spline = CubicSpline(y_nodes,v)
57
58      for i in range(N_z):
59          z = z_nodes[i]
60          xoverz = x_nodes/z # x/z at every x node
61          v_xoverz = v_spline(np.clip(xoverz,y_nodes[0],y_nodes[-1])
                ) # interpolate so we can evaluate v at each xnode/z
62          MuY_xoverz = BL_Density("GBPUSD",xoverz)
63
64          def target(w_guess):
65              return np.sum((np.exp(u + v_xoverz + xoverz*w_guess) *
                     (x_nodes**2/z**3) * MuX * MuY_xoverz * x_weights)
                   ) - MuZ[i]
66
67          low, high = -60.0,60.0 # stops exp(w...) overflowing over
                the limit and gives the bisect an interval to search
                within.
68
69          if target(low) > 0:
70              w_values[i] = low
71          elif target(high) < 0:
72              w_values[i] = high
73          else:
74              w_values[i] = bisect(target,low,high)
75
76  # Recovering the Joint Density
77
78  w_spline = CubicSpline(z_nodes,w_values)
79  W = w_spline(np.clip(ratioXY,z_nodes[0],z_nodes[-1]))
80
81  joint_exponent = u[:,None] + v[None,:] + y_nodes[None,:]*W
82  joint_weights = x_weights[:,None] * y_weights[None,:]
83  joint_density = np.exp(joint_exponent) * MuX[:,None] * MuY[None
        ,:]
84  joint_integral = np.sum(joint_density * joint_weights)
85
86  print(f"Joint Density mass = {joint_integral}")
87
88  recovered_MuX = np.sum(joint_density * y_weights[None,:], axis=1)
89  recovered_MuY = np.sum(joint_density * x_weights[:,None], axis=0)
90
```

```
print(f"Max X-marginal error = {np.max(np.abs(recovered_MuX - MuX
    )):.6f}")
print(f"Max Y-marginal error = {np.max(np.abs(recovered_MuY - MuY
    )):.6f}")

frac_clipped = np.mean((ratioXY < z_nodes[0]) | (ratioXY >
    z_nodes[-1]))
print(f"Fraction of (x,y) pairs with clipped w = {frac_clipped
    :.2%}")
```

A.6 Consistency Check Against Market Smiles

```
# Finding the implied vol as curve from each of the marginal's of
    our joint density

def implied_volatility(price,K):

    intrisic_value = max(1.0 - K, 0.0)

    if price <= intrisic_value + 1e-12:
        return 0.0

    def target2(vol):
        model_price = BlackScholesEuropeanCallOption(vol, T, 1.0,
            K)
        return model_price - price


    return brentq(target2, 1e-6, 5.0)


def model_call(kind, K):
    if kind == "X":
        return np.sum(joint_weights * np.maximum(x_nodes[:, None]
            - K, 0.0) * joint_density)
    elif kind == "Y":
        return np.sum(joint_weights * np.maximum(y_nodes[None, :]
            - K, 0.0) * joint_density)
    else:
        return np.sum(joint_weights * y_nodes[None, :] *
```

```
                    np.maximum(x_nodes[:, None] / y_nodes[None,
                        :] - K, 0.0) * joint_density)

fig, axes = plt.subplots(1, 3, figsize=(20, 10))
print("Consistency check (model implied vol vs market):")

pairs = [("EURUSD", "X"), ("GBPUSD", "Y"), ("EURGBP", "Z")]
for i in range(3):
    name, kind = pairs[i]
    F_0 = Market_Data[name]["F_0"]
    strikes = Market_Data[name]["K"] / F_0 # normalised strikes
    market_iv = Market_Data[name]["IV"]

    # model vs market at the five market strikes
    print(f" {name}:")
    for j in range(len(strikes)):
        K = strikes[j]
        iv_model = implied_volatility(model_call(kind, K), K)
        print(f"    K = {K:.4f}  model {iv_model:.5f} market {
            market_iv[j]:.5f}")

    # model implied-vol curve over a fine strike grid
    K_grid2 = np.linspace(strikes.min(), strikes.max(), 50)
    model_iv = np.zeros(len(K_grid2))
    for j in range(len(K_grid2)):
        model_iv[j] = implied_volatility(model_call(kind, K_grid2[
            j]), K_grid2[j])

    axes[i].plot(K_grid2, model_iv * 100, "b-", label="model (
        from mu*)")
    axes[i].plot(strikes, market_iv * 100, "ro", label="market")
    axes[i].set_title(name,fontsize=20)
    axes[i].set_xlabel("normalised strike K / F_0",fontsize=15)
    axes[i].set_ylabel("implied volatility (%)",fontsize=15)
    axes[i].legend(fontsize=15)
    axes[i].tick_params(axis='both',labelsize=15)
plt.tight_layout()
plt.savefig("Consistency_check-MarketvsModelIV.png")
plt.show()
```

A.7 Potential Functions and Exotic Option Pricing

```
fig, axes = plt.subplots(1, 3, figsize=(20, 10))

axes[0].plot(x_nodes, u, "b-")
axes[0].set_title("u(x)",fontsize=20)
axes[0].set_xlabel("x",fontsize=15)

axes[1].plot(y_nodes, v, "r-")
axes[1].set_title("v(y)",fontsize=20)
axes[1].set_xlabel("y",fontsize=15)

axes[2].plot(z_nodes, w_values, "g-")
axes[2].set_title("w(z)",fontsize=20)
axes[2].set_xlabel("z = x / y",fontsize=15)

plt.tight_layout()
plt.savefig("uvw_plots.png")
plt.show()



X = x_nodes[:, None]
Y = y_nodes[None, :]

# The payoffs

basket   = np.maximum(0.5*(X + Y) - 1.0, 0.0) # ( 0.5(X+Y) - 1 )+
quanto   = np.maximum(X/Y - 1.0, 0.0)         # ( X/Y - 1 )+
best_of  = np.maximum(np.maximum(X, Y) - 1.0, 0.0) # ( max(X,Y) -
    1 )+
quadratic = (X - Y)**2                        # ( X - Y )^2

# The Expected Payoff

price_basket = np.sum(joint_weights * basket * joint_density)
price_quanto = np.sum(joint_weights * quanto * joint_density)
price_best  = np.sum(joint_weights * best_of * joint_density)
price_quad  = np.sum(joint_weights * quadratic * joint_density)

print("Option prices:")
```

```
print(f" basket    E[(0.5(X+Y)-1)+] = {price_basket:.6f}")
print(f" quanto    E[(X/Y-1)+]     = {price_quanto:.6f}")
print(f" best-of   E[(max(X,Y)-1)+] = {price_best:.6f}")
print(f" quadratic E[(X-Y)^2]      = {price_quad:.6e}")
```

Appendix B

Sinkhorn Method Applied to Multi-Tenors

B.1 Data Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import differential_evolution

# SVI Calibration - All Maturities

Tenors = ["1M", "2M", "3M", "6M", "9M", "1Y"]

T_by_Tenor = {
    "1M" : 1/12,
    "2M" : 2/12,
    "3M" : 3/12,
    "6M" : 6/12,
    "9M" : 9/12,
    "1Y" : 1.0
}

# Raw market data per tenor per pair. Strikes are ordered
    ascending
# F_0 is taken as the ATMF "Strike" value for that tenor/pair

```

```
RawMarketData = {
    "1M": {
        "EURUSD": {"F_0": 1.1425, "K": np.array([1.1192, 1.1311, 1.1425, 1.1531, 1.1630]),
                   "IV": np.array([5.727, 5.2845, 4.962, 4.8415, 4.897])},
        "GBPUSD": {"F_0": 1.3366, "K": np.array([1.3039, 1.3207, 1.3366, 1.3511, 1.3642]),
                   "IV": np.array([6.8935, 6.323, 5.864, 5.6605, 5.6285])},
        "EURGBP": {"F_0": 0.85455, "K": np.array([0.84327, 0.84875, 0.85455, 0.86099, 0.86819]),
                   "IV": np.array([3.685, 3.5825, 3.656, 3.9485, 4.3755])}
    },
    "2M": {
        "EURUSD": {"F_0": 1.1447, "K": np.array([1.1088, 1.1268, 1.1447, 1.1601, 1.1754]),
                   "IV": np.array([5.9395, 5.4545, 5.113, 5.015, 5.113])},
        "GBPUSD": {"F_0": 1.3367, "K": np.array([1.2868, 1.3125, 1.3367, 1.3584, 1.3782]),
                   "IV": np.array([7.21, 6.5565, 6.0345, 5.792, 5.771])},
        "EURGBP": {"F_0": 0.85564, "K": np.array([0.83908, 0.84708, 0.85564, 0.86544, 0.87679]),
                   "IV": np.array([3.7105, 3.6175, 3.733, 4.095, 4.607])}
    },
    "3M": {
        "EURUSD": {"F_0": 1.1455, "K": np.array([1.1019, 1.1242, 1.1455, 1.1655, 1.1849]),
                   "IV": np.array([6.06, 5.547, 5.199, 5.106, 5.235])},
        "GBPUSD": {"F_0": 1.3367, "K": np.array([1.2739, 1.3065, 1.3367, 1.3637, 1.3886]),
                   "IV": np.array([7.525, 6.762, 6.179, 5.9115, 5.896])},
        "EURGBP": {"F_0": 0.85673, "K": np.array([0.83632, 0.84611, 0.85673, 0.86915, 0.88385]),
                   "IV": np.array([3.7605, 3.6865, 3.8315, 4.2475, 4.8235])}
    },
```

```
    "6M": {
        "EURUSD": {"F_0": 1.1498, "K": np.array([1.0848, 1.1182,
            1.1498, 1.1799, 1.2107]),
                   "IV": np.array([6.4665, 5.854, 5.461, 5.4085,
                       5.6445])},
        "GBPUSD": {"F_0": 1.3366, "K": np.array([1.2409, 1.2908,
            1.3366, 1.3774, 1.4164]),
                   "IV": np.array([8.274, 7.312, 6.591, 6.2945,
                       6.3385])},
        "EURGBP": {"F_0": 0.86005, "K": np.array([0.82975,
            0.84405, 0.86005, 0.87957, 0.90373]),
                   "IV": np.array([3.972, 3.938, 4.163, 4.6915,
                       5.422])}
    },
    "9M": {
        "EURUSD": {"F_0": 1.1539, "K": np.array([1.0689, 1.1127,
            1.1539, 1.1936, 1.2356]),
                   "IV": np.array([6.966, 6.244, 5.793, 5.7725,
                       6.105])},
        "GBPUSD": {"F_0": 1.3362, "K": np.array([1.2126, 1.2773,
            1.3362, 1.3886, 1.4396]),
                   "IV": np.array([8.863, 7.7375, 6.889, 6.5615,
                       6.642])},
        "EURGBP": {"F_0": 0.86335, "K": np.array([0.82324,
            0.84211, 0.86335, 0.88981, 0.92349]),
                   "IV": np.array([4.314, 4.2715, 4.5295, 5.156,
                       6.008])}
    },
    "1Y": {
        "EURUSD": {"F_0": 1.1579, "K": np.array([1.0548, 1.1081,
            1.1579, 1.2064, 1.2598]),
                   "IV": np.array([7.376, 6.542, 6.059, 6.0655,
                       6.501])},
        "GBPUSD": {"F_0": 1.3356, "K": np.array([1.1881, 1.2656,
            1.3356, 1.3981, 1.46]),
                   "IV": np.array([9.287, 8.013, 7.104, 6.7585,
                       6.866])},
        "EURGBP": {"F_0": 0.86677, "K": np.array([0.81836,
            0.84105, 0.86677, 0.89939, 0.94217]),
                   "IV": np.array([4.521, 4.4765, 4.766, 5.4635,
                       6.434])}
    }
}
```

```
73
74
75  Market_Data = {}
76
77  for tenor in Tenors:
78      Market_Data[tenor] = {}
79      for Pair, Value in RawMarketData[tenor].items():
80          F_0 = Value["F_0"]
81          K = Value["K"]
82          IV = Value["IV"] / 100.0
83          Market_Data[tenor][Pair] = {
84              "F_0": F_0,
85              "K": K,
86              "x": np.log(K/F_0),
87              "IV": IV
88          }
```

B.2 Calibration

```
1   # Black-76 Forward Formula
2
3   def BlackScholesEuropeanCallOption(sigma, T, F_0, K):
4       d1 = (np.log(F_0/K) + T*((sigma**2)/2))/(sigma*np.sqrt(T))
5       d2 = d1 - sigma*np.sqrt(T)
6       return F_0*norm.cdf(d1) - K*norm.cdf(d2)
7
8   # Gatheral's equation for w, Total Variance.
9
10  def w(Params,x):
11      a,b,rho,m,sigma = Params
12      return a + b*(rho*(x-m) + np.sqrt((x-m)**2 + sigma**2))
13
14  # Inverting w to find sigma in Gatheral's SVI
15
16  def Volfrom_w(Params,x,T):
17      return np.sqrt(w(Params,x)/T)
18
19  # Objective function minimised via Least-Squares (differential
        evolution)
20
```

```
def LeastSquaresError(Params,x,T,Market_IV,
    RegularisationForPenalty):

    a,b,rho,m,sigma = Params

    Model_IV = Volfrom_w(Params,x,T)

    Error = np.sum((Model_IV - Market_IV)**2)

    Penalty = 0.0
    if b*(1+np.abs(rho)) > 4/T:
        Penalty += b*(1+np.abs(rho)) - 4/T

    return Error + (RegularisationForPenalty*Penalty)

# Calibrates the parameter values (with bounds)

def CalibrationTime(x,T,Market_IV,reruns,RegularisationForPenalty
    ):
    x_range = x.max() - x.min()
    bounds = [
        (0.0,0.5),
        (1e-6,2.0),
        (-0.999,0.999),
        (x.min() - x_range,x.max() + x_range),
        (1e-4,x_range)
    ]

    seed = range(reruns)

    BestParameters = None
    BestError = np.inf

    for s in seed:
        results = differential_evolution(
            LeastSquaresError,
            args=(x,T,Market_IV,RegularisationForPenalty),
            bounds=bounds,
            seed=s,
            tol=1e-10,
            polish=True
        )
        if results.fun < BestError:
```

```
            BestParameters = results.x
            BestError = results.fun

    return BestParameters

RegularisationForPenalty = 1e3

CalibratedPairs = {} # CalibratedPairs[tenor][pair] = params

for tenor in Tenors:
    CalibratedPairs[tenor] = {}
    T = T_by_Tenor[tenor]
    for Pair, Value in Market_Data[tenor].items():
        parameters = CalibrationTime(Value["x"],T,Value["IV"],
            reruns=5,RegularisationForPenalty=
            RegularisationForPenalty)
        CalibratedPairs[tenor][Pair] = parameters
    print(f"Calibrated {tenor}")

# Parameter table per tenor

for tenor in Tenors:
    ParameterTable = pd.DataFrame(CalibratedPairs[tenor],index=["
        a","b","rho","m","sigma"]).T
    print(f"\n{tenor} SVI Parameters:")
    print(ParameterTable)

# RMS Error per pair, per tenor

print()
for tenor in Tenors:
    T = T_by_Tenor[tenor]
    print(f"{tenor}:")
    for Pair2, Value2 in Market_Data[tenor].items():
        params = CalibratedPairs[tenor][Pair2]
        Calibrated_IV = Volfrom_w(params,Value2["x"],T)
        print(f"  {Pair2} RMS Error : ", np.sqrt(np.mean((
            Calibrated_IV - Value2["IV"])**2)))

names = ["EURUSD", "GBPUSD", "EURGBP"]

for tenor in Tenors:
    T = T_by_Tenor[tenor]
```

```
    fig, axes = plt.subplots(1,3,figsize=(20,5))
    fig.suptitle(f"SVI Calibration - {tenor}", fontsize=14)

    for i,name in enumerate(names):
        F_0 = Market_Data[tenor][name]["F_0"]
        K_Market = Market_Data[tenor][name]["K"]
        IV_Market = Market_Data[tenor][name]["IV"]
        K_grid = np.linspace(K_Market.min(),K_Market.max(),400)
        x_grid = np.log(K_grid/F_0)
        Parameters = CalibratedPairs[tenor][name]
        SVI_Fit = Volfrom_w(Parameters,x_grid,T)

        axes[i].plot(K_grid,SVI_Fit, "b-",label="SVI Fit")
        axes[i].plot(K_Market,IV_Market,"ro",label="Market IV")
        axes[i].set_xlabel("Strike K")
        axes[i].set_ylabel("Implied Volatility")
        axes[i].set_title(f"{name}")
        axes[i].legend()
    plt.show()
```

B.3 Density Plots

```
def CallPrice2(params,T,F_0,K):
    x = np.log(K/F_0)
    sigma = Volfrom_w(params,x,T)
    return BlackScholesEuropeanCallOption(sigma, T, F_0, K)

def CentralSecondDifference(params,T,F_0,K,h=1e-3):
    CKplush = CallPrice2(params,T,F_0,K+h)
    CKminush = CallPrice2(params,T,F_0,K-h)
    TwoCK = 2*CallPrice2(params,T,F_0,K)
    return (CKplush + CKminush - TwoCK)/h**2

# Half-width in standard deviations: std of the normalised rate is sigma_ATM * sqrt(T),
# so we size the plotting/quadrature window as a fixed number of std rather than a flat +/-10%.
def DomainForTenor(tenor, k_std=6.0):
    T = T_by_Tenor[tenor]
    atm_vols = [Market_Data[tenor][p]["IV"][2] for p in ["EURUSD","GBPUSD","EURGBP"]] # ATMF quote = IV at K=F_0
```

```
    std = max(atm_vols) * np.sqrt(T)
    half = k_std * std
    return max(0.05, 1.0 - half), 1.0 + half

names = ["EURUSD", "GBPUSD", "EURGBP"]

for tenor in Tenors:
    T = T_by_Tenor[tenor]
    Lower, Upper = DomainForTenor(tenor)                # per-tenor window
    fig, axes = plt.subplots(1,3,figsize=(20,5))
    fig.suptitle(f"Density Sanity Checks - {tenor}", fontsize=14)
    print(f"\n{tenor} Density Sanity Checks (4 s.f.):")

    for i, Pair in enumerate(names):
        F_0 = Market_Data[tenor][Pair]["F_0"]
        K_grid = np.linspace(Lower*F_0, Upper*F_0, 4000)  # was 0.9*F_0, 1.1*F_0
        Parameters = CalibratedPairs[tenor][Pair]

        Density = CentralSecondDifference(Parameters,T,F_0,K_grid)

        print(f"{Pair} : ")
        print(f"Total Mass = {np.trapz(Density,K_grid):.4f}\t\t Mean = {np.trapz(Density*K_grid,K_grid):.4f} Should be F_0 = {F_0:.4f}\t\t Density Minimum : {Density.min():.4f}")

        axes[i].plot(K_grid,Density,"g-")
        axes[i].set_title(f"{Pair} Density Plot")
        axes[i].set_xlabel("Strike K")
        axes[i].set_ylabel("Density")
    plt.show()
```

B.4 Normalised Density Plots

```
def BL_Density(tenor, name, k, Lower, Upper):
    Density = CentralSecondDifference(CalibratedPairs[tenor][name],T_by_Tenor[tenor],1.0,k)
    Density[(k<Lower) | (k>Upper)] = 0.0
    return Density
```

```
5
6  for tenor in Tenors:
7      Lower, Upper = DomainForTenor(tenor)                    # per-
           tenor window
8      fig, axes = plt.subplots(1,3,figsize=(20,5))
9      fig.suptitle(f"Normalised Density Plots - {tenor}", fontsize
           =14)
10
11     for i, Pair in enumerate(names):
12         K_grid2 = np.linspace(Lower, Upper, 4000)          # was
               0.9, 1.1
13         Density2 = BL_Density(tenor, Pair, K_grid2, Lower, Upper)
14
15         axes[i].plot(K_grid2,Density2,"g-")
16         axes[i].set_title(f"{Pair} Density Plot")
17         axes[i].set_xlabel("Normalised Strike K")
18         axes[i].set_ylabel("Density")
19     plt.savefig(f"Normalised_Density_Plots_{tenor}.png")
20     plt.show()
21
22 # Overlaid, per tenor
23
24 colours = ["g", "b", "r"]
25
26 for tenor in Tenors:
27     Lower, Upper = DomainForTenor(tenor)
28     fig, ax = plt.subplots(1,1,figsize=(10,6))
29
30     for i, Pair in enumerate(names):
31         K_grid2 = np.linspace(Lower, Upper, 4000)
32         Density2 = BL_Density(tenor, Pair, K_grid2, Lower, Upper)
33         ax.plot(K_grid2,Density2,color=colours[i],label=Pair)
34
35     ax.set_title(f"Normalised Density Plots (Overlaid) - {tenor}"
           )
36     ax.set_xlabel("Normalised Strike K")
37     ax.set_ylabel("Density")
38     ax.legend()
39     plt.show()
```

B.5 Sinkhorn

```
from numpy.polynomial.legendre import leggauss
from scipy.optimize import bisect, brentq
from scipy.interpolate import CubicSpline

# Composite Gauss-Legendre: split [a,b] into panels so nodes spread evenly across

def Gauss_Legendre_Setup(a, b, n_panels=100, n_per_panel=8):
    xi, wi = leggauss(n_per_panel)
    edges = np.linspace(a, b, n_panels + 1)
    nodes, weights = [], []
    for p in range(n_panels):
        lo, hi = edges[p], edges[p+1]
        nodes.append(0.5*(hi-lo)*xi + 0.5*(lo+hi))
        weights.append(0.5*(hi-lo)*wi)
    return np.concatenate(nodes), np.concatenate(weights)

N_z = 100

def RunSinkhornForTenor(tenor, n_iterations=40, k_std=6.0):

    Lower, Upper = DomainForTenor(tenor, k_std)          # per-tenor window

    x_nodes, x_weights = Gauss_Legendre_Setup(Lower,Upper)
    y_nodes, y_weights = x_nodes.copy(), x_weights.copy()

    ratioXY = x_nodes[:,None] / y_nodes[None,:]

    MuX = BL_Density(tenor, "EURUSD", x_nodes, Lower, Upper)
    MuY = BL_Density(tenor, "GBPUSD", y_nodes, Lower, Upper)

    # z-grid sized to the cross's own +/- k_std, clamped inside [Lower, Upper]
    T = T_by_Tenor[tenor]
    z_std = Market_Data[tenor]["EURGBP"]["IV"][2] * np.sqrt(T)
    z_lo = max(Lower, 1.0 - k_std*z_std)
    z_hi = min(Upper, 1.0 + k_std*z_std)
    z_nodes = np.linspace(z_lo, z_hi, N_z)
    MuZ = BL_Density(tenor, "EURGBP", z_nodes, Lower, Upper)

```

```
    u = np.zeros(len(x_nodes))
    v = np.zeros(len(y_nodes))
    w_values = np.zeros(N_z)

    for iteration in range(n_iterations):

        w_spline = CubicSpline(z_nodes,w_values)
        W = w_spline(np.clip(ratioXY,z_nodes[0],z_nodes[-1]))

        u_exponent = v[None,:] + y_nodes[None,:] * W
        u = -np.log(np.sum(y_weights[None,:] * np.exp(u_exponent)
            * MuY[None,:],axis=1))

        v_exponent = u[:,None] + y_nodes[None,:] * W
        v = -np.log(np.sum(x_weights[:,None] * np.exp(v_exponent)
            * MuX[:,None], axis=0))

        v_spline = CubicSpline(y_nodes,v)

        for i in range(N_z):
            z = z_nodes[i]
            xoverz = x_nodes/z
            v_xoverz = v_spline(np.clip(xoverz,y_nodes[0],y_nodes
                [-1]))
            MuY_xoverz = BL_Density(tenor, "GBPUSD", xoverz, Lower
                , Upper) # window threaded in

            def target(w_guess):
                return np.sum((np.exp(u + v_xoverz + xoverz*
                    w_guess) * (x_nodes**2/z**3) * MuX * MuY_xoverz
                     * x_weights)) - MuZ[i]

            low, high = -60.0,60.0

            if target(low) > 0:
                w_values[i] = low
            elif target(high) < 0:
                w_values[i] = high
            else:
                w_values[i] = bisect(target,low,high)

    # final u,v rebalance against the converged w, so the
        marginals are exact and mass = 1
```

```
    w_spline = CubicSpline(z_nodes,w_values)
    W = w_spline(np.clip(ratioXY,z_nodes[0],z_nodes[-1]))
    u = -np.log(np.sum(y_weights[None,:] * np.exp(v[None,:] +
        y_nodes[None,:]*W) * MuY[None,:], axis=1))
    v = -np.log(np.sum(x_weights[:,None] * np.exp(u[:,None] +
        y_nodes[None,:]*W) * MuX[:,None], axis=0))

    joint_exponent = u[:,None] + v[None,:] + y_nodes[None,:]*W
    joint_weights = x_weights[:,None] * y_weights[None,:]
    joint_density = np.exp(joint_exponent) * MuX[:,None] * MuY[
        None,:]
    joint_integral = np.sum(joint_density * joint_weights)

    recovered_MuX = np.sum(joint_density * y_weights[None,:],
        axis=1)
    recovered_MuY = np.sum(joint_density * x_weights[:,None],
        axis=0)

    frac_clipped = np.mean((ratioXY < z_nodes[0]) | (ratioXY >
        z_nodes[-1]))

    print(f"\n{tenor} Sinkhorn Results:")
    print(f" Domain = [{Lower:.4f}, {Upper:.4f}]")
    print(f" Joint Density mass = {joint_integral:.6f}")
    print(f" Max X-marginal error = {np.max(np.abs(recovered_MuX
        - MuX)):.6f}")
    print(f" Max Y-marginal error = {np.max(np.abs(recovered_MuY
        - MuY)):.6f}")
    print(f" Fraction of (x,y) pairs with clipped w = {
        frac_clipped:.2%}")

    return {
        "x_nodes": x_nodes, "x_weights": x_weights,
        "y_nodes": y_nodes, "y_weights": y_weights,
        "z_nodes": z_nodes,
        "u": u, "v": v, "w_values": w_values,
        "v_spline": v_spline, "w_spline": w_spline,
        "MuX": MuX, "MuY": MuY, "MuZ": MuZ,
        "joint_density": joint_density, "joint_weights":
            joint_weights,
        "joint_integral": joint_integral,
        "frac_clipped": frac_clipped
    }
```

```
108
109 SinkhornResults = {}
110 for tenor in Tenors:
111     SinkhornResults[tenor] = RunSinkhornForTenor(tenor)
```

B.6 Market Consistency Check

```
1  def implied_volatility(price, T, K):
2      intrinsic_value = max(1.0 - K, 0.0)
3      if price <= intrinsic_value + 1e-12:
4          return 0.0
5      def target2(vol):
6          model_price = BlackScholesEuropeanCallOption(vol, T, 1.0,
               K)
7          return model_price - price
8      return brentq(target2, 1e-6, 5.0)
9
10 def model_call(results, kind, K):
11     x_nodes = results["x_nodes"]; y_nodes = results["y_nodes"]
12     joint_weights = results["joint_weights"]; joint_density =
           results["joint_density"]
13     if kind == "X":
14         return np.sum(joint_weights * np.maximum(x_nodes[:, None]
               - K, 0.0) * joint_density)
15     elif kind == "Y":
16         return np.sum(joint_weights * np.maximum(y_nodes[None, :]
               - K, 0.0) * joint_density)
17     else:
18         return np.sum(joint_weights * y_nodes[None, :] *
19                       np.maximum(x_nodes[:, None] / y_nodes[None,
                             :] - K, 0.0) * joint_density)
20
21 pairs = [("EURUSD", "X"), ("GBPUSD", "Y"), ("EURGBP", "Z")]
22
23 all_errors = []                                  # every (model -
       market) error, across everything
24 errors_by_pair = {name: [] for name, kind in pairs} # errors
       collected per pair, across all tenors
25 RMSTable = pd.DataFrame(index=Tenors, columns=[p[0] for p in
       pairs] + ["All"], dtype=float)
26
```

```
for tenor in Tenors:
    T = T_by_Tenor[tenor]
    results = SinkhornResults[tenor]

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    fig.suptitle(f"Consistency Check - {tenor}", fontsize=14)
    print(f"\n{tenor} consistency check (model implied vol vs market):")

    tenor_errors = []                           # errors for this tenor (all three pairs)

    for i in range(3):
        name, kind = pairs[i]
        F_0 = Market_Data[tenor][name]["F_0"]
        strikes = Market_Data[tenor][name]["K"] / F_0
        market_iv = Market_Data[tenor][name]["IV"]

        pair_errors = []                        # errors for this pair at this tenor

        print(f" {name}:")
        for j in range(len(strikes)):
            K = strikes[j]
            iv_model = implied_volatility(model_call(results, kind, K), T, K)
            error = market_iv[j] - iv_model
            pair_errors.append(error)
            print(f"    K = {K:.4f}  model {iv_model:.5f} market {market_iv[j]:.5f} Error {error:.5f}")

        pair_errors = np.array(pair_errors)
        pair_rms = np.sqrt(np.mean(pair_errors**2))
        print(f"    -> {name} RMS = {pair_rms:.5f} ({pair_rms*1e4:.1f} bp)")

        RMSTable.loc[tenor, name] = pair_rms
        tenor_errors.extend(pair_errors)
        errors_by_pair[name].extend(pair_errors)
        all_errors.extend(pair_errors)

        K_grid2 = np.linspace(strikes.min(), strikes.max(), 50)
        model_iv = np.zeros(len(K_grid2))
```

```
        for j in range(len(K_grid2)):
            model_iv[j] = implied_volatility(model_call(results,
                kind, K_grid2[j]), T, K_grid2[j])

        axes[i].plot(K_grid2, model_iv * 100, "b-", label="model (
            from mu*)")
        axes[i].plot(strikes, market_iv * 100, "ro", label="market
            ")
        axes[i].set_title(name)
        axes[i].set_xlabel("normalised strike K / F_0")
        axes[i].set_ylabel("implied volatility (%)")
        axes[i].legend()
    plt.show()

    # RMS across all three pairs at this tenor
    tenor_errors = np.array(tenor_errors)
    tenor_rms = np.sqrt(np.mean(tenor_errors**2))
    RMSTable.loc[tenor, "All"] = tenor_rms
    print(f" {tenor} overall RMS = {tenor_rms:.5f} ({tenor_rms*1
        e4:.1f} bp)")

# Summary
print("\n" + "="*60)
print("RMS implied-vol error by tenor and pair (decimal vol):")
print(RMSTable.round(6))

print("\nRMS per pair (across all tenors):")
for name, kind in pairs:
    e = np.array(errors_by_pair[name])
    rms = np.sqrt(np.mean(e**2))
    print(f" {name}: {rms:.5f} ({rms*1e4:.1f} bp)")

all_errors = np.array(all_errors)
overall_rms = np.sqrt(np.mean(all_errors**2))
print(f"\nOverall RMS (everything): {overall_rms:.5f} ({
    overall_rms*1e4:.1f} bp)")
```

B.7 Sinkhorn Potential Functions, Pricing and Covariance Table

```
for tenor in Tenors:
    results = SinkhornResults[tenor]

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    fig.suptitle(f"Sinkhorn Potentials - {tenor}", fontsize=14)

    axes[0].plot(results["x_nodes"], results["u"], "b-")
    axes[0].set_title("u(x)")
    axes[0].set_xlabel("x")

    axes[1].plot(results["y_nodes"], results["v"], "r-")
    axes[1].set_title("v(y)")
    axes[1].set_xlabel("y")

    axes[2].plot(results["z_nodes"], results["w_values"], "g-")
    axes[2].set_title("w(z)")
    axes[2].set_xlabel("z = x / y")

    plt.show()

    ExoticPrices = {}
CorrTable = pd.DataFrame(index=Tenors, columns=["Var_X","Var_Y","
    Cov_XY","Corr_XY"], dtype=float)

for tenor in Tenors:
    results = SinkhornResults[tenor]
    x_nodes = results["x_nodes"]; y_nodes = results["y_nodes"]
    joint_weights = results["joint_weights"]; joint_density =
        results["joint_density"]

    X = x_nodes[:, None]
    Y = y_nodes[None, :]

    basket   = np.maximum(0.5*(X + Y) - 1.0, 0.0)
    quanto   = np.maximum(X/Y - 1.0, 0.0)
    best_of  = np.maximum(np.maximum(X, Y) - 1.0, 0.0)
    quadratic = (X - Y)**2

    price_basket = np.sum(joint_weights * basket * joint_density)
    price_quanto = np.sum(joint_weights * quanto * joint_density)
    price_best  = np.sum(joint_weights * best_of * joint_density)
    price_quad  = np.sum(joint_weights * quadratic *
        joint_density)
```

```
41
42      ExoticPrices[tenor] = {
43          "basket": price_basket,
44          "quanto": price_quanto,
45          "best_of": price_best,
46          "quadratic": price_quad
47      }
48
49      EX  = np.sum(joint_weights * X * joint_density)
50      EY  = np.sum(joint_weights * Y * joint_density)
51      VarX = np.sum(joint_weights * (X - EX)**2 * joint_density)
52      VarY = np.sum(joint_weights * (Y - EY)**2 * joint_density)
53      CovXY = np.sum(joint_weights * (X - EX) * (Y - EY) *
            joint_density) # exact, no mean assumption
54      CorrXY = CovXY / np.sqrt(VarX * VarY)
55      CorrTable.loc[tenor] = [VarX, VarY, CovXY, CorrXY]
56
57      print(f"\n{tenor} option prices:")
58      print(f" basket    E[(0.5(X+Y)-1)+] = {price_basket:.6f}")
59      print(f" quanto    E[(X/Y-1)+]     = {price_quanto:.6f}")
60      print(f" best-of   E[(max(X,Y)-1)+] = {price_best:.6f}")
61      print(f" quadratic E[(X-Y)^2]      = {price_quad:.6e}")
62
63  ExoticPriceTable = pd.DataFrame(ExoticPrices).T
64  print("\nExotic option prices by maturity (EUR/USD/GBP triangle):
        ")
65  print(ExoticPriceTable)
66
67  print("\nImplied X-Y correlation by tenor:")
68  print(CorrTable.round(6))
```

Appendix C

Code: Multi-Heston Option 1: Short-Maturity Expansion

This appendix contains the full Python code underlying the method and results of the Multi-Heston: Short-Maturity Expansion approach.

C.1 Data

```
    # Data

import numpy as np

Tenors = {
    "1M": 1/12, "2M": 2/12, "3M": 3/12, "6M": 6/12, "9M": 9/12, "
        1Y": 1.0
}

Market_Data = {
    "EURUSD": {
        "1M": {"F_0": 1.1425, "K": np.array([1.1192, 1.1311,
            1.1425, 1.1531, 1.1630]), "IV": np.array([5.727,
            5.2845, 4.962, 4.8415, 4.897]) / 100},
        "2M": {"F_0": 1.1447, "K": np.array([1.1088, 1.1268,
            1.1447, 1.1601, 1.1754]), "IV": np.array([5.9395,
            5.4545, 5.113, 5.015, 5.113]) / 100},
        "3M": {"F_0": 1.1455, "K": np.array([1.1019, 1.1242,
            1.1455, 1.1655, 1.1849]), "IV": np.array([6.06, 5.547,
```

```
            5.199, 5.106, 5.235]) / 100},
        "6M": {"F_0": 1.1498, "K": np.array([1.0848, 1.1182,
            1.1498, 1.1799, 1.2107]), "IV": np.array([6.4665,
            5.854, 5.461, 5.4085, 5.6445]) / 100},
        "9M": {"F_0": 1.1539, "K": np.array([1.0689, 1.1127,
            1.1539, 1.1936, 1.2356]), "IV": np.array([6.966,
            6.244, 5.793, 5.7725, 6.105]) / 100},
        "1Y": {"F_0": 1.1579, "K": np.array([1.0548, 1.1081,
            1.1579, 1.2064, 1.2598]), "IV": np.array([7.376,
            6.542, 6.059, 6.0655, 6.501]) / 100},
    },
    "GBPUSD": {
        "1M": {"F_0": 1.3366, "K": np.array([1.3039, 1.3207,
            1.3366, 1.3511, 1.3642]), "IV": np.array([6.8935,
            6.323, 5.864, 5.6605, 5.6285]) / 100},
        "2M": {"F_0": 1.3367, "K": np.array([1.2868, 1.3125,
            1.3367, 1.3584, 1.3782]), "IV": np.array([7.21,
            6.5565, 6.0345, 5.792, 5.771]) / 100},
        "3M": {"F_0": 1.3367, "K": np.array([1.2739, 1.3065,
            1.3367, 1.3637, 1.3886]), "IV": np.array([7.525,
            6.762, 6.179, 5.9115, 5.896]) / 100},
        "6M": {"F_0": 1.3366, "K": np.array([1.2409, 1.2908,
            1.3366, 1.3774, 1.4164]), "IV": np.array([8.274,
            7.312, 6.591, 6.2945, 6.3385]) / 100},
        "9M": {"F_0": 1.3362, "K": np.array([1.2126, 1.2773,
            1.3362, 1.3886, 1.4396]), "IV": np.array([8.863,
            7.7375, 6.889, 6.5615, 6.642]) / 100},
        "1Y": {"F_0": 1.3356, "K": np.array([1.1881, 1.2656,
            1.3356, 1.3981, 1.4600]), "IV": np.array([9.287,
            8.013, 7.104, 6.7585, 6.866]) / 100},
    },
    "EURGBP": {
        "1M": {"F_0": 0.85455, "K": np.array([0.84327, 0.84875,
            0.85455, 0.86099, 0.86819]), "IV": np.array([3.685,
            3.5825, 3.656, 3.9485, 4.3755]) / 100},
        "2M": {"F_0": 0.85564, "K": np.array([0.83908, 0.84708,
            0.85564, 0.86544, 0.87679]), "IV": np.array([3.7105,
            3.6175, 3.733, 4.095, 4.607]) / 100},
        "3M": {"F_0": 0.85673, "K": np.array([0.83632, 0.84611,
            0.85673, 0.86915, 0.88385]), "IV": np.array([3.7605,
            3.6865, 3.8315, 4.2475, 4.8235]) / 100},
        "6M": {"F_0": 0.86005, "K": np.array([0.82975, 0.84405,
            0.86005, 0.87957, 0.90373]), "IV": np.array([3.972,
```

```
            3.938, 4.163, 4.6915, 5.422]) / 100},
        "9M": {"F_0": 0.86335, "K": np.array([0.82324, 0.84211,
            0.86335, 0.88981, 0.92349]), "IV": np.array([4.314,
            4.2715, 4.5295, 5.156, 6.008]) / 100},
        "1Y": {"F_0": 0.86677, "K": np.array([0.81836, 0.84105,
            0.86677, 0.89939, 0.94217]), "IV": np.array([4.521,
            4.4765, 4.766, 5.4635, 6.434]) / 100},
    },
}


MainPair1 = "EURUSD"
MainPair2 = "GBPUSD"
CrossPair = "EURGBP"


CrossIsProduct = False

Label = "EURUSDGBP - 22072026"
```

C.2 Parameterisation via Least-Squares and Pricing using Monte-Carlo Simulation

```


import pandas as pd
from scipy.optimize import least_squares

NumFactors = 2

# Use of De Col's Prop2's Volatility expansion - this is the
    equation
def ImpliedVarianceFromProp2(LoadingDiff, V_0, rho, xi,
    LogMoneyness):
    AtmVariance = 0.0
    for k in range(NumFactors):
        AtmVariance += LoadingDiff[k]**2 * V_0[k]
    SkewNumerator = 0.0
    for k in range(NumFactors):
```

```
        SkewNumerator += 0.5 * rho[k] * xi[k] * LoadingDiff[k]**3
            * V_0[k]
    SkewTerm = -SkewNumerator * LogMoneyness / AtmVariance
    CurvatureNumerator = 0.0
    for k in range(NumFactors):
        CurvatureNumerator += (1 + 2 * rho[k]**2) * xi[k]**2 *
            LoadingDiff[k]**4 * V_0[k]
    CrossTerm = 0.0
    for k in range(NumFactors):
        CrossTerm += rho[k] * xi[k] * LoadingDiff[k]**3 * V_0[k]
    CurvatureTerm = (LogMoneyness**2 / (12 * AtmVariance**2)) * (
        CurvatureNumerator - (15 / (4 * AtmVariance)) * CrossTerm
        **2)
    return AtmVariance + SkewTerm + CurvatureTerm


def UnpackParameters(Params):
    d = NumFactors
    Loading1 = Params[0 * d: 1 * d]
    Loading2 = Params[1 * d: 2 * d]
    V_0      = Params[2 * d: 3 * d]
    rho      = Params[3 * d: 4 * d]
    xi       = Params[4 * d: 5 * d]
    return Loading1, Loading2, V_0, rho, xi


def LoadingDifferences(Loading1, Loading2):
    # cross pair's loading adds if it's a product of the two
        mains, subtracts if its a ratio
    CrossLoading = Loading1 + Loading2 if CrossIsProduct else
        Loading1 - Loading2
    return {
        MainPair1: Loading1,
        MainPair2: Loading2,
        CrossPair: CrossLoading,
    }


def Residuals(Params, TenorData):
    Loading1, Loading2, V_0, rho, xi = UnpackParameters(Params)
    Diffs = LoadingDifferences(Loading1, Loading2)
    AllResiduals = []
    for Pair, LoadingDiff in Diffs.items():
```

```
        Data = TenorData[Pair]
        LogMoneyness = np.log(Data["F_0"] / Data["K"])
        ModelVariance = ImpliedVarianceFromProp2(LoadingDiff, V_0,
            rho, xi, LogMoneyness)
        ModelVol = np.sqrt(np.clip(ModelVariance, 1e-8, None))
        AllResiduals.append(Data["IV"] - ModelVol)
    return np.concatenate(AllResiduals)


d = NumFactors
LowerBounds = np.concatenate([[-3.0]*d, [-3.0]*d, [1e-5]*d,
    [-0.999]*d, [1e-5]*d])
UpperBounds = np.concatenate([[3.0]*d, [3.0]*d, [1.0]*d, [0.999]*
    d, [5.0]*d])


def CalibrateOneTenor(TenorData, reruns=15):
    BestParameters = None
    BestError = np.inf
    for s in range(reruns):
        rng = np.random.default_rng(s)
        Params0 = np.concatenate([
            rng.uniform(0.2, 1.6, 2*d),
            rng.uniform(0.01, 0.08, d),
            rng.uniform(-0.5, 0.5, d),
            rng.uniform(0.2, 1.5, d),
        ])
        results = least_squares(Residuals, Params0, args=(
            TenorData,), bounds=(LowerBounds, UpperBounds))
        if results.cost < BestError:
            BestParameters = results.x
            BestError = results.cost
    return BestParameters


NumPaths = 200_000
NumSteps = 100

#Risk-Neutral SDE used

def SimulateFXPaths(NumPaths, NumSteps, T, xi, rho, LoadingX,
    LoadingY, V_0):
    dt = T / NumSteps
```

```
    sqrt_dt = np.sqrt(dt)
    TotalPaths = 2 * NumPaths

    V = np.tile(V_0, (TotalPaths, 1)).astype(float)
    lnX = np.zeros(TotalPaths)
    lnY = np.zeros(TotalPaths)

    for step in range(NumSteps):
        Half = NumPaths
        W_NormalHalf = np.random.standard_normal((Half, NumFactors
            ))
        Z_NormalHalf = np.random.standard_normal((Half, NumFactors
            ))
        W_Normal = np.vstack([W_NormalHalf, -W_NormalHalf])
        Z_Normal = np.vstack([Z_NormalHalf, -Z_NormalHalf])

        TotalVarianceX = np.zeros(TotalPaths)
        TotalVarianceY = np.zeros(TotalPaths)
        DiffusionX = np.zeros(TotalPaths)
        DiffusionY = np.zeros(TotalPaths)

        for k in range(NumFactors):
            SqrtV_k = np.sqrt(np.maximum(V[:, k], 0.0))
            DeltaV_k = xi[k] * SqrtV_k * sqrt_dt * W_Normal[:, k]
            V[:, k] = np.maximum(V[:, k] + DeltaV_k, 0.0)
            DeltaZ_k = sqrt_dt * (rho[k] * W_Normal[:, k] + np.
                sqrt(1 - rho[k]**2) * Z_Normal[:, k])
            TotalVarianceX += LoadingX[k]**2 * SqrtV_k**2
            TotalVarianceY += LoadingY[k]**2 * SqrtV_k**2
            DiffusionX += LoadingX[k] * SqrtV_k * DeltaZ_k
            DiffusionY += LoadingY[k] * SqrtV_k * DeltaZ_k

        lnX += -0.5 * TotalVarianceX * dt + DiffusionX
        lnY += -0.5 * TotalVarianceY * dt + DiffusionY

    return np.exp(lnX), np.exp(lnY)


# Action

np.random.seed(1)
Rows = []
ParameterRows = []
```

```
CalibratedByTenor = {}

for Tenor, T in Tenors.items():
    TenorData = {Pair: Market_Data[Pair][Tenor] for Pair in
        Market_Data}

    CalibratedParams = CalibrateOneTenor(TenorData)
    CalibratedByTenor[Tenor] = CalibratedParams
    Loading1, Loading2, V_0, rho, xi = UnpackParameters(
        CalibratedParams)
    RMSError = np.sqrt(np.mean(Residuals(CalibratedParams,
        TenorData)**2))

    X, Y = SimulateFXPaths(NumPaths, NumSteps, T, xi, rho,
        Loading1, Loading2, V_0)

    BasketPrice   = np.maximum(0.5 * (X + Y) - 1, 0).mean()
    QuantoPrice   = np.maximum(X / Y - 1, 0).mean()
    BestOfPrice   = np.maximum(np.maximum(X, Y) - 1, 0).mean()
    QuadraticPrice = ((X - Y) ** 2).mean()

    Rows.append({
        "Tenor": Tenor,
        "RMS Vol Error": RMSError,
        "E((1/2(X+Y)-1)+)": BasketPrice,
        "E((X/Y-1)+)": QuantoPrice,
        "E((max(X,Y)-1)+)": BestOfPrice,
        "E((X-Y)^2)": QuadraticPrice,
    })

    ParameterRows.append({
        "Tenor": Tenor,
        "Loading1_1": Loading1[0], "Loading1_2": Loading1[1],
        "Loading2_1": Loading2[0], "Loading2_2": Loading2[1],
        "V_0_1": V_0[0], "V_0_2": V_0[1],
        "rho_1": rho[0], "rho_2": rho[1],
        "xi_1": xi[0], "xi_2": xi[1],
    })

PriceTable = pd.DataFrame(Rows).set_index("Tenor")
print(Label)
print(PriceTable)

```

```
ParameterTable = pd.DataFrame(ParameterRows).set_index("Tenor")
print()
print(f"{Label} - Calibrated Parameters ({MainPair1}=Loading1, {MainPair2}=Loading2)")
print(ParameterTable)
```

C.3 Calibration Fit Plots : Prop2 vs Market Data

```

import matplotlib
import matplotlib.pyplot as plt


DeltaLabels = ["10DP", "25DP", "ATMF", "25DC", "10DC"]


def PlotSmiles(Pair):
    fig, axes = plt.subplots(2, 3, figsize=(15, 8))
    axes = axes.flatten()
    for ax, (Tenor, Data) in zip(axes, Market_Data[Pair].items()):
        Loading1, Loading2, V_0, rho, xi = UnpackParameters(CalibratedByTenor[Tenor])
        LoadingDiff = LoadingDifferences(Loading1, Loading2)[Pair]
        LogMoneyness = np.log(Data["F_0"] / Data["K"])
        ModelVariance = ImpliedVarianceFromProp2(LoadingDiff, V_0, rho, xi, LogMoneyness)
        ModelIV = np.sqrt(np.clip(ModelVariance, 1e-8, None))

        ax.plot(DeltaLabels, Data["IV"] * 100, "x", markersize=10, color="black", label="Market")
        ax.plot(DeltaLabels, ModelIV * 100, "o", markersize=8, markerfacecolor="none", color="blue", label="Model")
        ax.set_title(f"{Pair} {Tenor}")
        ax.set_ylabel("Implied Vol (%)")
        ax.legend(fontsize=8)

    plt.suptitle(f"{Pair} - Market vs Model Expansion Fit Implied Volatility ({Label})")
```

```
    plt.tight_layout()
    plt.show()


for Pair in [MainPair1, MainPair2, CrossPair]:
    PlotSmiles(Pair)
```

C.4 Market Vs Model (Using Characteristic Function)

```
import matplotlib
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import brentq

NumQuadNodes = 64
QuadUpperLimit = 100.0
GL_x, GL_w = np.polynomial.legendre.leggauss(NumQuadNodes)
QuadNodes = 0.5 * QuadUpperLimit * (GL_x + 1.0)
QuadWeights = 0.5 * QuadUpperLimit * GL_w


def CharacteristicFunction(u, T, kappa, theta, xi, rho,
    LoadingDiff, V_0):
    LogG = 0.0
    for k in range(NumFactors):
        omega = 1j * u
        Bracket_k = -kappa[k] + omega * LoadingDiff[k] * rho[k] *
            xi[k]
        Delta_k = Bracket_k**2 - xi[k]**2 * (omega**2 - omega) *
            LoadingDiff[k]**2
        SqrtDelta_k = np.sqrt(Delta_k)
        LambdaPlus_k = (Bracket_k + SqrtDelta_k) / 2
        LambdaMinus_k = (Bracket_k - SqrtDelta_k) / 2
        g_k = LambdaPlus_k / LambdaMinus_k
        Exp_k = np.exp(-SqrtDelta_k * T)
        B_k = ((omega**2 - omega) / 2) * LoadingDiff[k]**2 * (1 -
            Exp_k) / (LambdaPlus_k * Exp_k - LambdaMinus_k)
        A_k = (kappa[k] * theta[k] / xi[k]**2) * (-2 *
            LambdaPlus_k * T - 2 * np.log((1 - g_k * Exp_k) / (1 -
```

```
            g_k)))
        LogG = LogG + A_k + B_k * V_0[k]
    return np.exp(LogG)


def TrueCallPrice(F_0, K, T, xi, rho, LoadingDiff, V_0):
    kappa_zero = np.zeros(NumFactors)
    theta_placeholder = np.zeros(NumFactors) # value never
        matters when kappa=0
    u = QuadNodes
    Phi = CharacteristicFunction(u - 0.5j, T, kappa_zero,
        theta_placeholder, xi, rho, LoadingDiff, V_0)
    LogMoneyness = np.log(K / F_0)
    Integrand = (Phi.real * np.cos(u * LogMoneyness) + Phi.imag *
         np.sin(u * LogMoneyness)) / (u**2 + 0.25)
    Integral = np.sum(QuadWeights * Integrand)
    return max(F_0 - np.sqrt(F_0 * K) / np.pi * Integral, 1e-12)


def Black76CallOption(sigma, T, F_0, K):
    d1 = (np.log(F_0 / K) + 0.5 * sigma**2 * T) / (sigma * np.
        sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return F_0 * norm.cdf(d1) - K * norm.cdf(d2)


def TrueImpliedVol(F_0, K, T, xi, rho, LoadingDiff, V_0):
    price = TrueCallPrice(F_0, K, T, xi, rho, LoadingDiff, V_0)
    intrinsic = max(F_0 - K, 0.0)
    if price <= intrinsic + 1e-10:
        return np.nan
    try:
        return brentq(lambda s: Black76CallOption(s, T, F_0, K) -
            price, 1e-6, 5.0)
    except ValueError:
        return np.nan  # true model produced an impossible price
            given these parameters


# Plot

DeltaLabels = ["10DP", "25DP", "ATMF", "25DC", "10DC"]
```

```
62
63
64  def PlotTrueSmiles(Pair):
65      fig, axes = plt.subplots(2, 3, figsize=(15, 8))
66      axes = axes.flatten()
67      for ax, (Tenor, Data) in zip(axes, Market_Data[Pair].items())
            :
68          T = Tenors[Tenor]
69          Loading1, Loading2, V_0, rho, xi = UnpackParameters(
                CalibratedByTenor[Tenor])
70          LoadingDiff = LoadingDifferences(Loading1, Loading2)[Pair]
71
72          TrueIVs = np.array([
73              TrueImpliedVol(Data["F_0"], K, T, xi, rho, LoadingDiff
                    , V_0) for K in Data["K"]
74          ])
75
76          ax.plot(DeltaLabels, Data["IV"] * 100, "x", markersize=10,
                 color="black", label="Market")
77          ax.plot(DeltaLabels, TrueIVs * 100, "o", markersize=8,
                markerfacecolor="none",
78                  color="blue", label="Model")
79          ax.set_title(f"{Pair} {Tenor}")
80          ax.set_ylabel("Implied Vol (%)")
81          ax.legend(fontsize=8)
82
83      plt.suptitle(f"{Pair} - Market vs Model Implied Volatility ({
            Label})")
84      plt.tight_layout()
85      plt.savefig(f"TrueModel_{Pair}_SmileFit.png", dpi=150)
86      plt.show()
87
88
89  for Pair in [MainPair1, MainPair2, CrossPair]:
90      PlotTrueSmiles(Pair)
```

C.5 RMS Error Market vs Model

```
1  ResidualsByPairTenor = {}
2  for Tenor, T in Tenors.items():
```

```
    Loading1, Loading2, V_0, rho, xi = UnpackParameters(
        CalibratedByTenor[Tenor])
    Diffs = LoadingDifferences(Loading1, Loading2)
    for Pair, LoadingDiff in Diffs.items():
        Data = Market_Data[Pair][Tenor]
        TrueIVs = np.array([
            TrueImpliedVol(Data["F_0"], K, T, xi, rho, LoadingDiff
                , V_0) for K in Data["K"]
        ])
        ResidualsByPairTenor[(Pair, Tenor)] = Data["IV"] - TrueIVs


def RMS(Residuals):
    return np.sqrt(np.nanmean(np.concatenate(Residuals) ** 2))


# RMS per pair, per tenor

BreakdownRows = []
for Tenor in Tenors:
    RowData = {"Tenor": Tenor}
    for Pair in Market_Data:
        RowData[Pair] = RMS([ResidualsByPairTenor[(Pair, Tenor)]])
    BreakdownRows.append(RowData)

RMSByPairAndTenor = pd.DataFrame(BreakdownRows).set_index("Tenor"
    )

# RMS per tenor-

RMSByTenor = pd.Series(
    {Tenor: RMS([ResidualsByPairTenor[(Pair, Tenor)] for Pair in
        Market_Data]) for Tenor in Tenors},
    name="RMS Vol Error",
)
RMSByTenor.index.name = "Tenor"

# RMS per pair

RMSByPair = pd.Series(
    {Pair: RMS([ResidualsByPairTenor[(Pair, Tenor)] for Tenor in
        Tenors]) for Pair in Market_Data},
    name="RMS Vol Error",
```

```
)
RMSByPair.index.name = "Pair"

print(f"{Label} - RMS vs TRUE Model, by Pair and Tenor")
print(RMSByPairAndTenor)
print()
print(f"{Label} - RMS vs TRUE Model, by Tenor (pooled across
    pairs)")
print(RMSByTenor)
print()
print(f"{Label} - RMS vs TRUE Model, by Pair (pooled across
    tenors)")
print(RMSByPair)
```

Appendix D

Code: Multi-Heston Option 2: Multi-Tenor Calibration via characteristic function

This appendix contains the full Python code underlying the method and results of the Multi-Heston: Multi-Tenor Calibration via characteristic function (Option 2)

D.1 Data

```
    # Data

import numpy as np

Tenors = {
    "1M": 1/12, "2M": 2/12, "3M": 3/12, "6M": 6/12, "9M": 9/12, "
        1Y": 1.0
}

Market_Data = {
    "EURUSD": {
        "1M": {"F_0": 1.1425, "K": np.array([1.1192, 1.1311,
            1.1425, 1.1531, 1.1630]), "IV": np.array([5.727,
            5.2845, 4.962, 4.8415, 4.897]) / 100},
        "2M": {"F_0": 1.1447, "K": np.array([1.1088, 1.1268,
            1.1447, 1.1601, 1.1754]), "IV": np.array([5.9395,
```

```
            5.4545, 5.113, 5.015, 5.113]) / 100},
        "3M": {"F_0": 1.1455, "K": np.array([1.1019, 1.1242,
            1.1455, 1.1655, 1.1849]), "IV": np.array([6.06, 5.547,
             5.199, 5.106, 5.235]) / 100},
        "6M": {"F_0": 1.1498, "K": np.array([1.0848, 1.1182,
            1.1498, 1.1799, 1.2107]), "IV": np.array([6.4665,
            5.854, 5.461, 5.4085, 5.6445]) / 100},
        "9M": {"F_0": 1.1539, "K": np.array([1.0689, 1.1127,
            1.1539, 1.1936, 1.2356]), "IV": np.array([6.966,
            6.244, 5.793, 5.7725, 6.105]) / 100},
        "1Y": {"F_0": 1.1579, "K": np.array([1.0548, 1.1081,
            1.1579, 1.2064, 1.2598]), "IV": np.array([7.376,
            6.542, 6.059, 6.0655, 6.501]) / 100},
    },
    "GBPUSD": {
        "1M": {"F_0": 1.3366, "K": np.array([1.3039, 1.3207,
            1.3366, 1.3511, 1.3642]), "IV": np.array([6.8935,
            6.323, 5.864, 5.6605, 5.6285]) / 100},
        "2M": {"F_0": 1.3367, "K": np.array([1.2868, 1.3125,
            1.3367, 1.3584, 1.3782]), "IV": np.array([7.21,
            6.5565, 6.0345, 5.792, 5.771]) / 100},
        "3M": {"F_0": 1.3367, "K": np.array([1.2739, 1.3065,
            1.3367, 1.3637, 1.3886]), "IV": np.array([7.525,
            6.762, 6.179, 5.9115, 5.896]) / 100},
        "6M": {"F_0": 1.3366, "K": np.array([1.2409, 1.2908,
            1.3366, 1.3774, 1.4164]), "IV": np.array([8.274,
            7.312, 6.591, 6.2945, 6.3385]) / 100},
        "9M": {"F_0": 1.3362, "K": np.array([1.2126, 1.2773,
            1.3362, 1.3886, 1.4396]), "IV": np.array([8.863,
            7.7375, 6.889, 6.5615, 6.642]) / 100},
        "1Y": {"F_0": 1.3356, "K": np.array([1.1881, 1.2656,
            1.3356, 1.3981, 1.4600]), "IV": np.array([9.287,
            8.013, 7.104, 6.7585, 6.866]) / 100},
    },
    "EURGBP": {
        "1M": {"F_0": 0.85455, "K": np.array([0.84327, 0.84875,
            0.85455, 0.86099, 0.86819]), "IV": np.array([3.685,
            3.5825, 3.656, 3.9485, 4.3755]) / 100},
        "2M": {"F_0": 0.85564, "K": np.array([0.83908, 0.84708,
            0.85564, 0.86544, 0.87679]), "IV": np.array([3.7105,
            3.6175, 3.733, 4.095, 4.607]) / 100},
        "3M": {"F_0": 0.85673, "K": np.array([0.83632, 0.84611,
            0.85673, 0.86915, 0.88385]), "IV": np.array([3.7605,
```

```
            3.6865, 3.8315, 4.2475, 4.8235]) / 100},
        "6M": {"F_0": 0.86005, "K": np.array([0.82975, 0.84405,
            0.86005, 0.87957, 0.90373]), "IV": np.array([3.972,
            3.938, 4.163, 4.6915, 5.422]) / 100},
        "9M": {"F_0": 0.86335, "K": np.array([0.82324, 0.84211,
            0.86335, 0.88981, 0.92349]), "IV": np.array([4.314,
            4.2715, 4.5295, 5.156, 6.008]) / 100},
        "1Y": {"F_0": 0.86677, "K": np.array([0.81836, 0.84105,
            0.86677, 0.89939, 0.94217]), "IV": np.array([4.521,
            4.4765, 4.766, 5.4635, 6.434]) / 100},
    },
}


MainPair1 = "EURUSD"
MainPair2 = "GBPUSD"
CrossPair = "EURGBP"

CrossIsProduct = False

Label = "EURUSDGBP - 22072026"
```

D.2 Calibration via Characteristic Function

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import least_squares, brentq

NumFactors = 2 # number of shared CIR variance factors, d in the
    paper

# characteristic function -> Fourier price -> implied vol

def CharacteristicFunction(u, T, kappa, theta, xi, rho,
    LoadingDiff, V_0):
    LogG = 0.0
    for k in range(NumFactors):
        omega = 1j * u
        Bracket_k = -kappa[k] + omega * LoadingDiff[k] * rho[k] *
            xi[k]
```

```
        Delta_k = Bracket_k**2 - xi[k]**2 * (omega**2 - omega) *
            LoadingDiff[k]**2
        SqrtDelta_k = np.sqrt(Delta_k)
        LambdaPlus_k = (Bracket_k + SqrtDelta_k) / 2
        LambdaMinus_k = (Bracket_k - SqrtDelta_k) / 2
        g_k = LambdaPlus_k / LambdaMinus_k
        Exp_k = np.exp(-SqrtDelta_k * T)
        B_k = ((omega**2 - omega) / 2) * LoadingDiff[k]**2 * (1 -
            Exp_k) / (LambdaPlus_k * Exp_k - LambdaMinus_k)
        A_k = (kappa[k] * theta[k] / xi[k]**2) * (-2 *
            LambdaPlus_k * T - 2 * np.log((1 - g_k * Exp_k) / (1 -
             g_k)))
        LogG = LogG + A_k + B_k * V_0[k]
    return np.exp(LogG)


NumQuadNodes = 64
QuadUpperLimit = 100.0
GL_x, GL_w = np.polynomial.legendre.leggauss(NumQuadNodes)
QuadNodes = 0.5 * QuadUpperLimit * (GL_x + 1.0)
QuadWeights = 0.5 * QuadUpperLimit * GL_w


def CallPricesForSmile(F_0, K, T, kappa, theta, xi, rho,
    LoadingDiff, V_0):
    u = QuadNodes
    Phi = CharacteristicFunction(u - 0.5j, T, kappa, theta, xi,
        rho, LoadingDiff, V_0)

    Prices = np.empty(len(K))
    for i, Strike in enumerate(K):
        LogMoneyness = np.log(Strike / F_0)
        Integrand = (Phi.real * np.cos(u * LogMoneyness) + Phi.
            imag * np.sin(u * LogMoneyness)) / (u**2 + 0.25)
        Integral = np.sum(QuadWeights * Integrand)
        Prices[i] = max(F_0 - np.sqrt(F_0 * Strike) / np.pi *
            Integral, 1e-12)
    return Prices


def Black76CallOption(sigma, T, F_0, K):
    d1 = (np.log(F_0 / K) + 0.5 * sigma**2 * T) / (sigma * np.
        sqrt(T))
```

```
    d2 = d1 - sigma * np.sqrt(T)
    return F_0 * norm.cdf(d1) - K * norm.cdf(d2)


def ImpliedVolFromPrice(Price, F_0, K, T):
    Intrinsic = max(F_0 - K, 0.0)
    if Price <= Intrinsic + 1e-10:
        return np.nan
    return brentq(lambda sigma: Black76CallOption(sigma, T, F_0,
        K) - Price, 1e-6, 5.0)


# Parameters

def UnpackParameters(Params):
    d = NumFactors
    Loading1 = Params[0 * d: 1 * d]
    Loading2 = Params[1 * d: 2 * d]
    kappa    = Params[2 * d: 3 * d]
    theta    = Params[3 * d: 4 * d]
    xi       = Params[4 * d: 5 * d]
    rho      = Params[5 * d: 6 * d]
    V_0      = Params[6 * d: 7 * d]
    return Loading1, Loading2, kappa, theta, xi, rho, V_0


def LoadingDifferences(Loading1, Loading2):
    CrossLoading = Loading1 + Loading2 if CrossIsProduct else
        Loading1 - Loading2
    return {
        MainPair1: Loading1,
        MainPair2: Loading2,
        CrossPair: CrossLoading,
    }


# Calibration

def Residuals(Params):
    Loading1, Loading2, kappa, theta, xi, rho, V_0 =
        UnpackParameters(Params)
    Diffs = LoadingDifferences(Loading1, Loading2)

```

```
    AllResiduals = []
    for Pair, LoadingDiff in Diffs.items():
        for Tenor, Data in Market_Data[Pair].items():
            T = Tenors[Tenor]
            ModelPrices = CallPricesForSmile(Data["F_0"], Data["K"
                ], T, kappa, theta, xi, rho, LoadingDiff, V_0)
            for Strike, MarketIV, Price in zip(Data["K"], Data["IV
                "], ModelPrices):
                ModelIV = ImpliedVolFromPrice(Price, Data["F_0"],
                    Strike, T)
                if np.isnan(ModelIV):
                    AllResiduals.append(1.0)
                else:
                    AllResiduals.append(MarketIV - ModelIV)
    return np.array(AllResiduals)


d = NumFactors
LowerBounds = np.concatenate([[-3.0]*d, [-3.0]*d, [0.05]*d, [1e
    -4]*d, [0.05]*d, [-0.95]*d, [1e-4]*d])
UpperBounds = np.concatenate([[3.0]*d, [3.0]*d, [5.0]*d, [0.5]*d,
     [3.0]*d, [0.95]*d, [0.5]*d])

# Loadings' bounds (-3, 3) - arbitrary, just wide enough not to
    constrain
# kappa, xi floored at 0.05 (kept meaningfully away from zero)
# theta, V_0 floored at 1e-4 (variance can't be negative)
# kappa capped at 5.0, theta capped at 0.5, xi capped at 3.0


def CalibrationTime(reruns):
    BestParameters = None
    BestError = np.inf

    for s in range(reruns):
        rng = np.random.default_rng(s)
        Params0 = np.concatenate([
            rng.uniform(0.2, 1.2, 2*d),
            rng.uniform(0.5, 2.0, d),
            rng.uniform(0.01, 0.08, d),
            rng.uniform(0.3, 1.5, d),
            rng.uniform(-0.5, 0.5, d),
            rng.uniform(0.01, 0.08, d),
```

```
        ])
        results = least_squares(Residuals, Params0, bounds=(
            LowerBounds, UpperBounds))
        print(f" rerun {s+1}/{reruns}: cost = {results.cost:.6e}")
        if results.cost < BestError:
            BestParameters = results.x
            BestError = results.cost

    return BestParameters


# Run the calibration

CalibratedParams = CalibrationTime(reruns=3)
Loading1, Loading2, kappa, theta, xi, rho, V_0 = UnpackParameters
    (CalibratedParams)
Diffs = LoadingDifferences(Loading1, Loading2)

print()
print("RMS vol error =", np.sqrt(np.mean(Residuals(
    CalibratedParams)**2)))


NumPaths = 200_000
NumSteps = 100


def SimulateFXPaths(NumPaths, NumSteps, T, kappa, theta, xi, rho,
     LoadingX, LoadingY, V_0):
    dt = T / NumSteps
    sqrt_dt = np.sqrt(dt)
    TotalPaths = 2 * NumPaths

    V = np.tile(V_0, (TotalPaths, 1)).astype(float)
    lnX = np.zeros(TotalPaths)
    lnY = np.zeros(TotalPaths)

    for step in range(NumSteps):
        Half = NumPaths
        W_NormalHalf = np.random.standard_normal((Half, NumFactors
            ))
        Z_NormalHalf = np.random.standard_normal((Half, NumFactors
            ))
```

```
        W_Normal = np.vstack([W_NormalHalf, -W_NormalHalf])
        Z_Normal = np.vstack([Z_NormalHalf, -Z_NormalHalf])

        TotalVarianceX = np.zeros(TotalPaths)
        TotalVarianceY = np.zeros(TotalPaths)
        DiffusionX = np.zeros(TotalPaths)
        DiffusionY = np.zeros(TotalPaths)

        for k in range(NumFactors):
            SqrtV_k = np.sqrt(np.maximum(V[:, k], 0.0))

            MeanReversion_k = kappa[k] * (theta[k] - V[:, k]) * dt
            DeltaV_k = xi[k] * SqrtV_k * sqrt_dt * W_Normal[:, k]
            V[:, k] = np.maximum(V[:, k] + MeanReversion_k +
                DeltaV_k, 0.0)

            DeltaZ_k = sqrt_dt * (rho[k] * W_Normal[:, k] + np.
                sqrt(1 - rho[k]**2) * Z_Normal[:, k])

            TotalVarianceX += LoadingX[k]**2 * SqrtV_k**2
            TotalVarianceY += LoadingY[k]**2 * SqrtV_k**2
            DiffusionX += LoadingX[k] * SqrtV_k * DeltaZ_k
            DiffusionY += LoadingY[k] * SqrtV_k * DeltaZ_k

        lnX += -0.5 * TotalVarianceX * dt + DiffusionX
        lnY += -0.5 * TotalVarianceY * dt + DiffusionY

    return np.exp(lnX), np.exp(lnY)


np.random.seed(1)

PriceRows = []
for Tenor, T in Tenors.items():
    X, Y = SimulateFXPaths(NumPaths, NumSteps, T, kappa, theta,
        xi, rho, Loading1, Loading2, V_0)

    BasketPrice   = np.maximum(0.5 * (X + Y) - 1, 0).mean()
    QuantoPrice   = np.maximum(X / Y - 1, 0).mean()
    BestOfPrice   = np.maximum(np.maximum(X, Y) - 1, 0).mean()
    QuadraticPrice = ((X - Y) ** 2).mean()

    PriceRows.append({
```

```
        "Tenor": Tenor,
        "E((1/2(X+Y)-1)+)": BasketPrice,
        "E((X/Y-1)+)": QuantoPrice,
        "E((max(X,Y)-1)+)": BestOfPrice,
        "E((X-Y)^2)": QuadraticPrice,
    })

PriceTable = pd.DataFrame(PriceRows).set_index("Tenor")
print(Label)
print(PriceTable)
```

D.3 Market vs Model Implied Volatility

```
DeltaLabels = ["10DP", "25DP", "ATMF", "25DC", "10DC"]


def PlotSmiles(Pair, LoadingDiff):
    fig, axes = plt.subplots(2, 3, figsize=(15, 8))
    axes = axes.flatten()
    for ax, (Tenor, Data) in zip(axes, Market_Data[Pair].items()):
        T = Tenors[Tenor]
        ModelPrices = CallPricesForSmile(Data["F_0"], Data["K"], T, kappa, theta, xi, rho, LoadingDiff, V_0)
        ModelIV = np.array([ImpliedVolFromPrice(P, Data["F_0"], K, T) for K, P in zip(Data["K"], ModelPrices)])

        ax.plot(DeltaLabels, Data["IV"] * 100, "x", markersize=10, color="black", label="Market")
        ax.plot(DeltaLabels, ModelIV * 100, "o", markersize=8, markerfacecolor="none", color="blue", label="Model")
        ax.set_title(f"{Pair} {Tenor}")
        ax.set_ylabel("Implied Vol (%)")
        ax.legend(fontsize=8)

    plt.suptitle(f"{Pair} - Market vs Model Implied Volatility ({Label})")
    plt.tight_layout()
    plt.savefig(f"Model_{Pair}_SmileFitMTCF.png", dpi=150)
    plt.show()

```

```

for Pair, LoadingDiff in Diffs.items():
    PlotSmiles(Pair, LoadingDiff)
```

D.4 Parameter Table and RMS by Tenor

```
    ParameterTable = pd.DataFrame({
    "Loading1_1": [Loading1[0]], "Loading1_2": [Loading1[1]],
    "Loading2_1": [Loading2[0]], "Loading2_2": [Loading2[1]],
    "kappa_1": [kappa[0]], "kappa_2": [kappa[1]],
    "theta_1": [theta[0]], "theta_2": [theta[1]],
    "xi_1": [xi[0]], "xi_2": [xi[1]],
    "rho_1": [rho[0]], "rho_2": [rho[1]],
    "V_0_1": [V_0[0]], "V_0_2": [V_0[1]],
}, index=[Label])

print(f"{Label} - Calibrated Parameters ({MainPair1}=Loading1, {MainPair2}=Loading2)")
print(ParameterTable)

RMSRows = []
for Tenor, T in Tenors.items():
    TenorResiduals = []
    for Pair, LoadingDiff in Diffs.items():
        Data = Market_Data[Pair][Tenor]
        ModelPrices = CallPricesForSmile(Data["F_0"], Data["K"], T, kappa, theta, xi, rho, LoadingDiff, V_0)
        for Strike, MarketIV, Price in zip(Data["K"], Data["IV"], ModelPrices):
            ModelIV = ImpliedVolFromPrice(Price, Data["F_0"], Strike, T)
            if np.isnan(ModelIV):
                TenorResiduals.append(1.0)
            else:
                TenorResiduals.append(MarketIV - ModelIV)
    RMSRows.append({"Tenor": Tenor, "RMS Vol Error": np.sqrt(np.mean(np.array(TenorResiduals)**2))})

RMSTable = pd.DataFrame(RMSRows).set_index("Tenor")
print()
print(f"{Label} - RMS Vol Error by Tenor")
```

```
print(RMSTable)
```

D.5 RMS Breakdown

```
ResidualsByPairTenor = {}
for Pair, LoadingDiff in Diffs.items():
    for Tenor, T in Tenors.items():
        Data = Market_Data[Pair][Tenor]
        ModelPrices = CallPricesForSmile(Data["F_0"], Data["K"], T
            , kappa, theta, xi, rho, LoadingDiff, V_0)
        ModelIV = np.array([ImpliedVolFromPrice(P, Data["F_0"], K,
             T) for K, P in zip(Data["K"], ModelPrices)])
        ResidualsByPairTenor[(Pair, Tenor)] = Data["IV"] - ModelIV


# Diagnostic: confirms whether calibration's fixed-1.0-penalty
    and this
# table's nanmean treat failed inversions differently in practice
    , or
# whether it's a moot inconsistency because no failures occur at
    the
# calibrated optimum.
TotalPoints = sum(len(ResidualsByPairTenor[key]) for key in
    ResidualsByPairTenor)
TotalNaNs = sum(int(np.isnan(ResidualsByPairTenor[key]).sum())
    for key in ResidualsByPairTenor)

NaNRows = []
for Tenor in Tenors:
    RowData = {"Tenor": Tenor}
    for Pair in Market_Data:
        RowData[Pair] = int(np.isnan(ResidualsByPairTenor[(Pair,
            Tenor)]).sum())
    NaNRows.append(RowData)

NaNCountTable = pd.DataFrame(NaNRows).set_index("Tenor")

print(f"{Label} - NaN (failed inversion) count, by Pair and Tenor
    ")
print(NaNCountTable)
print(f"\nTotal NaNs: {TotalNaNs} out of {TotalPoints} points")
```

```
29
30 def RMS(Residuals):
31     return np.sqrt(np.nanmean(np.concatenate(Residuals) ** 2))
32
33
34 # RMS per pair, per tenor
35
36 BreakdownRows = []
37 for Tenor in Tenors:
38     RowData = {"Tenor": Tenor}
39     for Pair in Market_Data:
40         RowData[Pair] = RMS([ResidualsByPairTenor[(Pair, Tenor)]])
41     BreakdownRows.append(RowData)
42
43 RMSByPairAndTenor = pd.DataFrame(BreakdownRows).set_index("Tenor"
       )
44
45 # RMS per tenor
46
47 RMSByTenor = pd.Series(
48     {Tenor: RMS([ResidualsByPairTenor[(Pair, Tenor)] for Pair in
           Market_Data]) for Tenor in Tenors},
49     name="RMS Vol Error",
50 )
51 RMSByTenor.index.name = "Tenor"
52
53 #RMS per pair
54
55 RMSByPair = pd.Series(
56     {Pair: RMS([ResidualsByPairTenor[(Pair, Tenor)] for Tenor in
           Tenors]) for Pair in Market_Data},
57     name="RMS Vol Error",
58 )
59 RMSByPair.index.name = "Pair"
60
61 print(f"{Label} - RMS Vol Error by Pair and Tenor")
62 print(RMSByPairAndTenor)
63 print()
64 print(f"{Label} - RMS Vol Error by Tenor (pooled across pairs)")
65 print(RMSByTenor)
66 print()
67 print(f"{Label} - RMS Vol Error by Pair (pooled across tenors)")
68 print(RMSByPair)
```

Appendix E

Code Add-Ons

E.1 Orientation Residual from inverted correlation - Option 1

Code pasted at the end of the AUD/USD/JPY code block for option 1.

```
Rows_both = []
for Tenor in Tenors:
    TenorData = {Pair: Market_Data[Pair][Tenor] for Pair in
        Market_Data}
    CalibratedParams = CalibrateOneTenor(TenorData)
    Loading1, Loading2, V_0, rho, xi = UnpackParameters(
        CalibratedParams)
    T = Tenors[Tenor]
    X, Y = SimulateFXPaths(NumPaths, NumSteps, T, xi, rho,
        Loading1, Loading2, V_0)

    EX, EY = X.mean(), Y.mean()
    CovXY = np.mean((X-EX)*(Y-EY))
    CorrXY = CovXY/np.sqrt(X.var()*Y.var())

    Yinv = 1.0/Y
    EYinv = Yinv.mean()
    CovXYinv = np.mean((X-EX)*(Yinv-EYinv))
    CorrXYinv = CovXYinv/np.sqrt(X.var()*Yinv.var())

    Residual = CorrXY + CorrXYinv

```

```
    # rough MC standard error, treating antithetic pairs conservatively as
    # 200,000 independent units (not the full 400,000), via the standard
    # large-sample approximation SE(r) =~ (1-r^2)/sqrt(N-1)
    N_conservative = NumPaths # 200,000, NOT 400,000
    SE_direct = (1 - CorrXY**2) / np.sqrt(N_conservative - 1)
    SE_inv = (1 - CorrXYinv**2) / np.sqrt(N_conservative - 1)

    Rows_both.append({
        "Tenor": Tenor, "E[X]": EX, "E[Y]": EY,
        "Corr(AUDUSD, USDJPY)": CorrXY, "SE (direct)": SE_direct,
        "Corr(AUDUSD, JPYUSD)": CorrXYinv, "SE (inverted)": SE_inv,
        "Residual (sum)": Residual,
    })

import pandas as pd
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 220)
BothTable = pd.DataFrame(Rows_both).set_index("Tenor")
print(BothTable)
```

E.2 Orientation Residual from inverted correlation - Option 2

Code pasted at the end of the AUD/USD/JPY code block for option 2.

```
Rows_both = []
for Tenor, T in Tenors.items():
    X, Y = SimulateFXPaths(NumPaths, NumSteps, T, kappa, theta, xi, rho, Loading1, Loading2, V_0)

    EX, EY = X.mean(), Y.mean()
    CovXY = np.mean((X-EX)*(Y-EY))
    CorrXY = CovXY/np.sqrt(X.var()*Y.var())

    Yinv = 1.0/Y
    EYinv = Yinv.mean()
    CovXYinv = np.mean((X-EX)*(Yinv-EYinv))
    CorrXYinv = CovXYinv/np.sqrt(X.var()*Yinv.var())
```

```
13
14      Residual = CorrXY + CorrXYinv
15
16      N_conservative = NumPaths
17      SE_direct = (1 - CorrXY**2) / np.sqrt(N_conservative - 1)
18      SE_inv = (1 - CorrXYinv**2) / np.sqrt(N_conservative - 1)
19
20      Rows_both.append({
21          "Tenor": Tenor, "E[X]": EX, "E[Y]": EY,
22          "Corr(AUDUSD, USDJPY)": CorrXY, "SE (direct)": SE_direct,
23          "Corr(AUDUSD, JPYUSD)": CorrXYinv, "SE (inverted)": SE_inv
                ,
24          "Residual (sum)": Residual,
25      })
26
27  pd.set_option("display.max_columns", None)
28  pd.set_option("display.width", 220)
29  BothTable = pd.DataFrame(Rows_both).set_index("Tenor")
30  print(BothTable)
```

E.3 Cross-Rate Normalised Density Comparison

This code block would be put at the end of a block where both the Sinkhorn and Multi-Heston Option 2 codes are written in the same file. (Simplest this way)

```
1  # Cross-rate density comparison: Sinkhorn vs Multi-Heston (from
       its own CF)
2
3
4  Tenor = "1M"                              # tenor to compare (can
       adjust this)
5  T = T_by_Tenor[Tenor]
6
7  # 1. Sinkhorn cross density: MuZ on its own z-grid (already
       computed)
8
9  Results = SinkhornResults[Tenor]
10 SinkhornZ = Results["z_nodes"]
```

```
SinkhornDensity = Results["MuZ"] / np.trapz(Results["MuZ"],
    SinkhornZ) # normalise to a density

# 2. Heston cross density: invert the calibrated characteristic
    function
#    Phi(u) = E[e^{i u ln Z}] for the cross loading, then f_lnZ
    -> f_Z.

def HestonCrossDensity(z_grid, T, UpperLimit=300.0, NumNodes
    =6000):
    u = np.linspace(1e-8, UpperLimit, NumNodes)
    du = u[1] - u[0]
    Phi = CharacteristicFunction(u, T, kappa, theta, xi, rho,
        Diffs[CrossPair], V_0)
    LogZ = np.log(z_grid)
    Integrand = np.real(np.exp(-1j * np.outer(LogZ, u)) * Phi[
        None, :])
    LogDensity = Integrand.sum(axis=1) * du / np.pi     # density
        of ln Z
    Density = np.clip(LogDensity / z_grid, 0.0, None)  # density
        of Z
    return Density

HestonZ = SinkhornZ.copy()                              # compare
    on the same grid
HestonDensity = HestonCrossDensity(HestonZ, T)
HestonDensity = HestonDensity / np.trapz(HestonDensity, HestonZ)
    # renormalise (u-truncation)

# 3. Sanity check: both standard deviations must bracket the
    market ATM cross vol

ATMCrossVol = Market_Data[CrossPair][Tenor]["IV"][2]   # decimal
    ATM vol

def StandardDeviationOf(Grid, Density):
    Density = Density / np.trapz(Density, Grid)
    Mean = np.trapz(Grid * Density, Grid)
    return np.sqrt(np.trapz((Grid - Mean)**2 * Density, Grid))

print(f"{CrossPair} {Tenor} cross density check")
print(f"market-implied std ~ sigma_ATM*sqrt(T) = {ATMCrossVol*np.
    sqrt(T):.4f}")
```

```
print(f"Sinkhorn std = {StandardDeviationOf(SinkhornZ, SinkhornDensity):.4f}")
print(f"Heston  std = {StandardDeviationOf(HestonZ, HestonDensity):.4f}")
print(f"(both should sit near the market value, if one is ~2-3x off it is misscaled)")

# 4. Quantify the tails

def Moments(Grid, Density):
    Density = np.clip(Density, 0, None)
    Density = Density / np.trapz(Density, Grid)
    Mean = np.trapz(Grid * Density, Grid)
    Variance = np.trapz((Grid - Mean)**2 * Density, Grid)
    StdDev = np.sqrt(Variance)
    Skew = np.trapz((Grid - Mean)**3 * Density, Grid) / Variance**1.5
    ExcessKurtosis = np.trapz((Grid - Mean)**4 * Density, Grid) / Variance**2 - 3.0
    UpperTailProb = np.trapz(Density[Grid > Mean + 3*StdDev], Grid[Grid > Mean + 3*StdDev])
    LowerTailProb = np.trapz(Density[Grid < Mean - 3*StdDev], Grid[Grid < Mean - 3*StdDev])
    return StdDev, Skew, ExcessKurtosis, UpperTailProb, LowerTailProb

print(f"\n{'method':14s}{'std':>8}{'skew':>8}{'exc.kurt':>10}{'P(>+3s)':>10}{'P(<-3s)':>10}")
for Name, Grid, Density in [("Sinkhorn", SinkhornZ, SinkhornDensity), ("Heston(Opt2)", HestonZ, HestonDensity)]:
    StdDev, Skew, ExcessKurtosis, UpperTailProb, LowerTailProb = Moments(Grid, Density)
    print(f"{Name:14s}{StdDev:8.4f}{Skew:8.3f}{ExcessKurtosis:10.3f}{UpperTailProb:10.4f}{LowerTailProb:10.4f}")

# 5. Plot: overlay and upper-tail zoom

fig, (axFull, axTail) = plt.subplots(1, 2, figsize=(14, 5))

axFull.plot(SinkhornZ, SinkhornDensity, "b-", lw=1.8, label="Sinkhorn (entropic)")
axFull.plot(HestonZ, HestonDensity, "r-", lw=1.8, label="Multi-Heston (Opt 2, from CF)")
```

```
axFull.set_title(f"{CrossPair} cross density -- {Tenor}")
axFull.set_xlabel("normalised strike")
axFull.set_ylabel("density")
axFull.legend()

Mean = np.trapz(SinkhornZ * SinkhornDensity, SinkhornZ)
StdDev = StandardDeviationOf(SinkhornZ, SinkhornDensity)
TailMask = SinkhornZ > Mean + 1.8*StdDev

axTail.plot(SinkhornZ, SinkhornDensity, "b-", lw=1.8)
axTail.plot(HestonZ, HestonDensity, "r-", lw=1.8)
axTail.set_xlim(Mean + 1.8*StdDev, SinkhornZ[-1])
axTail.set_ylim(0, max(SinkhornDensity[TailMask].max(),
    HestonDensity[TailMask].max()) * 1.4)
axTail.set_title("upper tail (zoom)")
axTail.set_xlabel("normalised strike")

plt.tight_layout()
plt.savefig(f"{CrossPair}_{Tenor}_density_compare.png", dpi=150)
plt.show()
```

E.4 Residual Code for Sinkhorn (AUD/USD/JPY Triangle)

```
# Sinkhorn orientation residual on the AUD triangle
# r = Corr(X, USD/JPY) + Corr(X, JPY/USD), computed from the same
     joint density


def WeightedCorrelation(Values1, Values2, JointDensity,
    JointWeights):
    Mass = JointWeights * JointDensity               # probability
        mass per (i,j) cell
    Mean1 = np.sum(Mass * Values1)
    Mean2 = np.sum(Mass * Values2)
    Cov   = np.sum(Mass * (Values1 - Mean1) * (Values2 - Mean2))
    Var1  = np.sum(Mass * (Values1 - Mean1)**2)
    Var2  = np.sum(Mass * (Values2 - Mean2)**2)
    return Cov / np.sqrt(Var1 * Var2)

```

```
ResidualRows = []
for Tenor in Tenors:
    Results = SinkhornResults[Tenor]
    x_nodes = Results["x_nodes"]; y_nodes = Results["y_nodes"]
    joint_density = Results["joint_density"]; joint_weights =
        Results["joint_weights"]

    X_grid   = x_nodes[:, None] * np.ones_like(y_nodes[None, :])
        # AUDUSD, broadcast to (N,N)
    Y_grid   = np.ones_like(x_nodes[:, None]) * y_nodes[None, :]
        # JPYUSD (= Y)
    Yinv_grid = 1.0 / Y_grid                                    #
        USDJPY (= 1/Y)

    CorrJPYUSD = WeightedCorrelation(X_grid, Y_grid,
        joint_density, joint_weights) # Corr(X, JPY/USD)
    CorrUSDJPY = WeightedCorrelation(X_grid, Yinv_grid,
        joint_density, joint_weights) # Corr(X, USD/JPY)
    Residual  = CorrUSDJPY + CorrJPYUSD

    ResidualRows.append({
        "Tenor": Tenor,
        "Corr(X,JPYUSD)": CorrJPYUSD,
        "Corr(X,USDJPY)": CorrUSDJPY,
        "Residual r": Residual,
    })

ResidualTable = pd.DataFrame(ResidualRows).set_index("Tenor")
print("Sinkhorn orientation residual (AUD triangle):")
print(ResidualTable.round(4))
```